

Predicting Cache Evolution: The Growing Capacity and Latency Cost of CPU Caching

Hong Chan Lee Justin Kuethe

Department of Electrical and Computer Engineering, North Carolina State University
Homework I — Fall 2026

GitHub: <https://github.com/NovaCorz/CSC591-Project1>

Overleaf: <https://www.overleaf.com/read/gyrqznmvwsd#2b5b1c>

Abstract—Timing-only experiments on eight lab systems support L1-like and L2-like capacity brackets and conditional associativity estimates, together with a reproducible 64-byte spatial boundary. Matched independent-load tests demonstrate load overlap on every machine. Deeper-cache interpretation is limited by address translation, indexing, residency and interference: exact sharing domains, LLC associativity, global inclusion policy and isolated hardware miss penalties remain unresolved. The software timing classifier is threshold-sensitive and lacks hardware hit-rate validation. We present timing inferences, counter verification, chronological predictions, Hazel evaluation and two scoped cache laws, retaining the remaining experimental limitations.

I. Introduction

We study what timing alone reveals about cache organization on eight lab systems before consulting counters or published cache descriptions. Reproducible dependent-access distributions, page/spacing controls, spatial confirmations, conflict tests and cross-core pressure controls support the candidates in Table IV. Source and raw-data provenance preserve their traceability; specification values do not replace ambiguous observations.

Phase I requires evidence-based inference before hardware verification [1] (§5, §7 and §8.2). We retain supported estimates with their assumptions and report inconclusive tests separately from unperformed work. Lack of Phase-II confirmation does not invalidate a timing inference; missing residency or eviction controls can still prevent a particular interpretation.

II. Methodology Overview

The required sequence is blind timing inference, PMU/-documentation verification after a recorded Phase-I freeze, then lab-only prediction frozen before Hazel measurements. The Phase-I methodology uses timing without PMUs, cache-reporting interfaces or published cache geometry. Counter verification appears in Section VIII; chronological predictions and Hazel evaluation appear in Section XI.

II-A. Measurement Environment

Each full configuration contains 10^6 post-warm-up intervals; smoke measurements are excluded from

inference. Primary intervals batch N dependent loads; single-target reloads and classifier measurements are labelled separately. Table II lists primary/final batch sizes and additional-round quality counts. Native GCC flags are `-O0 -g -std=c11 -Wall -Wextra -Werror -fno-omit-frame-pointer`, with `-pthread` for the threaded extension. Baseline compilers are GCC 11.5.0 (20240719, Red Hat 11.5.0-14) on Sunbird, Thunderbird, Skylark and Artemisia; GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1) on Charnwood, Crux and Ookay; and GCC 12.3.0 (Ubuntu 12.3.0-1ubuntu1~22.04.3) on Upgrade. Later builds retain their own versions, commands and assembly.

On x86-64, intervals begin with `lfence; rdtsc; lfence` and end with `rdtscp; lfence`. AArch64 uses `dsb sy; isb; mrs cntvct_el0; isb`. We divide native TSC or CNTVCT ticks by the actual timed load count without converting through nominal GHz. These units are not instantaneous core cycles and cannot be ranked across ISAs. Timer/loop distributions are characterized without subtraction; every additional batch meets the recorded 5% timer-overhead limit.

`sched_setaffinity` precedes allocation and first touch. The selected core and all SMT siblings must pass two one-second windows at no more than 5% busy time. Workers recheck, reselect when needed and wait if no core qualifies; no reservation is assumed. Each attempt records logical CPU, physical core, socket, node and both activity snapshots. Pinned first touch encourages local placement without verifying it. Paired tests use another physical core on the same socket; this topology does not establish cache sharing.

II-B. Statistics, selection and controls

Saved statistics include count, mean, standard deviation, median, quartiles, 5th/95th percentiles, zero count and outlier count. Boxes show median/IQR and P_5 – P_{95} whiskers; all tails remain in the raw data. Outliers fall outside $[Q_1 - 1.5 IQR, Q_3 + 1.5 IQR]$, a different criterion from the whiskers. Correlated million-interval sequences do not make box widths or operational capacity brackets independent-sample confidence intervals.

Selection takes the first clean result chronologically, or the first eligible noisy result after clean attempts are

exhausted, never the lowest latency. Noisy data cannot establish a confident boundary. Repairs address noise or core mismatch regardless of timing direction, selecting the first clean same-core cohort and retaining every attempt. Randomized dependent traversal is primary; regular traversal, page policy and address spacing are controls. The register-only, load-only chase avoids inner-loop stack traffic; reload wrappers retain call/timer/sink overhead (Section VII).

III. Machine and Microarchitecture Research

Table I lists the CPU, ISA, microarchitecture and year in chronological order; Table II lists acquisition conditions and counts. Year denotes the public introduction of the represented processor generation or platform, rather than the machine’s installation date or a later SKU shipment. Thunderbird’s permitted model-line query returned no model name, so its Ampere identity comes from the assignment; Ampere’s Altra launch record supplies the platform taxonomy and year.

III-A. Pre-Experiment Hypotheses

The dated premeasurement record `docs/hypotheses.md` proposes that (1) effective capacity of at least one deeper residency class may increase across generations, with server/desktop variation; and (2) larger effective capacity may carry higher dependent-load latency, with nonmonotonicity and frequency, topology and placement confounds. Neither hypothesis has chronological or held-out confirmation.

IV. Phase I: Timing-Only Reverse Engineering

IV-A. Shared Benchmark Infrastructure

Figure 1 shows the shared address constructions and inference logic; Section VII prints all four native programs and both architecture-specific timer/dependency paths. The baseline builds and warms a seeded random cycle, sweeps footprint/spacing/count through command-line parameters and records every native interval. The extension adds aligned/page-controlled mappings, explicit address lists and a pressure thread. Separate programs test translation/backing effects and serialized versus independent chains. Plotting and generic format utilities remain in the reproduction package.

IV-B. Cache Levels and Hierarchy

Random cycles sweep 1 KiB–256 MiB by powers of two, then refine measured transitions. Fully huge-backed dense (8 B) and sparse (64 B) traversals test ordinary-page steps. BIC partitions identify effective timing classes rather than physical cache levels: Sunbird, Thunderbird, Crux and Upgrade have five classes at both spacings; the other hosts have five dense and four sparse classes. Extra classes and spacing-dependent boundaries remain part of the evidence.

IV-C. Cache Capacity

The L1-like departure is the first value more than 10% above the coherent small-footprint center. Deeper brackets combine dense/sparse timing classes. Refinement requires the earliest qualifying adjacent clean median increase above 15%, separated IQRs, the same core and required page backing; it does not bypass an unusable earlier edge. Figures 3 and 4 show boundary distributions and broad/noisy repeats; Table IV gives the final bounds.

Matched repeats place Upgrade’s replicated timing departure at 7.6875–8.125 MiB within its earlier 2–16 MiB envelope, without establishing physical LLC capacity. Sunbird, Charnwood and Crux retain their earlier bounds. Page backing, sparse occupancy, replacement and core changes affect the curves, so logical footprint need not equal cache-resident bytes.

The brackets summarize timing transitions; they do not imply a cache whose capacity varies within the bracket. Table IV preserves these observations alongside compatible way/set candidates. We do not round a transition to a power of two or substitute an assumed physical capacity.

IV-D. Cache Line Size

The `spatial` mode (Listings 1 and 2) links $a_i + \alpha$ to $a_i + \alpha + \delta$ and randomizes pair order. Sweeping δ and α distinguishes a shifted spatial boundary from alignment changes. The inference requires the smallest reproducible IQR-separated edge with at least two native interval ticks of separation after batch scaling, repeated across footprints and alignments. The earlier 10% criterion remains a sensitivity check.

All eight machines reproduce the 64 B boundary in eight of eight additional cohorts at two allocation scales and new seeds (Figs. 7 and 8). The additional 256 B pair spacing changes active-line count and set coverage across the boundary; larger allocations therefore do not establish lower-level service. Separate L2 and LLC physical line sizes remain unestablished.

IV-E. Associativity and Geometry

Conflict rings vary live addresses at fixed stride. An L1-like way candidate must recur across strides, with its smallest tested period yielding a capacity product compatible with the independent departure bracket. Derived sets, $S = C/(AB)$, provide a conditional cross-check because the period is not proven fundamental. Figures 9 and 10 show count sweeps and threshold neighborhoods.

Target-reload tests vary lower-candidate addresses under fixed upper pressure, checking stability through saturation, held-out seeds and two scrub patterns. They support conditional L2 geometry on six hosts. Crux’s stronger upper calibration has materially overlapping block ranges; Thunderbird lacks a supported stable upper scrub. Neither failed calibration supports isolated L2 measurements.

TABLE I: Lab-machine taxonomy sorted by processor-generation or platform introduction year; citations identify the primary vendor sources. *Primary contributor: Hong Chan Lee.*

Machine	CPU	Vendor	ISA	Microarch.	Year
Sunbird	2 × Xeon E5-2680 v3	Intel	x86-64	Haswell	2014 [15]
Charnwood	Core i7-6700	Intel	x86-64	Skylake	2015 [18]
Ookay	Core i7-7700	Intel	x86-64	Kaby Lake	2017 [19]
Upgrade	Core i7-8700	Intel	x86-64	Coffee Lake	2017 [20]
Crux	Core i7-9700	Intel	x86-64	Coffee Lake Refresh	2018 [21], [22]
Skylark	2 × AMD EPYC 7532	AMD	x86-64	Zen 2 / Rome	2019 [23], [24]
Thunderbird	Ampere Q80-30	Ampere	AArch64	Arm Neoverse N1	2020 [25]
Artemisia	2 × Xeon Gold 5420+	Intel	x86-64	Sapphire Rapids	2023 [14]

TABLE II: Assigned lab machines and verified additional-round acquisition. N_B/N_A : primary baseline/additional dependent-load batch sizes. Selected/noisy counts refer to configurations; full/flagged counts refer to attempts, with flagged counts including smoke. There are 36 additional smoke attempts in total. CPU identities are exact assignment labels, corroborated by permitted model strings except as stated for Thunderbird. *Primary contributor: Hong Chan Lee.*

Machine	CPU identity	ISA	N_B	N_A	Selected	Noisy	Full attempts	Flagged
sunbird	2 × Xeon E5-2680 v3	x86_64	512	512	132	0	141	9
thunderbird	Ampere Q80-30	aaarch64	1024	1024	1099	101	1379	381
skylark	2 × AMD EPYC 7532	x86_64	512	512	29	0	30	1
artemisia	2 × Xeon Gold 5420+	x86_64	256	256	29	0	30	1
charnwood	Core i7-6700	x86_64	512	512	107	33	197	123
crux	Core i7-9700	x86_64	512	512	509	2	523	16
ookay	Core i7-7700	x86_64	512	512	26	0	26	0
upgrade	Core i7-8700	x86_64	512	512	136	4	157	25

Mixed-ring tests time candidate and fixed scrub loads together. Independent repairs and confirmation reproduce conditional 4-way and 8-way candidates on Crux and Thunderbird, respectively, at two capacity-compatible strides (Fig. 11). Ring averages and target reloads remain distinct observables. Odd-multiple scrub addresses and candidate strides implement a virtual-index hypothesis without proving physical congruence; physical-way fields remain unset.

The empirical pressure-set search uses full sets, deletion and held-out seeds (Fig. 12). Without isolated lower-level residency, the hot/full-pressure slowdown cannot establish an LLC-specific eviction bound. This completed test leaves LLC ways and sets unknown on every host.

IV-F. Hit and Miss/Next-Level Latency

The dependent register loop prevents load parallelism from explaining low latency alone. Figure 14 contains all dense/sparse class representatives; Tables V and VI give sparse-class statistics. Reported increments subtract representative medians only on the same CPU. They do not isolate hardware miss penalties or establish physical L2/LLC hit times and first-load next-level misses.

IV-F1. Independent-load diagnostic

The diagnostic compares one dependency chain (D) with four mutually independent chains (I) on the same randomized 1 KiB, 128-node ring (Fig. 1a). Both execute four loads per loop iteration and 4,096 per timed interval after at least 100 ms of warm-up. Three seeds (5924101–5924103), each in D/I and I/D order, yield six same-core pairs per host with 10^6 intervals per arm. Listing 4 uses the shared timers and idle/noise controls.

All 48 pairs qualify, with lower median native ticks/load for independent chains in every pair (Fig. 13). Table III gives first-pair statistics and ratio variability across all six pairs. This demonstrates load overlap, without establishing physical hit latency, port count or another cache property. Retained wrapper costs, pointer resets between batches and between-run frequency/interference limit interpretation; intervals are not independent replications.

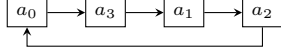
The supplement retains 180 full attempts (20 noisy) and 32 smoke attempts. After noise/core mismatch, Charnwood used fresh calibration cohorts preceded by 20 consecutive one-second idle windows, alongside the ordinary per-run checks. Accepted calibrations meet the 5% timer-fraction limit. Checks cover raw counts/hashes, statistics, block drift, affinity/idle evidence, selection and both native inner loops; all 48 tests pass. Failed calibrations and noisy attempts remain archived.

IV-G. Inclusion/Exclusion and Sharing

The pressure kernel (Listing 2) warms a target, has another idle physical core traverse a pressure region, then times the reload. A no-load helper provides the same-handshake control. Saved same-core pressure and hot-target controls remain available, although same-core eviction can directly displace an upper copy.

Figure 15 retains all six comparisons per host, including pair/quality failures. Sunbird and Artemisia each have six qualified comparisons, Crux two, Ookay five and the other hosts none. Even qualified pairs do not establish lower-only eviction: unchanged medians cannot prove non-inclusion, nor slowdowns global inclusivity. Inclusion/exclusion and exact sharing domains remain uncertain on every machine.

(a) Hierarchy, capacity, hit and next-class timing

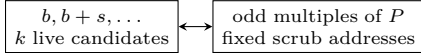


one randomized cycle within W bytes
time N dependent loads; record $(t_1 - t_0)/N$

1 KiB diagnostic: four independent starts
 $(a_{\pi_0}, a_{\pi_{32}}, a_{\pi_{64}}, a_{\pi_{96}})$
 $\Downarrow$ one load per register
 $(a_{\pi_1}, a_{\pi_{33}}, a_{\pi_{65}}, a_{\pi_{97}})$

Sweep W : powers of two, then dense refinements. Hold timer, batch and traversal fixed; compare page backing and spacing. Median/IQR steps and class distributions support candidate residency boundaries, not automatic physical-level assignments.

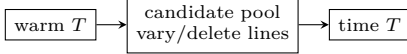
(c) Conflict geometry and fixed upper pressure



vary k , hold s and scrub fixed; seed the order
reload target (follow-up) or time entire ring (additional)

Repeat threshold neighborhoods using independent seed-s/patterns and test $C = As$, $S = C/(AB)$. Reload wrappers and mixed-ring averages are distinct observables. Fundamental physical congruence and level isolation remain assumptions.

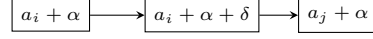
(e) Empirical pressure-set / LLC diagnostic



independent held-out hot/full/deletion controls
target reload includes timer/call/sink overhead

A pressure effect establishes interference under this construction. Without lower-level residency isolation it does not establish an LLC-specific way count or bound. Preserve failed calibration and broad distributions.

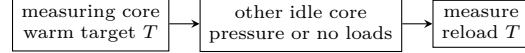
(b) Spatial granularity



paired dependent touches; random order of pairs
sweep offset δ ; repeat base shifts α

A shifted edge in offset, repeated across allocations, alignments and seeds, supports visible transfer granularity. Confirm below/at/above the edge; allocation size alone does not prove the supplying level.

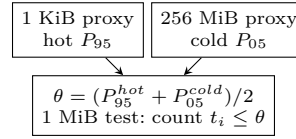
(d) Upper-copy survival under remote pressure



identical request/completion handshake in both arms
compare only qualified measuring/helper pairs

Compare hot/pressure distributions. Global inclusion requires demonstrated lower-only eviction and calibrated upper residency; neither remote traffic nor an unchanged reload median supplies that proof.

(f) Software timing classifier



Require separated proxy quantiles, otherwise return unresolved. Report the saved estimate, block range and threshold sensitivity. The matched PMU comparison appears in Section X.

Fig. 1: Concrete shared access patterns. Arrows denote dependencies or ordered experimental steps, not observed cache transfers. All measured values appear in the evidence figures; these diagrams contain no hypothetical cache answers. Phase-II verification appears in Section VIII, with the estimator comparison in Section X. *Primary contributor: Hong Chan Lee.*

IV-H. Final Inferred Cache Table

V. Software-Only Cache Hit-Rate Estimator

V-A. Calibration and classifier

The `software_metric` tests randomized dependent footprints of 1 KiB, 1 MiB and 256 MiB, with batch one and one million intervals each on the same logical CPU per machine. The smallest/largest footprints are empirical hot/cold proxies with unverified residency. When

$P_{95}^{hot} < P_5^{cold}$, the estimator (Listing 8) sets

$$\theta = \frac{1}{2}(P_{95}^{hot} + P_5^{cold}), \quad \hat{H} = \frac{1}{10^6} \sum_{i=1}^{10^6} \mathbf{1}[t_i \leq \theta].$$

Otherwise it returns unresolved. Figure 16 and Table VII report calibration and classifications.

V-B. Uncertainty and validation status

Sensitivity evaluates both calibration endpoints; block ranges cover 100 consecutive 10,000-access blocks. Both reveal uncertainty omitted by binomial intervals. Saved Wilson intervals assume independence and exclude calibration/classification error, so they are not calibrated

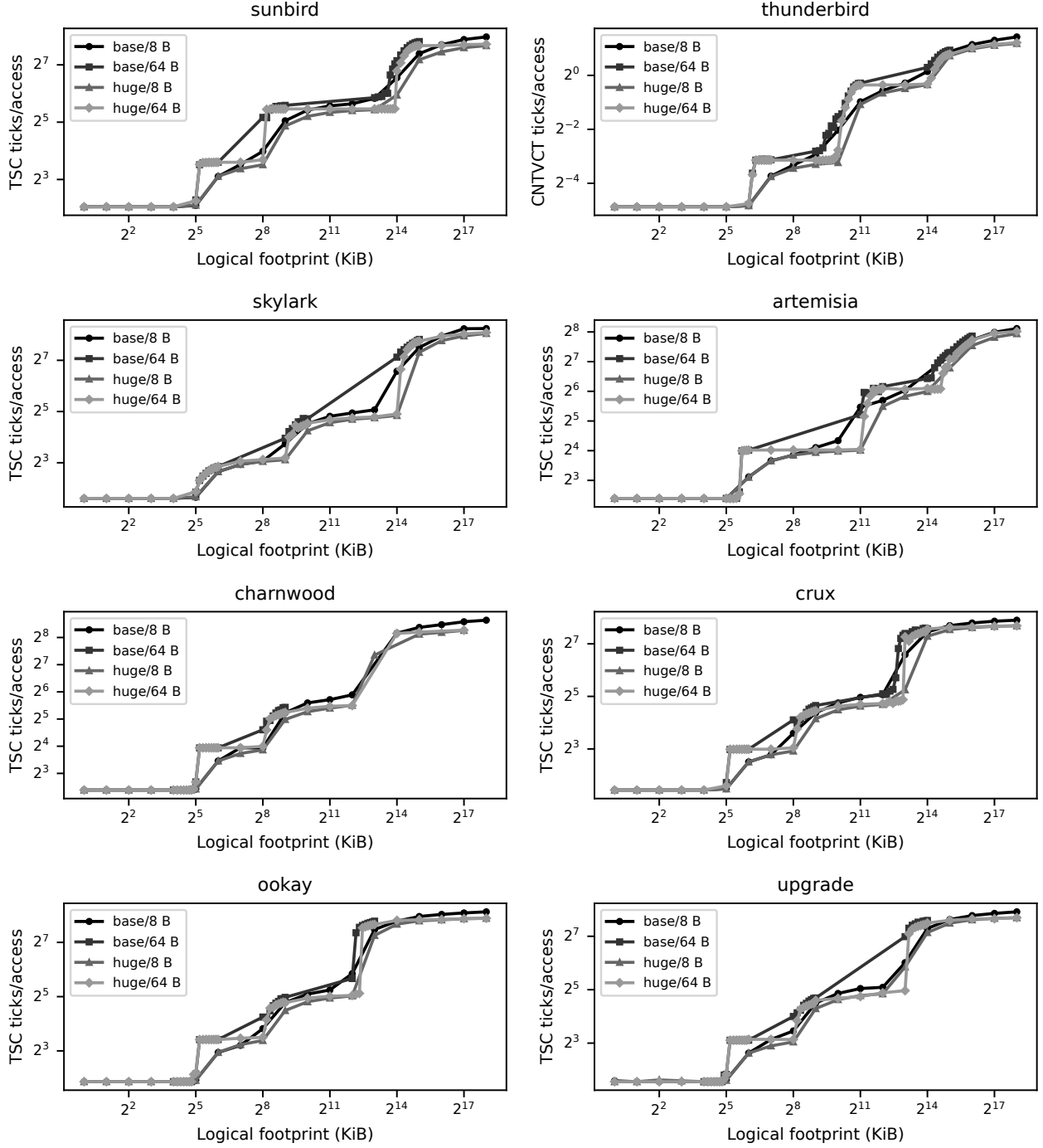


Fig. 2: All-machine capacity/hierarchy evidence: the saved clean median curves for ordinary/huge-page and dense/sparse dependent accesses (8/64 B spacing). The x axis is logical footprint, not total mapped backing; the y axis retains each architecture's native ticks/access. Repeated regions support effective residency classes; neither every local step nor a fitted class count proves a physical cache level. *Primary contributor: Hong Chan Lee.*

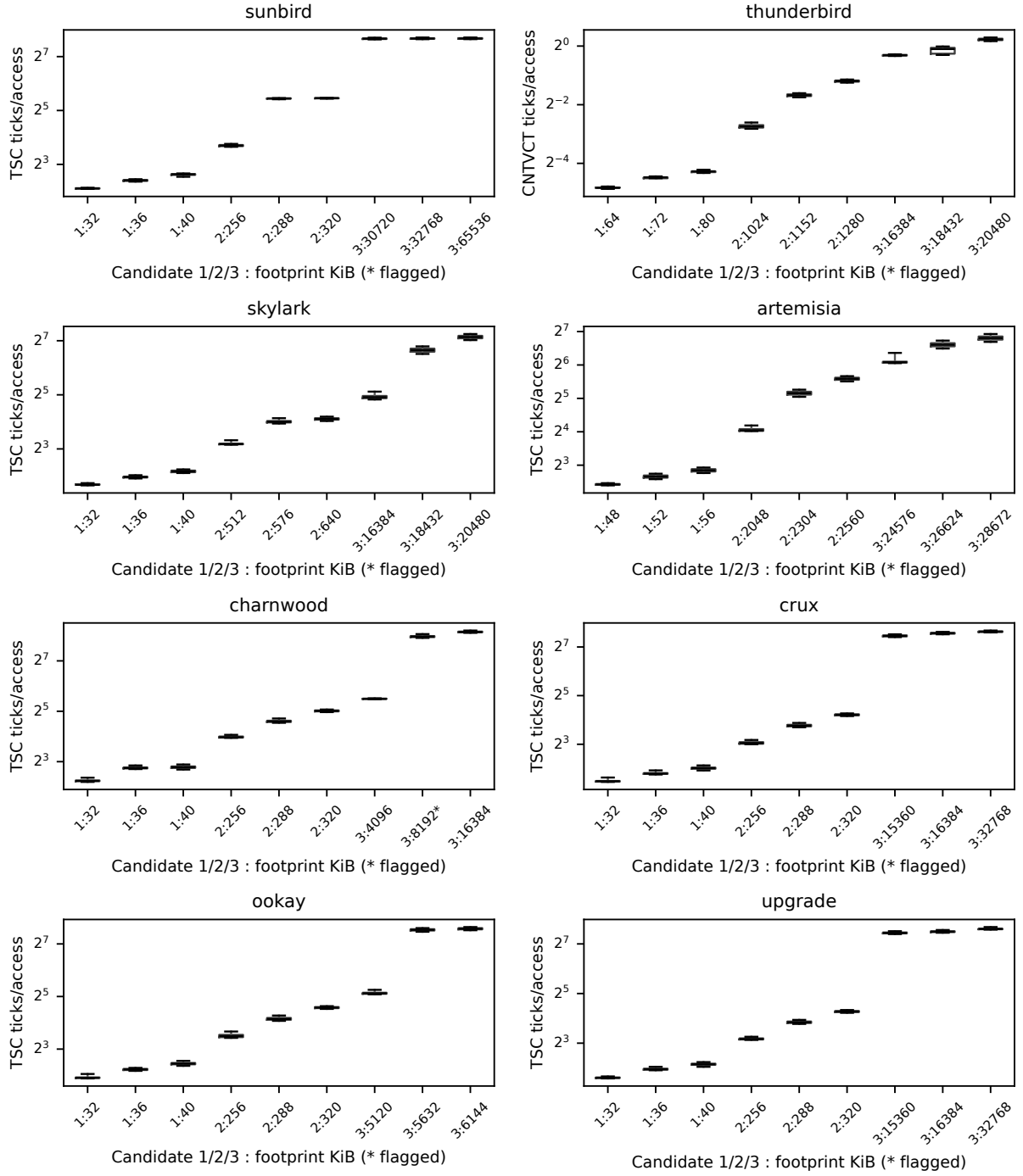


Fig. 3: Representative distributions around all three stored candidate boundaries on every machine. Prefix 1/2/3 identifies the candidate; remaining labels give KiB. L1 panels use three original points around the bracket's upper edge. Deeper panels use the saved refined edge pair plus the nearest same-refinement neighbor where available, otherwise three huge/sparse points near the stored upper edge. Stars retain flagged cases. Cross-core and broad-bracket neighborhoods are descriptive and do not strengthen the inference. *Primary contributor: Hong Chan Lee.*

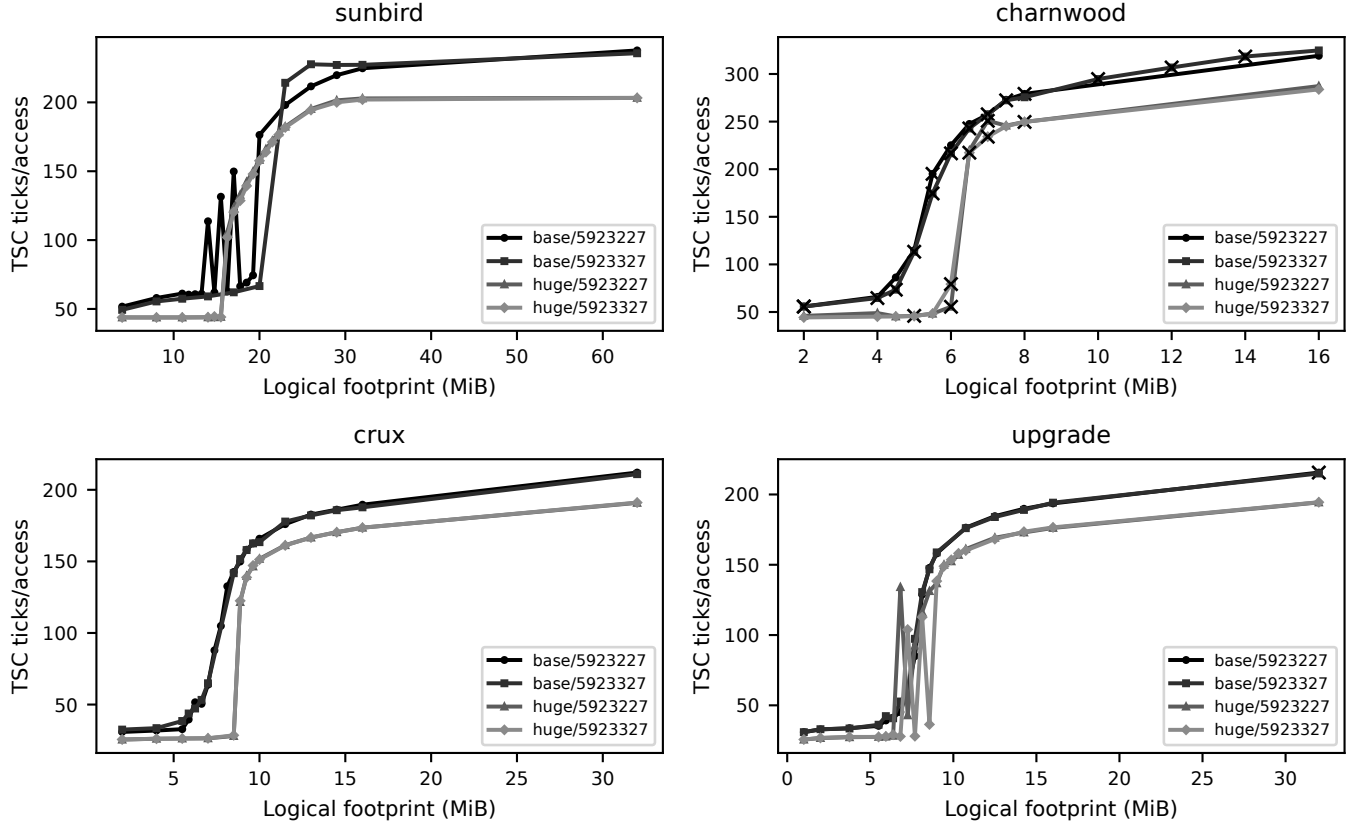


Fig. 4: Additional capacity repeats on the four hosts with broad prior envelopes. Both page policies and new seeds remain visible; crosses mark noisy selected distributions. Original curves are preserved even where matched comparisons were later acquired. Only the final qualified comparison evidence can narrow a bracket. *Primary contributor: Hong Chan Lee.*

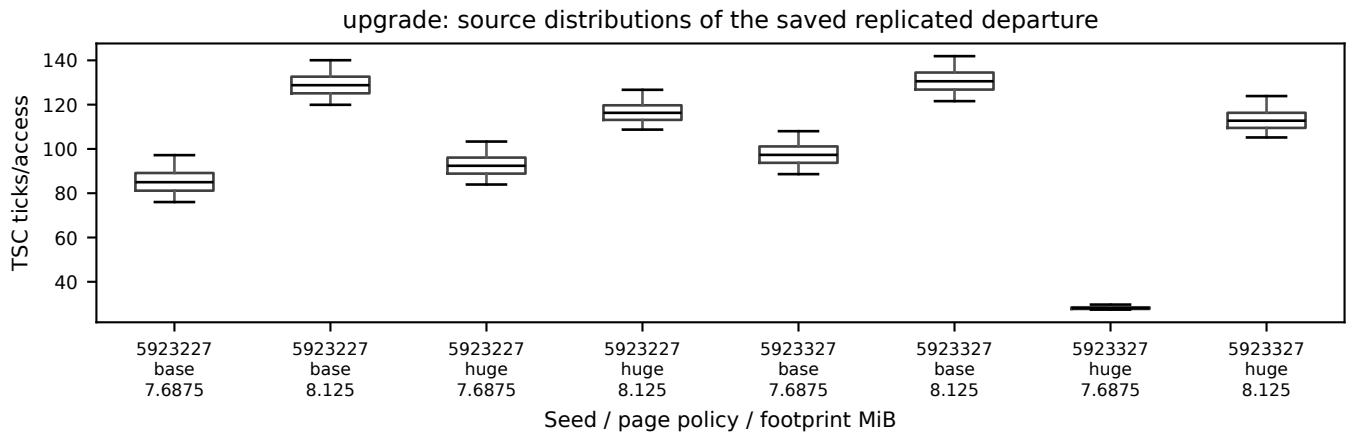


Fig. 5: Upgrade distributions referenced by the saved replicated departure. Labels retain the exact seed, page policy and logical MiB. The report does not rerun the edge detector or select a different crossing. *Primary contributor: Hong Chan Lee.*

TABLE III: Diagnostic statistics for the first planned pair (seed 5924101, D then I) on every host; each row has $n = 10^6$. Native units follow Fig. 13. O counts retained 1.5-IQR outliers. Mean/SD use seven significant digits; quantiles retain saved values. The final column gives the median and full range of D/I median ratios across all six qualified pairs per host, displayed to four decimals. Exact per-run statistics and all selected CPUs are in the supplement CSV/JSON. *Primary contributor: Hong Chan Lee.*

Host/kernel	CPU	Median	Mean SD	Q_1 Q_3	P_5 P_{95}	O	D/I median Range
sunbird/D	5	4.0185546875	4.028881 0.378863 1.023463	4.017578125 4.0185546875 1.0205078125	4.017578125 4.0185546875 1.0205078125	17388	3.9378 3.9340–3.9378
sunbird/I	5	1.0205078125	0.06628626 0.03361015	1.021484375 0.033447265625	1.021484375 0.033447265625	5422	3.9429 3.9429–3.9429
thunderbird/D	1	0.03369140625	0.001123925 0.008621403	0.03369140625 0.008544921875	0.03369140625 0.008544921875	573	3.9429–3.9429
thunderbird/I	1	0.008544921875	0.0007403292 2.943166	0.0087890625 2.9296875	0.0087890625 2.923828125	154	3.9062 3.9062–3.9062
skylark/D	3	2.9296875	0.1198537 0.7530178	2.9296875 0.75	2.9296875 0.744140625	257100	3.9062–3.9062
skylark/I	3	0.75	0.05056531 5.083964	0.75 5.0205078125	0.755859375 5.017578125	230442	3.9611 3.9596–3.9615
artemisia/D	28	5.0263671875	8.130415 1.272356	5.032265625 1.267578125	5.03515625 1.26611328125	10331	3.9611 3.9596–3.9615
artemisia/I	28	1.26904296875	0.07212944 3.731096	1.2705078125 3.65576171875	1.27197265625 3.64697265625	3019	3.9709 3.9664–4.0781
charnwood/D	2	3.74560546875	0.3277051 0.9257623	3.7529296875 0.91748046875	3.75830078125 0.916015625	7463	3.9664–4.0781
charnwood/I	2	0.91845703125	0.2472332 2.657436	0.919921875 2.605224609375	0.92138671875 2.600830078125	18723	3.9925 3.9855–3.9948
crux/D	2	2.61083984375	0.5111649 0.6628726	2.661376953125 0.652587890625	2.6669921875 0.651611328125	6404	3.9855–3.9948
crux/I	2	0.653564453125	0.2728895 3.54661	0.654541015625 3.49267578125	0.66796875 3.48779296875	150699	3.9850 3.9828–3.9855
ookay/D	3	3.49853515625	0.4712765 0.8914434	3.57421875 0.87646484375	3.58642578125 0.875	6828	3.9828–3.9855
ookay/I	3	0.87841796875	0.2271104 2.852876	0.89794921875 2.828857421875	0.900390625 2.826416015625	8902	3.9880 3.9839–3.9893
upgrade/D	4	2.832275390625	0.444568 0.7145234	2.835693359375 0.70947265625	2.838623046875 0.70849609375	38380	3.9839–3.9893
upgrade/I	4	0.71044921875	0.1852042	0.711181640625	0.712158203125	8666	

accuracy bounds. Timer quantization, wrapper overhead, mixed residency and sample correlation can dominate classification; the high point estimates require their broad sensitivity ranges. Section X reports the matched Phase-II software/PMU comparison and errors; the reference populations limit its interpretation as individual-load accuracy.

VI. Validation and Reproducibility

The additional round contains 2,483 full and 36 smoke attempts: 2,067 selected full configurations, 140 noisy selected distributions and 556 flagged attempts including smoke. Raw counts/hashes, recomputed statistics, affinity/idle evidence, source provenance and planned coverage pass, with no incomplete raw attempts. Earlier unsuccessful/flagged runs remain archived and cannot serve as clean evidence. These checks establish integrity and coverage rather than physical identification.

The saved release passes 42 functional tests. Curated-copy checks add seven archive-restore tests, recompute 47 representative full distributions (47 million intervals across eight machines and three rounds), and resolve 27,233 evidence references. This spot check supplements

full archived verification. Report generation reads verified inputs and changes layout only, without acquiring data or rerunning inference.

`report/latex-phase1/figure-sources.json` links figures to records and input hashes. Records preserve raw filenames/hashes, arguments, seeds, compiler/build details, environment, CPU/core/socket/node and `idle.json`. Source manifests distinguish deployed versions from final analysis code. The curated import manifest maps original paths to retained files; other paths resolve through the external archive overlay.

The report companion `evidence-statistics.csv` copies the saved count, mean, standard deviation, median, quartiles, 5th/95th percentiles and outlier count for every record referenced by an evidence figure. It includes commands, parameters and provenance references without recomputing statistics. `evidence-index.json` records their hashes and the inclusion-stage chronology. All unplotted attempts remain in the full archives.

The independent-load supplement uses `independent-sources.json`, verified records under `data_processed/independent-load/`, and the full raw archive `phase1-independent-load.zip`. Its manifest

TABLE IV: Final timing-only candidates. All capacities are operational brackets; ways/sets are conditional on the tested period and level interpretation. Sets use the saved jointly compatible candidate geometry, not an arbitrary value inside a capacity bracket; L2 set counts additionally assume a physical 64 B line. The 64 B entry is visible spatial granularity, not a separately measured per-level line size. U = unestablished. Native hit units are CNTVCT ticks/access for Thunderbird and TSC ticks/access otherwise. The reported L1-like medians are the stored candidate values; effective class distributions and descriptive next-class increments are in Tables V–VI. Sharing and inclusion/exclusion are U at every level. Upgrade additionally has the qualified 7.6875–8.125 MiB departure discussed in the text. *Primary contributor: Hong Chan Lee.*

Machine	Role	Capacity bracket	Line B	Ways	Sets	Hit ticks/access	Miss/next	Sharing	Incl.
sunbird	L1-like	32–36 KiB	64	8	64	4.1328125	U	U	U
	L2-like	256–288 KiB	U	8	512	U	U	U	U
	LLC-like	8–32 MiB	U	U	U	U	U	U	U
thunderbird	L1-like	64–72 KiB	64	4	256	0.0341796875	U	U	U
	L2-like	1024–1152 KiB	U	8	2048	U	U	U	U
	LLC-like	16–18 MiB	U	U	U	U	U	U	U
skylark	L1-like	32–36 KiB	64	8	64	3.046875	U	U	U
	L2-like	512–576 KiB	U	8	1024	U	U	U	U
	LLC-like	16–18 MiB	U	U	U	U	U	U	U
artemisia	L1-like	48–52 KiB	64	12	64	5.25	U	U	U
	L2-like	2048–2304 KiB	U	16	2048	U	U	U	U
	LLC-like	24–26 MiB	U	U	U	U	U	U	U
charnwood	L1-like	32–36 KiB	64	8	64	4.55859375	U	U	U
	L2-like	256–288 KiB	U	4	1024	U	U	U	U
	LLC-like	4–8 MiB	U	U	U	U	U	U	U
crux	L1-like	32–36 KiB	64	8	64	2.705078125	U	U	U
	L2-like	256–288 KiB	U	4	1024	U	U	U	U
	LLC-like	4–16 MiB	U	U	U	U	U	U	U
ookay	L1-like	32–36 KiB	64	8	64	3.640625	U	U	U
	L2-like	256–288 KiB	U	4	1024	U	U	U	U
	LLC-like	5–5.5 MiB	U	U	U	U	U	U	U
upgrade	L1-like	32–36 KiB	64	8	64	2.943359375	U	U	U
	L2-like	256–288 KiB	U	4	1024	U	U	U	U
	LLC-like	2–16 MiB	U	U	U	U	U	U	U

records SHA-256 and full member readback verification. The curated clone retains code, statistics, build/idle evidence and one SVG figure; raw intervals remain external.

The earlier rounds reside in `phase1-revised.zip`, followed by `phase1-uncertainty.zip`; sizes and full SHA-256 values appear in the archive manifest and report companion. The `machines/<host>/full`, `followup-v1` and `uncertainty-v1` trees preserve all earlier rounds. Selected plots and the lean clone do not replace the required raw-data submission.

TABLE V: Saved sparse (64 B spacing) class distributions, first four hosts. Each row has $n = 1,000,000$ intervals. W is the representative footprint in KiB; CPU is the measuring logical CPU. All times are native ticks/access (CNTVCT only for Thunderbird, TSC otherwise). Stacked entries retain exact saved values. Δ is the saved same-CPU difference from the preceding class median; U means no qualified increment. O counts retained 1.5-IQR outliers. These are effective classes, not physical-level hit/miss measurements. *Primary contributor: Hong Chan Lee.*

Host	Class	W KiB	CPU	Median	Mean Std. dev.	Q_1 Q_3	P_5 P_{95}	O	Δ
sunbird	1	4	7	4.1328125	4.1449843359375 0.355512804562587	4.1328125 4.140625	4.125 4.140625	1074	U
sunbird	2	128	7	12.1328125	12.16592675 0.7066953043838891	12.1328125 12.1328125	12.125 12.15625	389608	8.0
sunbird	3	2048	7	44.0625	44.206302265625 1.427375641235678	43.8671875 44.2578125	43.59375 44.6796875	49883	31.9296875
sunbird	4	16384	7	110.5703125	112.2182209765625 8.24018847478049	107.171875 114.8203125	102.71875 128.6796875	64480	66.5078125
sunbird	5	131072	7	208.640625	209.2173629140625 4.187925734760392	207.0625 210.4140625	204.953125 214.140625	47301	98.0703125
thunderbird	1	8	0	0.0341796875	0.034319037109375 0.0035492257726338614	0.0341796875 0.0341796875	0.0341796875 0.03515625	133213	U
thunderbird	2	256	0	0.11328125	0.1134740244140625 0.004898643185868331	0.11328125 0.11328125	0.11328125 0.1142578125	172802	0.0791015625
thunderbird	3	4096	0	0.7822265625	0.782517951171875 0.014873852281934413	0.7802734375 0.7841796875	0.77734375 0.7861328125	12365	0.6689453125
thunderbird	4	32768	0	1.724609375	1.7271563017578124 0.04415242300633343	1.7021484375 1.748046875	1.6728515625 1.7861328125	15443	0.9423828125
thunderbird	5	131072	0	2.2158203125	2.21746873046875 0.04251045368255168	2.193359375 2.2392578125	2.1630859375 2.2763671875	12264	0.4912109375
skylark	1	4	1	3.046875	3.06364996875 0.20353415823477908	3.046875 3.046875	3.046875 3.09375	188263	U
skylark	2	128	1	8.34375	8.356172859375 0.38405276507311265	8.25 8.390625	8.15625 8.484375	10370	5.296875
skylark	3	4096	1	27.0	27.070641328125 0.7803271717191348	26.859375 27.1875	26.625 27.375	13856	18.65625
skylark	4	65536	1	245.671875	245.835638859375 4.931346720820529	242.578125 248.8125	238.265625 253.921875	16040	218.671875
artemisia	1	4	28	5.25	5.258905828125 0.48587672845159185	5.2421875 5.2578125	5.2265625 5.265625	5283	U
artemisia	2	256	28	16.2734375	16.3093955859375 0.9060393678243347	16.25 16.2890625	16.2265625 16.3125	6630	11.0234375
artemisia	3	8192	29	67.078125	67.2486189765625 2.229691398040708	66.71875 67.4375	66.1953125 67.9765625	20467	U
artemisia	4	65536	28	211.5625	212.4311838359375 121.69961710162289	207.359375 215.9296875	201.3671875 223.96875	30187	U

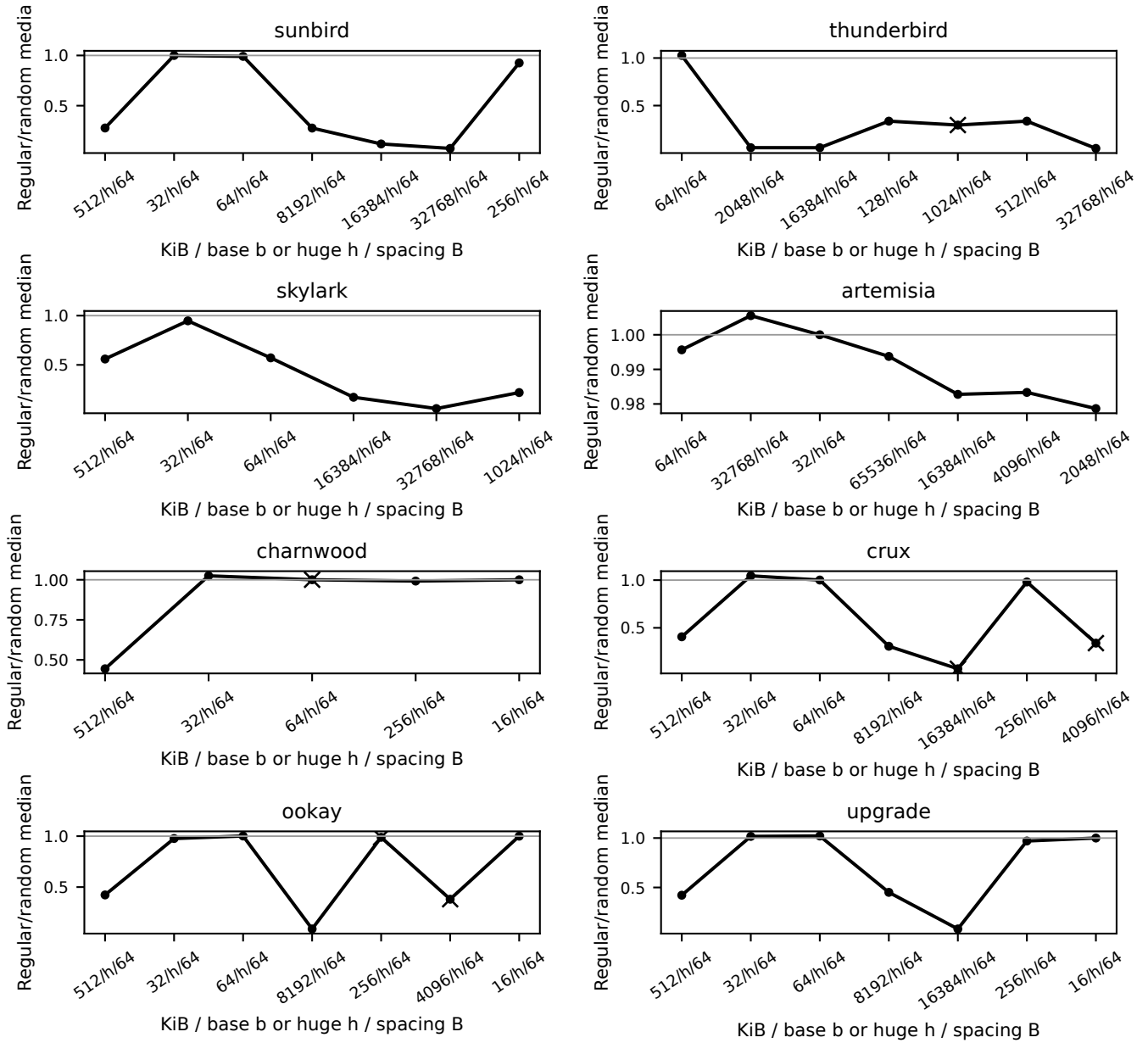


Fig. 6: Regular/random median ratios at every saved follow-up prefetch-control condition. The ordinate is the exact stored ratio, not a refit; x labels identify KiB footprint, base (b)/huge (h) page policy, and spacing in bytes. Crosses mark noisy or different-core pairs. A ratio below one is descriptive evidence that regular traversal is faster; only qualified pairs support a within-core comparison. These controls motivate retaining the randomized dependent curves as primary evidence. *Primary contributor: Hong Chan Lee.*

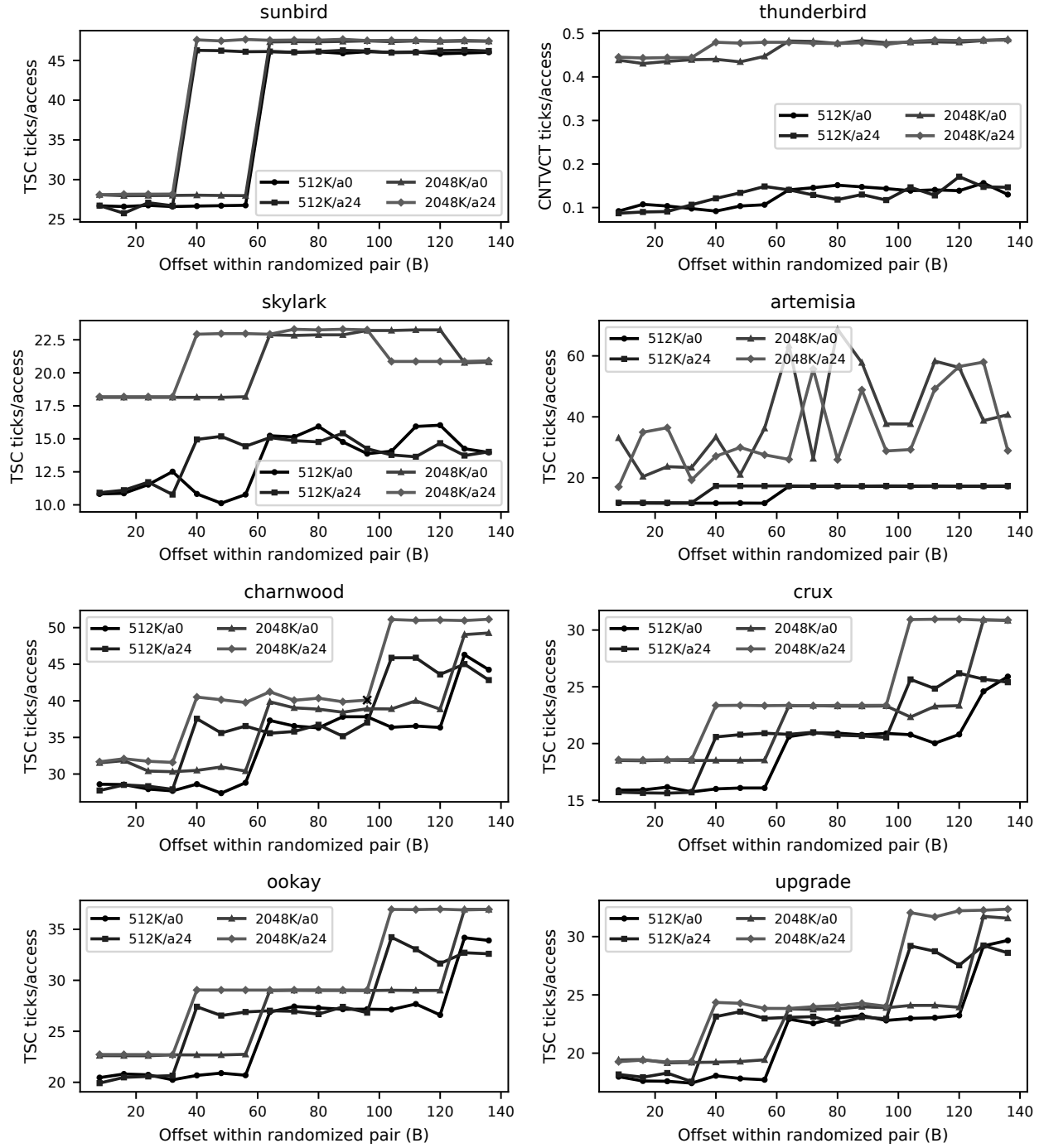


Fig. 7: Spatial-offset curves from the verified follow-up: each legend identifies logical KiB and base shift a in bytes. Crosses retain noisy configurations. The shift-dependent crossing, rather than a single stride slowdown, supports a common visible transfer boundary. *Primary contributor: Hong Chan Lee.*

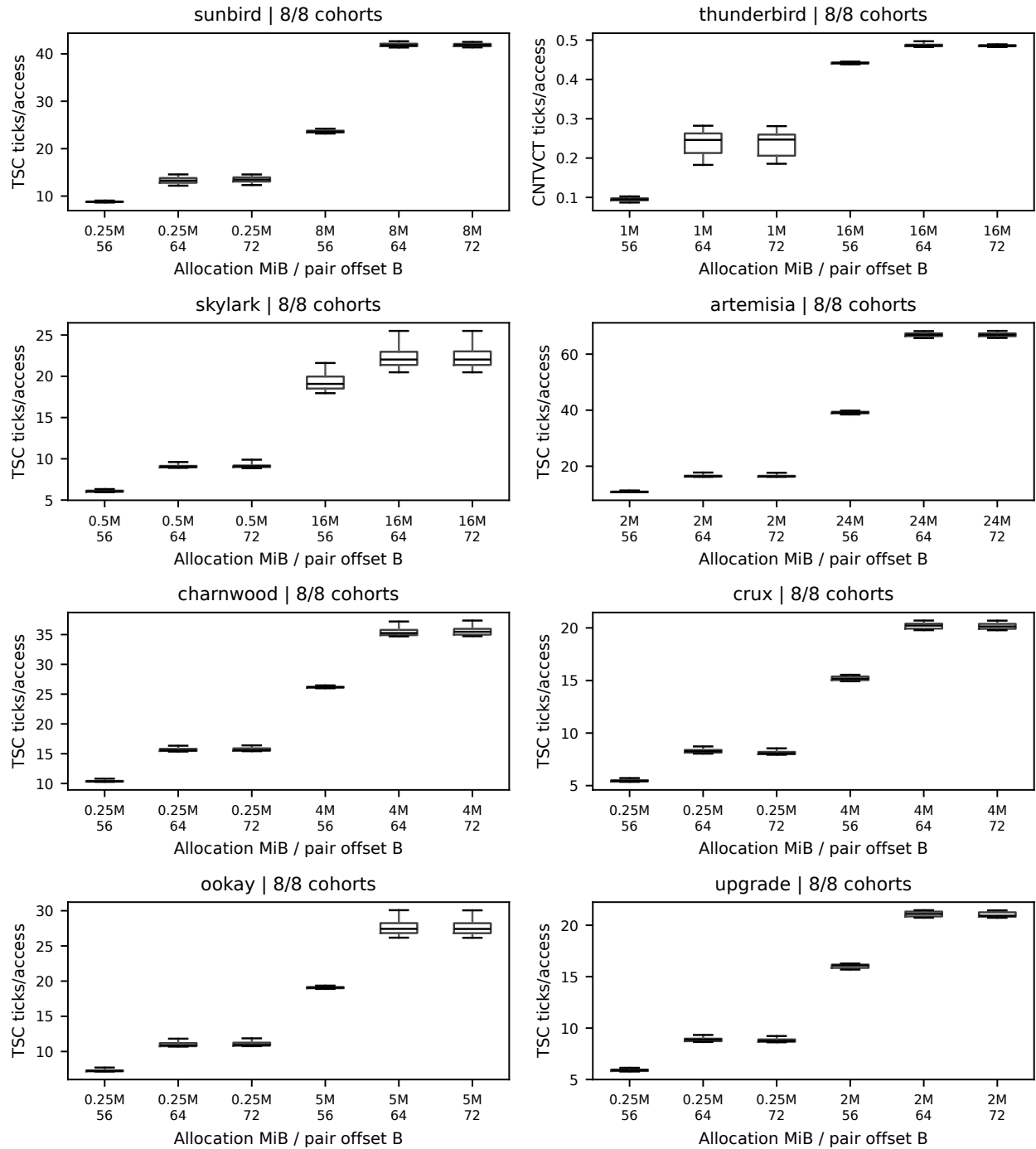


Fig. 8: Below/at/above spatial-boundary distributions from the first parameter-ordered alignment/seed/repetition at each additional allocation scale. Stars mark flagged displayed distributions. The recorded support count summarizes all cohorts, not just the displayed triplets. Boxes do not identify the physical supplying level. *Primary contributor: Hong Chan Lee.*

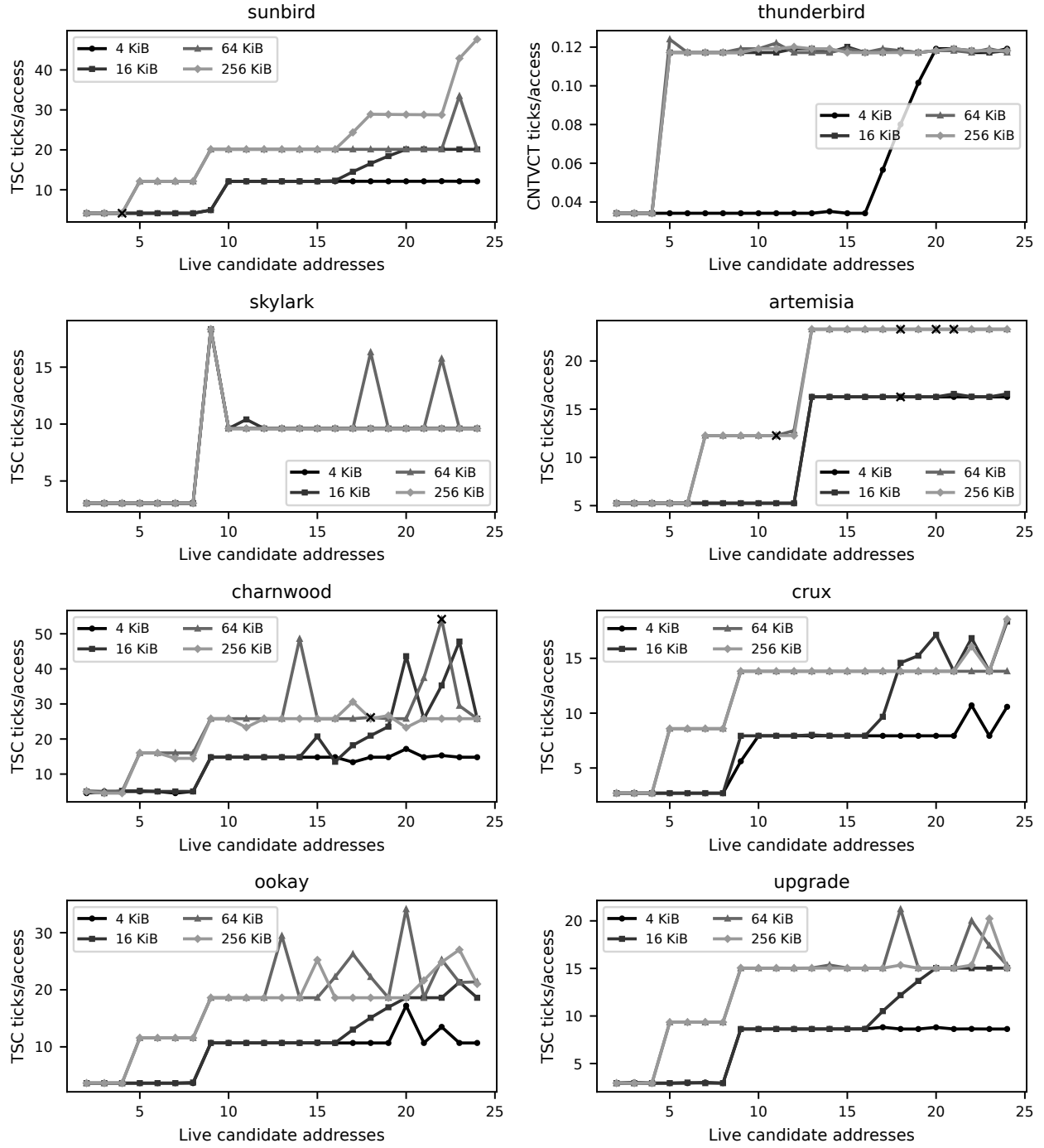


Fig. 9: Baseline primary-suite latency versus live candidate addresses at all tested strides. Crosses mark noisy configurations. Each step is a candidate conflict effect; only the saved capacity-compatible replicated geometry enters Table IV. *Primary contributor: Hong Chan Lee.*

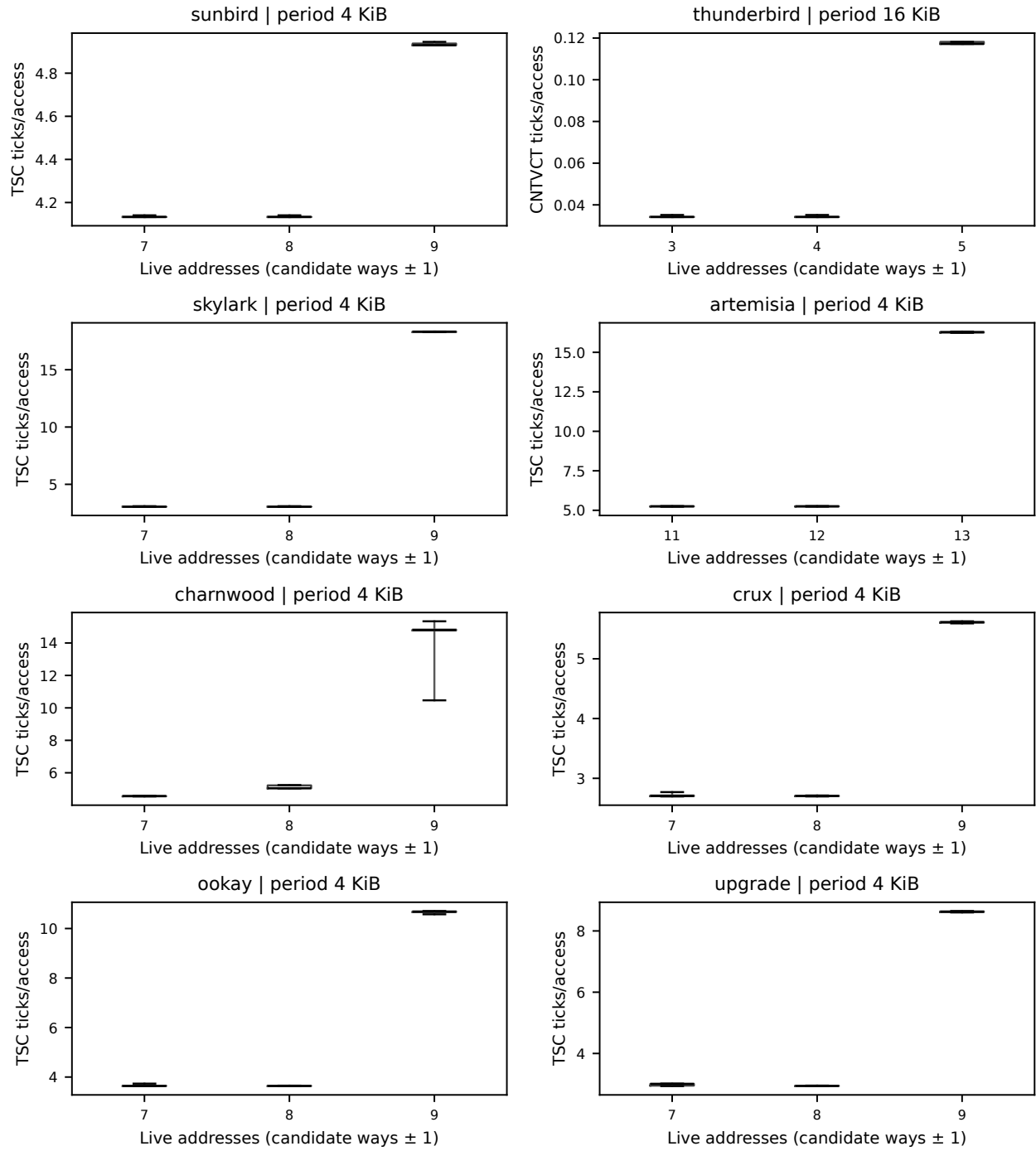


Fig. 10: Distributions one address below, at and above each stored L1-like way candidate at its smallest agreeing tested period. The complete sweep and competing thresholds are retained in Figure 9 and the processed evidence. *Primary contributor: Hong Chan Lee.*

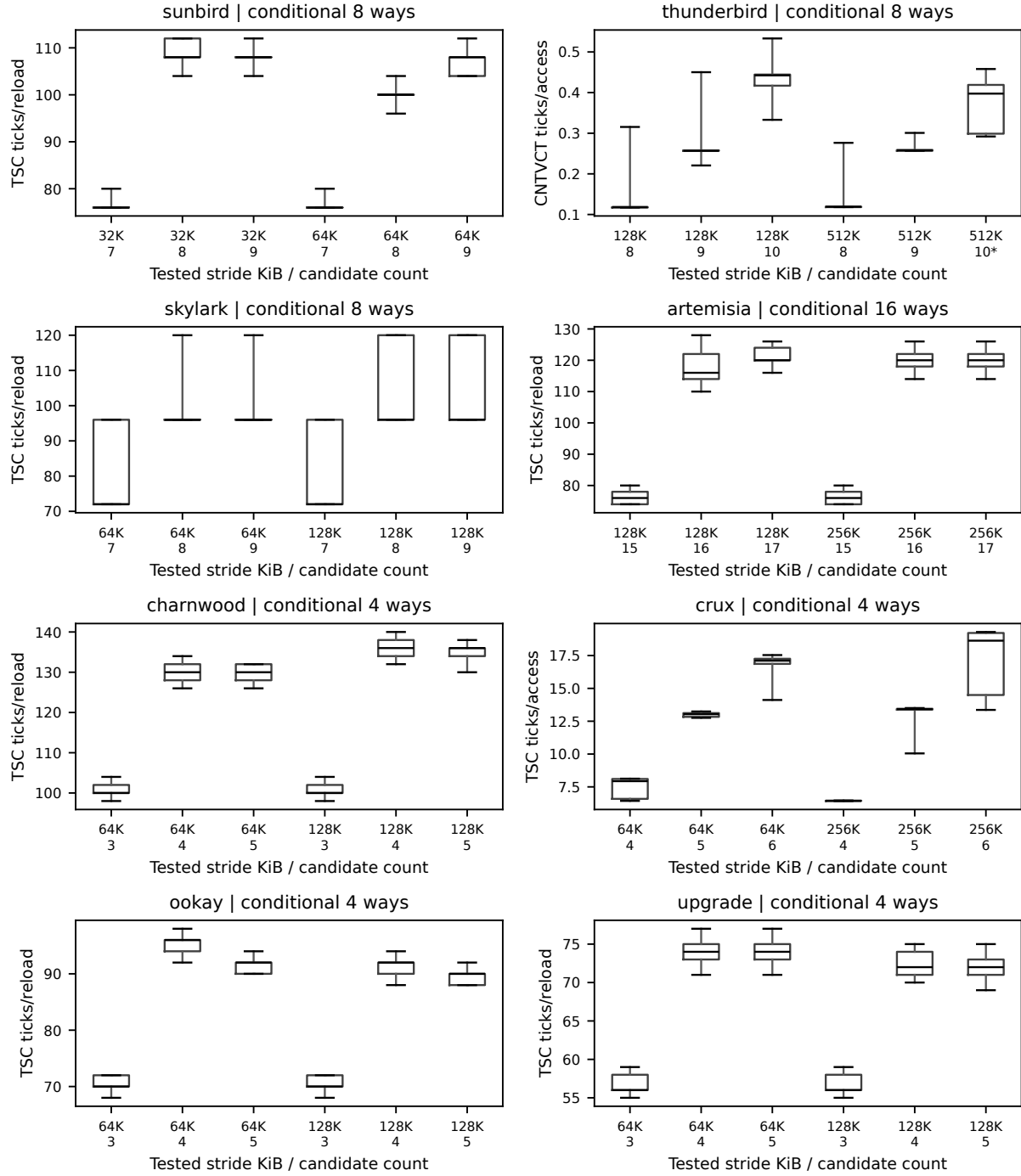


Fig. 11: Confirmed endpoints at the first two agreeing strides and first saved seed/pattern, with the next adjacent saved count in the same parameter cohort where available. A starred neighbor is noisy or on a different core and cannot establish a new edge. The six follow-up cases measure target-reload ticks; Crux and Thunderbird measure additional ring-average ticks/access including fixed scrub. The full replicated inference is preserved; these representative boxes do not replace its required seeds/patterns. *Primary contributor: Hong Chan Lee.*

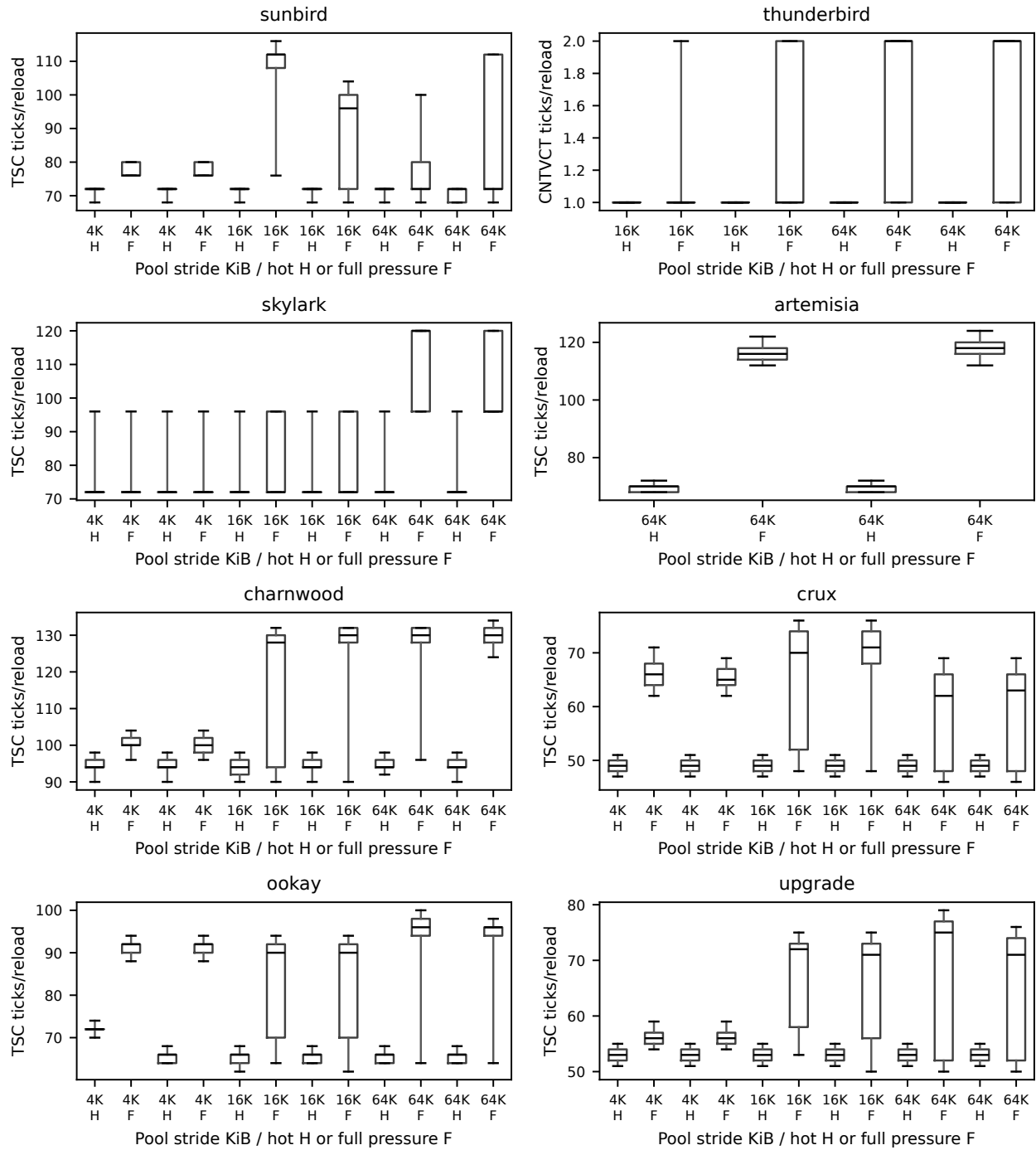


Fig. 12: All held-out hot (H) and full-pressure (F) empirical conflict controls, labelled by pool stride; repeated labels distinguish the saved seeds. Stars mark noisy points. Timer quantization, wrapper overhead and uncalibrated residency prevent an LLC-specific way count from this comparison. *Primary contributor: Hong Chan Lee.*

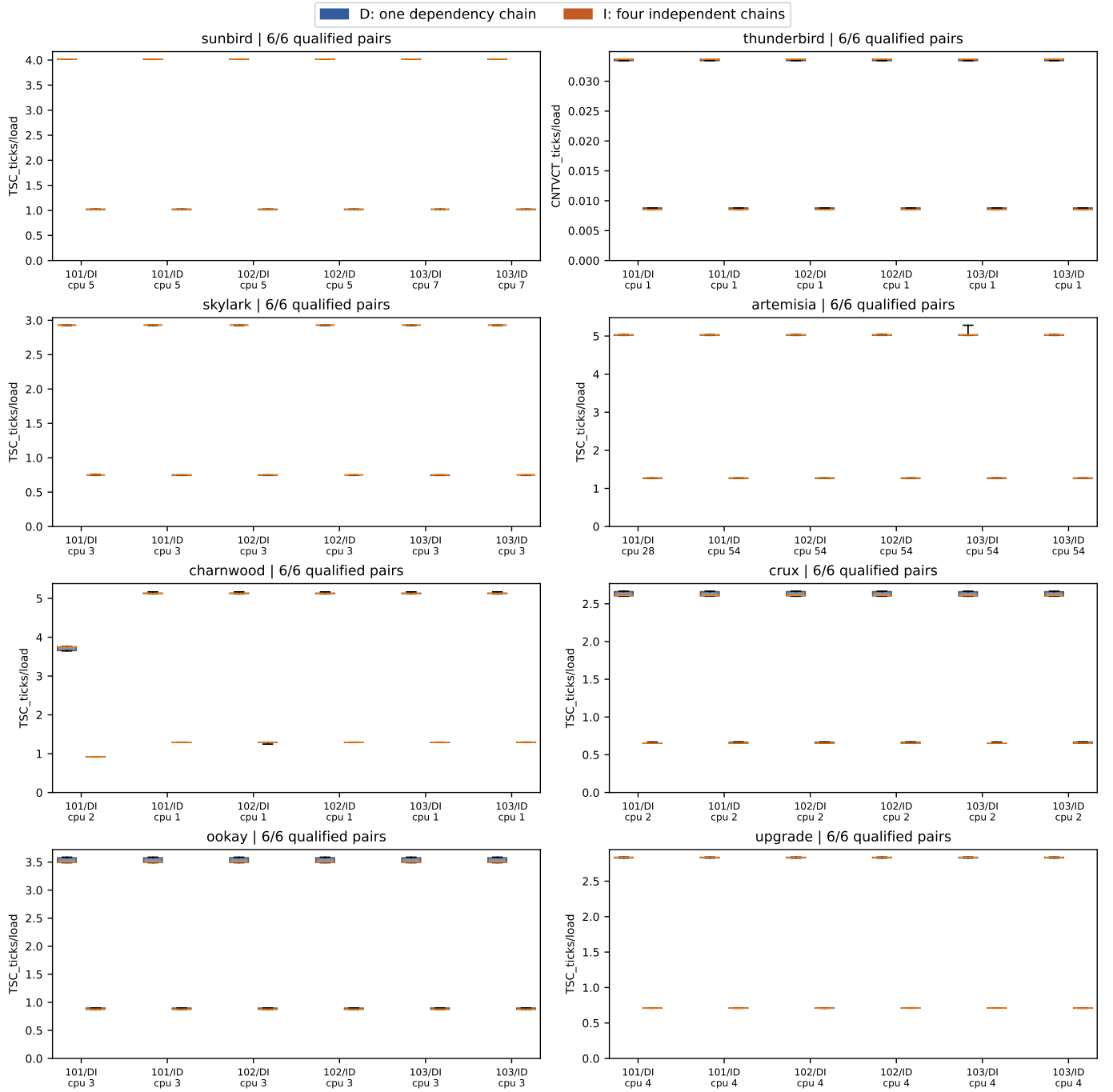


Fig. 13: Matched independent-load diagnostic on all eight machines. D (blue): one dependency chain; I (orange): four independent chains. Each box summarizes one million intervals: median/IQR, with P_5 - P_{95} whiskers. Labels show seed suffix, execution order and selected logical CPU. Every qualified pair is plotted; omitted retries remain archived with their flags. Units are CNTVCT ticks/load for Thunderbird and TSC ticks/load otherwise. The faster I distributions demonstrate throughput/parallelism and must not replace D as latency evidence. *Primary contributor: Hong Chan Lee.*

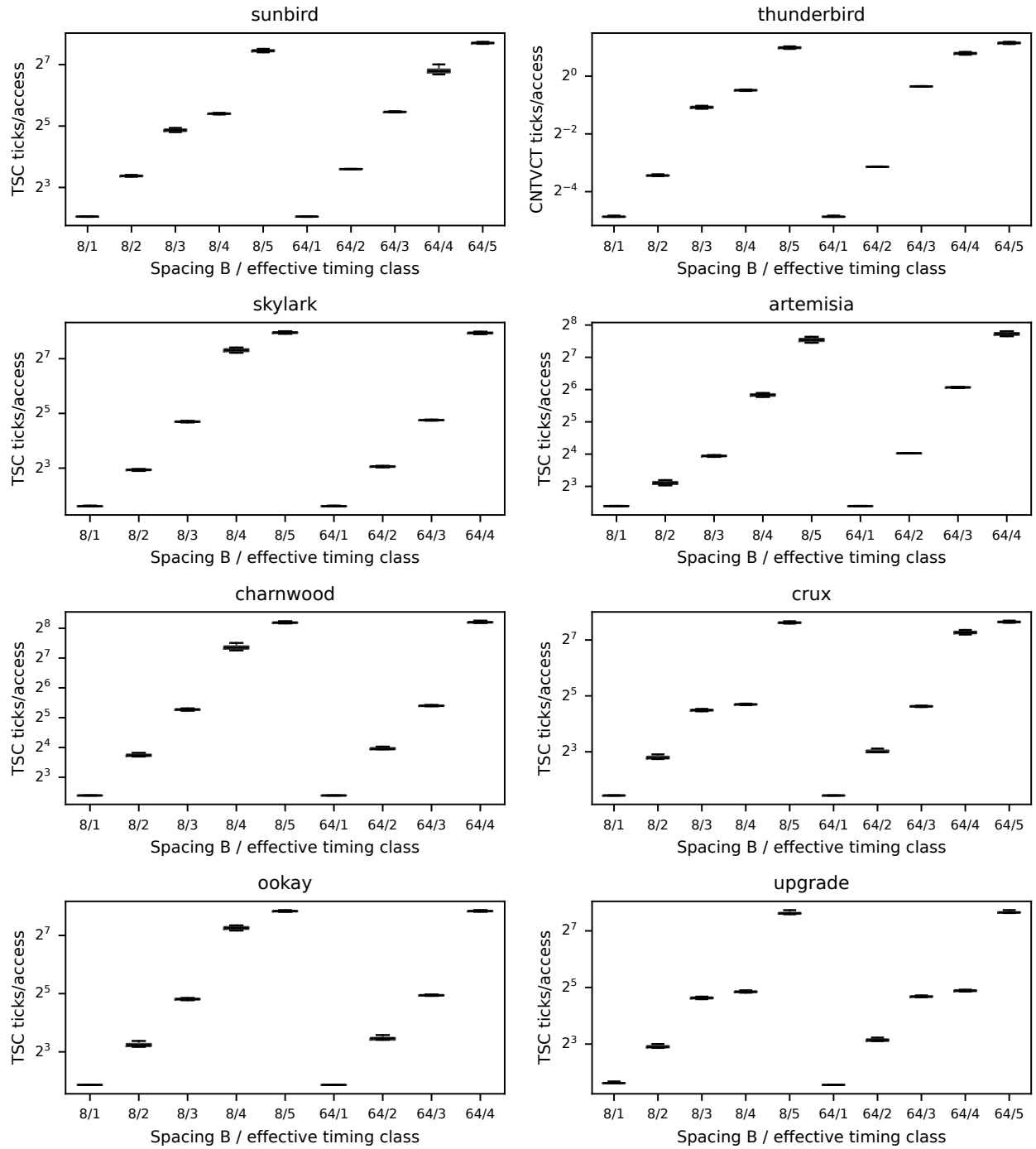


Fig. 14: All saved effective timing-class representatives at both spacings. Labels give spacing in bytes / class index. Boxes retain median, IQR and P_5 - P_{95} ; the complete raw distributions include tails beyond the displayed whiskers. Class indices and differing numbers of modes are not physical cache-level assignments. *Primary contributor: Hong Chan Lee.*

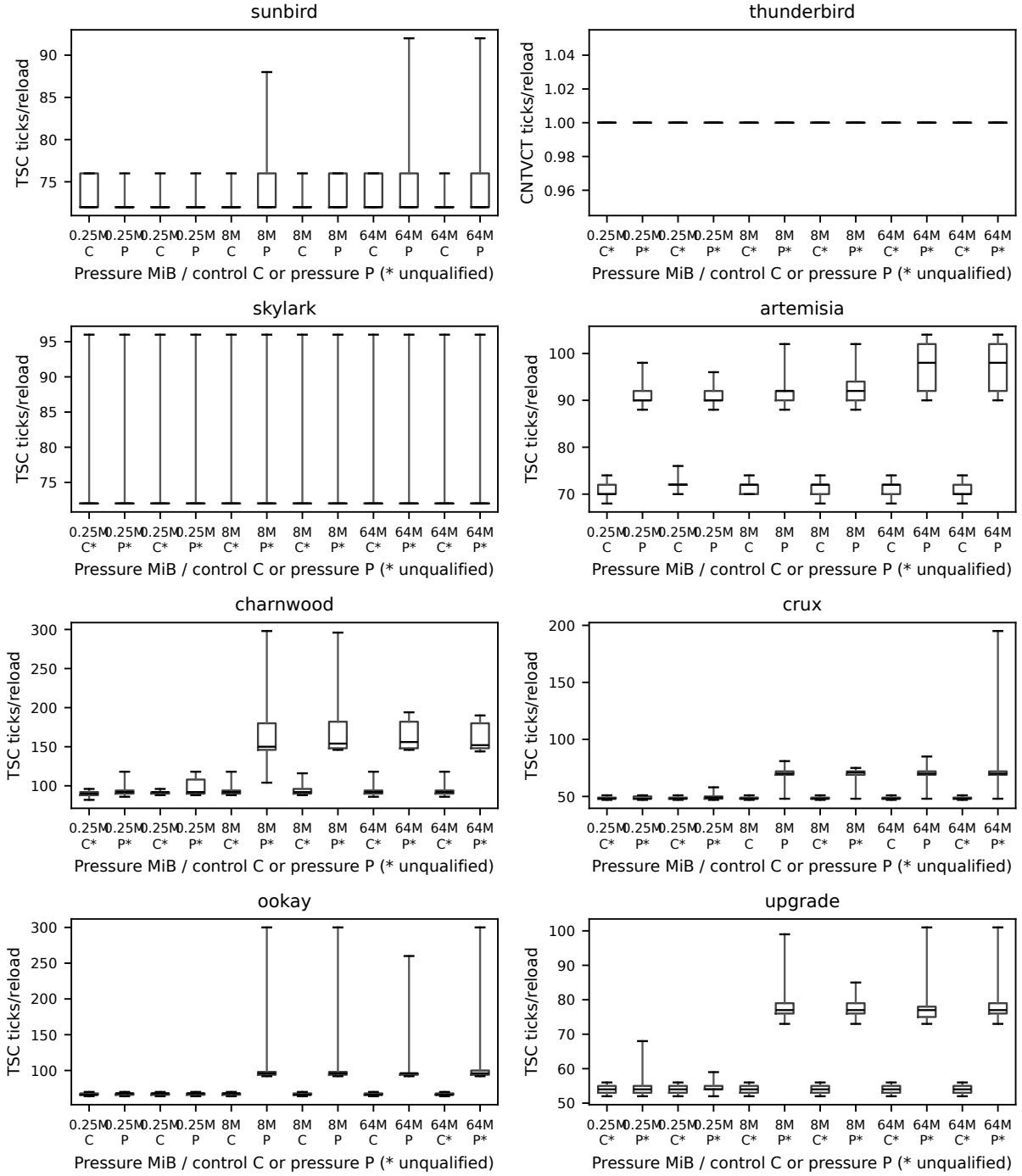


Fig. 15: Cross-core target-reload distributions: each C/P pair is handshake-only control versus pressure, labelled by pressure footprint in MiB; repeated footprints retain the two saved seeds. Asterisks mark noise or different measuring/helper core pairs, which block a causal comparison. Quantized or overlapping distributions and the absence of lower-only eviction calibration leave global policy unresolved. The machine-readable paired metadata are in `historical-behavior.csv`. *Primary contributor: Hong Chan Lee.*

TABLE VI: Saved sparse class distributions, remaining hosts; definitions and units follow Table V. All rows contain one million intervals, and all times here are TSC ticks/access. No cross-core median difference is substituted for an absent Δ . *Primary contributor: Hong Chan Lee.*

Host	Class	W KiB	CPU	Median	Mean Std. dev.	Q_1 Q_3	P_5 P_{95}	O	Δ
charnwood	1	4	2	5.23046875	5.261276703125 0.758460411474933 15.695776390625	5.2265625 5.23828125 15.3515625	5.21484375 5.24609375 15.32421875	21055	U
charnwood	2	128	2	15.3828125	1.425221437516519 42.39641094140625	15.79296875 41.8984375	16.28125 41.6640625	41183	10.15234375
charnwood	3	1024	2	42.171875	2.229685158673251 295.58380798046875	42.46484375 291.9375	43.01171875 289.02734375	19550	26.7890625
charnwood	4	32768	2	293.90234375	10.995070313641353 2.728962080078125	295.94140625 2.701171875	305.2150390624997 2.697265625	60112	251.73046875
crux	1	4	2	2.705078125	1.1639473931489084 8.17849697265625	2.708984375 7.93359375	2.712890625 7.919921875	20243	U
crux	2	128	2	7.94921875	1.9308357549428667 24.847923392578124	8.26171875 24.51953125	8.66015625 24.275390625	39747	5.244140625
crux	3	1024	2	24.69921875	3.2166938567318417 155.35475590625	24.90234375 150.9375	25.130859375 146.64453125	9318	16.75
crux	4	8192	2	154.109375	13.245001889938864 201.58294838867187	157.490234375 198.46484375	163.890625 196.080078125	27523	129.41015625
crux	5	65536	2	200.19140625	13.780578781759857 3.66205953515625	202.0703125 3.6328125	206.61328125 3.62890625	45781	46.08203125
ookay	1	4	2	3.640625	0.5314214877048775 11.10181905078125	3.64453125 10.6875	3.6484375 10.6640625	32988	U
ookay	2	128	2	11.05078125	1.039589831423433 30.88924405078125	11.15625 30.6015625	11.9296875 30.34765625	62679	7.41015625
ookay	3	1024	2	30.7890625	1.4188409211300659 229.0882268828125	30.97265625 226.8828125	31.3515625 224.87890625	22809	19.73828125
ookay	4	32768	2	228.37890625	4.488294904282772 3.019299033203125	230.0859375 2.935546875	233.9140625 2.931640625	42201	197.58984375
upgrade	1	4	5	2.939453125	2.476799485285456 9.013488734375	2.943359375 8.6328125	2.94921875 8.611328125	27156	U
upgrade	2	128	5	8.81640625	3.5315690663561097 26.122651740234375	8.9765625 25.37109375	9.36328125 25.189453125	38535	5.876953125
upgrade	3	1024	2	25.57421875	6.661235430747013 30.0664060859375	25.81640625 29.21875	26.314453125 28.951171875	25404	U
upgrade	4	4096	5	29.51953125	6.237096803752287 204.9470811328125	29.927734375 199.841796875	30.298828125 198.353515625	11842	U
upgrade	5	65536	5	200.93546875	21.674237630482683	202.21875	212.822265625	65624	171.416015625

TABLE VII: Software timing proxy for the 1 MiB workload. Hot/cold columns are the threshold endpoints P_{95}^{hot}/P_5^{cold} in raw native ticks (CNTVCT for Thunderbird, TSC otherwise); θ has the same unit. Hits are exact counts out of 10^6 . Rates and ranges are percentages; slashes separate lower/upper endpoints. These are frozen Phase-I estimates; the later matched PMU comparison is in Section X. *Primary contributor: Hong Chan Lee.*

Host	CPU	Hot P_{95}	Cold P_5	θ	Hits	$\hat{H}$ (%)	Block range (%)	Sensitivity (%)
sunbird	5	72.0	264.0	168.0	999858	99.9858	99.9400/100.0000	0.7233/99.9933
thunderbird	5	1.0	3.0	2.0	999950	99.9950	99.9500/100.0000	72.3907/99.9988
skylark	1	96.0	168.0	132.0	998161	99.8161	95.8600/99.9600	79.9723/99.9908
artemisia	29	74.0	172.0	123.0	988489	98.8489	97.4000/99.6600	6.7616/99.9837
charnwood	1	84.0	342.0	213.0	998592	99.8592	98.8100/100.0000	0.7464/99.9784
crux	4	51.0	236.0	143.5	995035	99.5035	97.6600/99.9900	1.1347/99.8354
ookay	2	68.0	292.0	180.0	989148	98.9148	97.4700/99.9700	1.1408/99.7781
upgrade	1	57.0	252.0	154.5	971276	97.1276	96.2500/99.5500	1.7128/99.3712

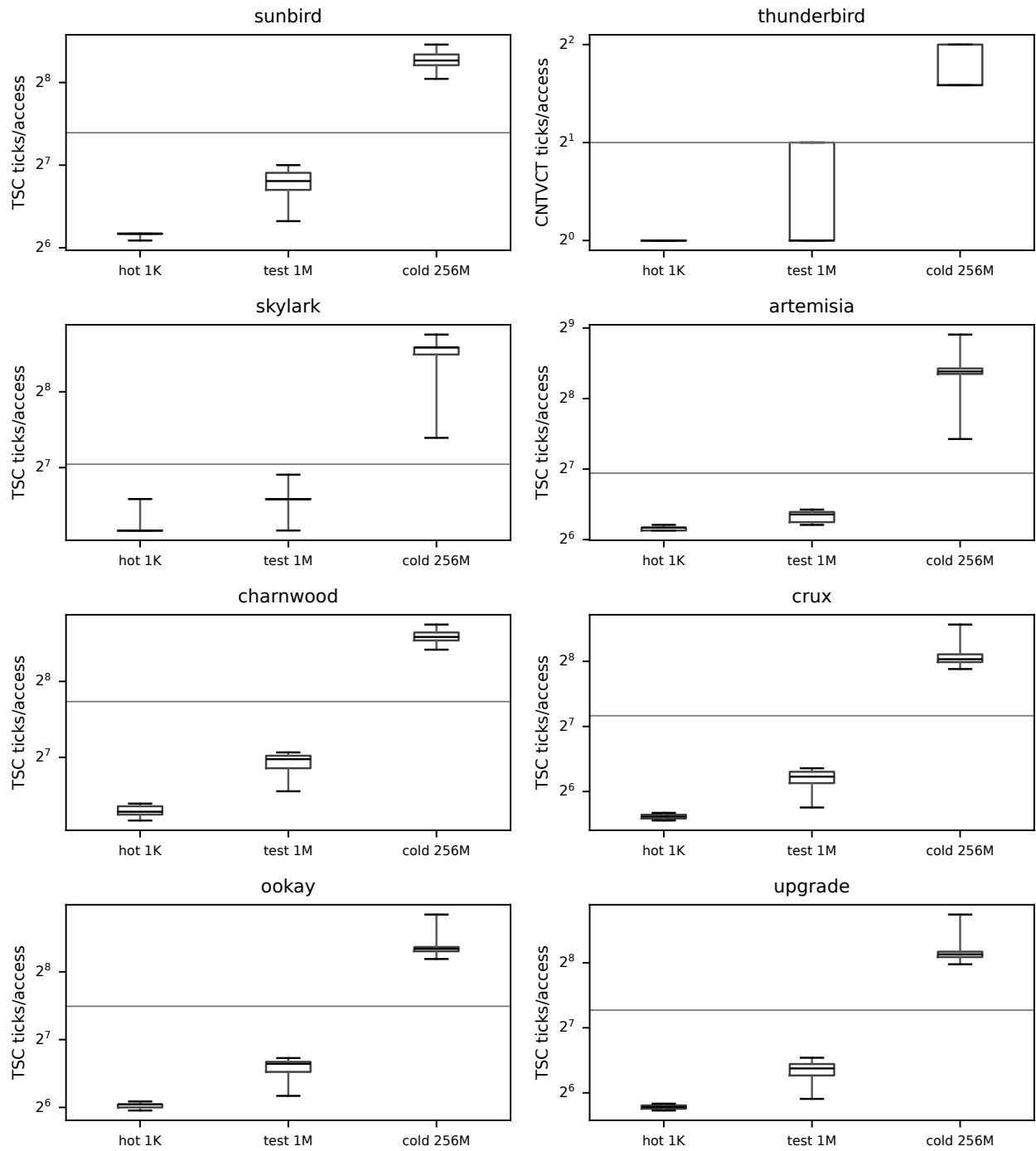


Fig. 16: Empirical hot, test and cold single-access timing distributions for all machines. Horizontal lines show the saved threshold. The box separation supports an operational timing classifier; it does not prove hardware cache-hit labels.
Primary contributor: Hong Chan Lee.

VII. Complete Main Implementations

The complete native source appears in the main body as required by §8.2 and §10. Listings 1–4 include all four verified files without pseudocode substitutions. Shared `cache_bench.c` code appears once with both x86-64 and AArch64 paths, reused by the extensions. Program arguments control sweeps; saved JSON plans and controller records specify grids and seeds. The companion manifest records source SHA-256 identities.

```
1 #define _GNU_SOURCE
2 #include <errno.h>
3 #include <inttypes.h>
4 #include <sched.h>
5 #include <stdint.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include <sys/mman.h>
10 #include <sys/resource.h>
11 #include <time.h>
12 #include <unistd.h>
13
14 /* Phase I: timers and ordinary loads only; no cache discovery interfaces. */
15 static volatile uintptr_t sink;
16 static uint64_t rng(uint64_t *s) {
17     uint64_t x = *s; x ^= x << 13; x ^= x >> 7; x ^= x << 17;
18     return *s = x;
19 }
20 static uint64_t mono(void) {
21     struct timespec t; clock_gettime(CLOCK_MONOTONIC_RAW, &t);
22     return (uint64_t)t.tv_sec * 1000000000ULL + t.tv_nsec;
23 }
24 static uint64_t tick_start(void) {
25     #if defined(FALLBACK_TIMER)
26         return mono();
27     #elif defined(__x86_64__)
28         uint32_t lo, hi;
29         __asm__ volatile("lfence; rdtsc; lfence" : "=a"(lo), "=d"(hi) :: "memory");
30         return ((uint64_t)hi << 32) | lo;
31     #elif defined(__aarch64__)
32         uint64_t t;
33         __asm__ volatile("dsb sy; isb; mrs %0, cntvct_el0; isb" : "=r"(t) :: "memory");
34         return t;
35     #else
36         #error Unsupported ISA: build with -DFALLBACK_TIMER for CLOCK_MONOTONIC_RAW
37     #endif
38 }
39 static uint64_t tick_stop(void) {
40     #if defined(__x86_64__) && !defined(FALLBACK_TIMER)
41         uint32_t lo, hi, aux;
42         __asm__ volatile("rdtscp; lfence" : "=a"(lo), "=d"(hi), "=c"(aux) :: "memory");
43         return ((uint64_t)hi << 32) | lo;
44     #else
45         return tick_start();
46     #endif
47 }
48 /* Explicit register-only dependent loop avoids -O0 stack traffic inside the loop.
49  * The entire program is still compiled at -O0. Assembly is saved and checked. */
50 __attribute__((noinline)) static uintptr_t chase(uintptr_t p, uint64_t n) {
51     #if defined(__x86_64__)
52         __asm__ volatile("1: mov (%0), %0; dec %1; jnz 1b" : "+r"(p), "+r"(n) :: "cc", "memory");
53     #elif defined(__aarch64__)
54         __asm__ volatile("1: ldr %0, [%0]; subs %1, %1, #1; b.ne 1b" : "+r"(p), "+r"(n) :: "cc", "memory");
55     #else
56         while (n--) p = *(volatile uintptr_t *)p;
57     #endif
58     return p;
59 }
60 __attribute__((noinline)) static void loop_control(uint64_t n) {
61     #if defined(__x86_64__)
62         __asm__ volatile("1: dec %0; jnz 1b" : "+r"(n) :: "cc", "memory");
63     #elif defined(__aarch64__)
64         __asm__ volatile("1: subs %0, %0, #1; b.ne 1b" : "+r"(n) :: "cc", "memory");
65     #else
66         while (n--) __asm__ volatile("" :: "memory");
67     #endif
68 }
69 static uint64_t number(const char *s) {
70     char *e; errno=0; unsigned long long v=strtoull(s,&e,10);
71     if (errno || !*s || *e || *s=='-') { fprintf(stderr,"Invalid integer: %s\n",s); exit(2); }
72     return v;
73 }
74 int main(int argc, char **argv) {
75     if (argc != 11) {
```

```

76     fprintf(stderr, "usage: cache_bench MODE BYTES STRIDE OFFSET SAMPLES BATCH SEED CPU ORDER RAW\n"); return
77     2;
78     const char *mode=argv[1], *order=argv[9];
79     size_t bytes=number(argv[2]), stride=number(argv[3]), offset=number(argv[4]);
80     uint64_t samples=number(argv[5]), batch=number(argv[6]), seed=number(argv[7]), state=seed;
81     int cpu=(int)number(argv[8]);
82     if (!seed || !samples || samples>10000000 || !batch || cpu<0 || cpu>=CPU_SETSIZE ||
83         stride<sizeof(uintptr_t) || stride%sizeof(uintptr_t) || bytes<stride*2 || bytes>1073741824ULL ||
84         (strcmp(order,"random") && strcmp(order,"regular")) ||
85         (strcmp(mode,"chase") && strcmp(mode,"spatial") && strcmp(mode,"reload") && strcmp(mode,"overhead") &&
86             strcmp(mode,"loop"))) return 2;
87     if (!strcmp(mode,"spatial") && (offset<sizeof(uintptr_t) || offset%sizeof(uintptr_t) || offset+sizeof(
88         uintptr_t)>stride)) return 2;
89     cpu_set_t set; CPU_ZERO(&set); CPU_SET(cpu,&set);
90     if(sched_setaffinity(0,sizeof(set),&set)) { perror("affinity"); return 3; }
91     cpu_set_t actual; CPU_ZERO(&actual); sched_getaffinity(0,sizeof(actual),&actual);
92     if(CPU_COUNT(&actual)!=1 || !CPU_ISSET(cpu,&actual) || sched_getcpu()!=cpu) return 3;
93     /* All mappings are created and touched AFTER affinity: local first touch. */
94     unsigned char *mem=mmap(NULL,bytes,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0);
95     if(mem==MAP_FAILED) { perror("mmap"); return 4; }
96     int huge_advice=madvise(mem,bytes,MADV_NOHUGEPAGE);
97     memset(mem,0,bytes);
98     size_t n=bytes/stride;
99     size_t *perm=malloc(n*sizeof(*perm)); uint64_t *raw=calloc(samples,sizeof(*raw));
100     if (!perm || !raw) { perror("allocation"); return 4; }
101     /* Fault in raw pages before timing. */
102     for(uint64_t i=0;i<samples;i++) raw[i]=1;
103     for(size_t i=0;i<n;i++) perm[i]=i;
104     if(!strcmp(order,"random")) for(size_t i=n-1;i>0;i--) {
105         size_t j=rng(&state)%(i+1), t=perm[i]; perm[i]=perm[j]; perm[j]=t;
106     }
107     for(size_t i=0;i<n;i++) {
108         uintptr_t *a=(uintptr_t *) (mem+perm[i]*stride);
109         uintptr_t next=(uintptr_t) (mem+perm[(i+1)%n]*stride);
110         if(!strcmp(mode,"spatial")) {
111             *a=(uintptr_t) (mem+perm[i]*stride+offset); *(uintptr_t *) (mem+perm[i]*stride+offset)=next;
112         } else *a=next;
113     }
114     uintptr_t p=(uintptr_t) (mem+perm[0]*stride); free(perm);
115     uint64_t nodes=n*(!strcmp(mode,"spatial")?2:1), warm=nodes*2+batch;
116     p=chase(p,warm);
117     /* Independent target for eviction/reload diagnostic. Pressure is NOT assumed
118        congruent or isolated from upper levels: global inclusion is never inferred. */
119     uintptr_t *target=mmap(NULL,4096,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0);
120     if(target==MAP_FAILED) return 4;
121     *target=(uintptr_t) target;
122     struct timespec pause={0,200000000};
123     uint64_t m0=mono(), t0=tick_start(); nanosleep(&pause,NULL);
124     uint64_t t1=tick_stop(), m1=mono();
125     double hz=(double)(t1-t0)*1e9/(double)(m1-m0);
126     uint64_t freq=0;
127     #if defined(__aarch64__) && !defined(FALLBACK_TIMER)
128     __asm__ volatile("mrs %0, cntfrq_el0" : "=r"(freq));
129     #endif
130     /* Stabilize wake-up/frequency ramp after timer calibration; excluded from samples. */
131     uint64_t settle_start=mono(), settle_loads=0;
132     do { p=chase(p,4096); settle_loads+=4096; } while(mono()-settle_start<100000000ULL);
133     warm+=settle_loads;
134     struct rusage r0,r1; getrusage(RUSAGE_SELF,&r0);
135     int kind=!strcmp(mode,"loop")?1:!strcmp(mode,"reload")?2:!strcmp(mode,"overhead")?3:0;
136     uint64_t begin=mono();
137     for(uint64_t i=0;i<samples;i++) {
138         if(kind==2) { sink=chase((uintptr_t) target,32); p=chase(p,batch); }
139         uint64_t a=tick_start();
140         if(kind==1) loop_control(batch);
141         else if(kind==2) sink=chase((uintptr_t) target,1);
142         else if(kind!=3) p=chase(p,batch);
143         uint64_t b=tick_stop(); raw[i]=b-a;
144     }
145     uint64_t end=mono(); getrusage(RUSAGE_SELF,&r1); sink=p;
146     FILE *out=fopen(argv[10],"wb");
147     if(!out || fwrite(raw,sizeof(*raw),samples,out)!=samples || fclose(out)) { perror("raw output"); return 5; }
148     uint16_t endian=1;
149     #if defined(FALLBACK_TIMER)
150     const char *unit="monotonic_raw_ns";
151     #elif defined(__x86_64__)
152     const char *unit="TSC_ticks";
153     #else
154     const char *unit="CNTVCT_ticks";
155     #endif
156     printf("{ \"mode\": \"%s\", \"bytes\": %zu, \"stride\": %zu, \"offset\": %zu, \"nodes\": %\" PRIu64 \" , \"samples\": %\"
157         PRIu64 \" , \"batch\": %\" PRIu64 \" , \"seed\": %\" PRIu64 \" , \"cpu\": %d, \"final_cpu\": %d, \"unit\": \"%s\", \"

```

```

        counter_frequency\":" PRIu64 " ,\\"empirical_ticks_per_second\":"%.3f,\\"calibration_ns\":" PRIu64 " ,\\"
        elapsed_ns\":" PRIu64 " ,\\"warmup_loads\":" PRIu64 " ,\\"alignment\":"%ld,\\"madvise_nohugepage_rc\":"%d,\\"
        little_endian\":"%s,\\"minor_faults\":"%ld,\\"major_faults\":"%ld,\\"voluntary_switches\":"%ld,\\"
        involuntary_switches\":"%ld,\\"address\":"%p"}\n",
155     mode,bytes, stride,offset,nodes,samples,batch,seed,cpu,sched_getcpu(),unit,freq,hz,m1-m0,end-begin,warm,
        sysconf(_SC_PAGESIZE),huge_advice,*(&unsigned char*)&endian?"true":"false",r1.ru_minflt-r0.ru_minflt,
        r1.ru_majflt-r0.ru_majflt,r1.ru_nvcsw-r0.ru_nvcsw,r1.ru_nivcsw-r0.ru_nivcsw,(&void*)mem);
156     free(raw); munmap(mem,bytes); munmap(target,4096); return 0;
157 }

```

Listing 1: Complete baseline: hierarchy/capacity, spatial pairs, conflict cycles, dependent latency, reload and software-classifier timing. Source: src/cache_bench.c; x86-64 Intel/AMD and AArch64 paths.

```

1 #define main baseline_benchmark_main
2 #include "cache_bench.c"
3 #undef main
4 #include <pthread.h>
5 #include <stdatomic.h>
6
7 /* Phase-I extension. The included source supplies only the same ordinary-load
8 * kernels and architectural timers. Alignment and page policy are experimental
9 * variables; no CPU cache-description interface is used. */
10 static unsigned long huge_kib(void *address) {
11     FILE *f=fopen("/proc/self/smaps","r");
12     char line[512]; int matched=0; unsigned long value=0;
13     if(!f) return 0;
14     while(fgets(line,sizeof(line),f)) {
15         unsigned long start,end,n;
16         if(sscanf(line,"%lx-%lx",&start,&end)==2) {
17             if(matched) break;
18             matched=(uintptr_t)address>=start && (uintptr_t)address<end;
19             } else if(matched && sscanf(line,"AnonHugePages: %lu kB",&n)==1) value=n;
20     }
21     fclose(f); return value;
22 }
23 static unsigned char *mapping(size_t bytes, int huge, size_t *length) {
24     size_t page=(size_t)sysconf(_SC_PAGESIZE);
25     if(huge) {
26         FILE *f=fopen("/sys/kernel/mm/transparent_hugepage/hpage_pmd_size","r");
27         unsigned long n=0;
28         if(f) { if(fscanf(f,"%lu",&n)!=1) n=0; fclose(f); }
29         if(!n || (n&(n-1)) || n>67108864) return MAP_FAILED;
30         page=n;
31     }
32     *length=(bytes+page-1)/page*page;
33     size_t reserve=*length+page;
34     unsigned char *raw=mmap(NULL,reserve,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0);
35     if(raw==MAP_FAILED) return raw;
36     uintptr_t a=((uintptr_t)raw+page-1)&~(uintptr_t)(page-1);
37     size_t prefix=a-(uintptr_t)raw,suffix=reserve-prefix-*length;
38     if(prefix) munmap(raw,prefix);
39     if(suffix) munmap((void *) (a+*length),suffix);
40     unsigned char *mem=(unsigned char *)a;
41     if(madvise(mem,*length,huge?MADV_HUGEPAGE:MADV_NOHUGEPAGE)) {
42         munmap(mem,*length); return MAP_FAILED;
43     }
44     memset(mem,0,*length); return mem;
45 }
46 typedef struct {
47     _Alignas(1024) atomic_uint request;
48     _Alignas(1024) atomic_uint done;
49     _Alignas(1024) atomic_int ready;
50     uintptr_t p;
51     uint64_t loads;
52     int cpu;
53 } helper_state;
54 static void *helper(void *arg) {
55     helper_state *h=arg;
56     cpu_set_t mask; CPU_ZERO(&mask); CPU_SET(h->cpu,&mask);
57     if(sched_setaffinity(0,sizeof(mask),&mask)) { atomic_store(&h->ready,-1); return NULL; }
58     uintptr_t p=h->p;
59     atomic_store(&h->ready,1);
60     unsigned previous=0;
61     for(;;) {
62         unsigned seq=atomic_load_explicit(&h->request,memory_order_acquire);
63         if(seq==UINT32_MAX) break;
64         if(seq!=previous) {
65             if(h->loads) p=chase(p,h->loads);
66             previous=seq;
67             atomic_store_explicit(&h->done,seq,memory_order_release);
68         }
69     }

```

```

70     h->p=p; return NULL;
71 }
72 int main(int argc, char **argv) {
73     if(argc!=15) {
74         fprintf(stderr, "usage: followup MODE BYTES STRIDE OFFSET SAMPLES BATCH SEED CPU ORDER RAW ALIGN
75             PAGE_POLICY ADDRESS_LIST HELPER_CPU\n");
76         return 2;
77     }
78     const char *mode=argv[1], *order=argv[9];
79     size_t bytes=number(argv[2]), stride=number(argv[3]), offset=number(argv[4]), align=number(argv[11]);
80     uint64_t samples=number(argv[5]), batch=number(argv[6]), seed=number(argv[7]), state=seed;
81     int cpu=(int)number(argv[8]), is_spatial=!strcmp(mode, "spatial"), is_probe=!strcmp(mode, "probe"), is_cross=!
82         strcmp(mode, "cross");
83     int is_hot=!strcmp(mode, "hot"), is_empty=!strcmp(mode, "overhead"), huge=!strcmp(argv[12], "huge");
84     int helper_cpu=atoi(argv[14]);
85     if(!seed || !samples || samples>10000000 || !batch || cpu<0 || cpu>=CPU_SETSIZE ||
86         stride<8 || stride%8 || bytes<stride*2 || bytes>536870912 || align%8 || align>4096 ||
87         (strcmp(argv[12], "base") && !huge) || (strcmp(order, "random") && strcmp(order, "regular"))) ||
88         (strcmp(mode, "chase") && !is_spatial && !is_probe && !is_cross && !is_hot && !is_empty)) return 2;
89     if(is_spatial && (offset<8 || offset%8 || align+offset+8>stride)) return 2;
90     if(is_cross && (helper_cpu<0 || helper_cpu>=CPU_SETSIZE || helper_cpu==cpu)) return 2;
91     cpu_set_t mask; CPU_ZERO(&mask); CPU_SET(cpu, &mask);
92     if(sched_setaffinity(0, sizeof(mask), &mask)) { perror("affinity"); return 3; }
93     size_t mapped=0;
94     unsigned char *mem=mapping(bytes+align+8, huge, &mapped);
95     if(mem==MAP_FAILED) { perror("mapping"); return 4; }
96     size_t n=bytes/stride, *positions=malloc((n+1)*sizeof(size_t));
97     uint64_t *raw=malloc(samples*sizeof(uint64_t));
98     if(!positions || !raw) return 4;
99     if(strcmp(argv[13], "-")) {
100         FILE *f=fopen(argv[13], "r"); size_t at=0, v;
101         if(!f) return 2;
102         while(fscanf(f, "%zu", &v)==1) {
103             if(at>=n || v%8 || v+8>bytes || ((is_probe||is_cross)&&v==0)) { fclose(f); return 2; }
104             for(size_t j=0; j<at; j++) if(positions[j]==v) { fclose(f); return 2; }
105             positions[at++]=v;
106         }
107         fclose(f); n=at;
108         if(!n) return 2;
109     } else for(size_t i=0; i<n; i++) positions[i]=i*stride;
110     /* A probe's target is outside the pressure mapping unless an explicit list
111        selects a target-relative pressure set. Both conditions are recorded. */
112     uintptr_t *target;
113     size_t target_length=0;
114     if(strcmp(argv[13], "-")==0) {
115         target=(uintptr_t *)mapping(4096, 0, &target_length);
116         if(target==MAP_FAILED) return 4;
117     } else target=(uintptr_t *)mem;
118     *target=(uintptr_t)target;
119     for(size_t i=n-1; i>0 && !strcmp(order, "random"); i--) {
120         size_t j=rng(&state)%(i+1), t=positions[i]; positions[i]=positions[j]; positions[j]=t;
121     }
122     for(size_t i=0; i<n; i++) {
123         uintptr_t *a=(uintptr_t *) (mem+align+positions[i]);
124         uintptr_t next=(uintptr_t) (mem+align+positions[(i+1)%n]);
125         if(is_spatial) { *a=(uintptr_t)a+offset; *((unsigned char *)a+offset)=next; }
126         else *a=next;
127     }
128     uintptr_t p=(uintptr_t) (mem+align+positions[0]); free(positions);
129     for(uint64_t i=0; i<samples; i++) raw[i]=1;
130     p=chase(p, n*(is_spatial?4:2)+batch);
131     uint64_t m0=mono(), t0=tick_start(); struct timespec pause={0, 20000000}; nanosleep(&pause, NULL);
132     uint64_t t1=tick_stop(), m1=mono(), freq=0;
133     #if defined(__aarch64__) && !defined(FALLBACK_TIMER)
134     __asm__ volatile("mrs %0, cntfrq_el0" : "=r"(freq));
135     #endif
136     double hz=(double)(t1-t0)*1e9/(m1-m0);
137     uint64_t settle=mono(); do {p=chase(p, 4096);} while(mono()-settle<100000000ULL);
138     helper_state *h=NULL; pthread_t thread;
139     if(is_cross) {
140         h=aligned_alloc(1024, ((sizeof(*h)+1023)/1024)*1024);
141         if(!h) return 4;
142         memset(h, 0, sizeof(*h)); h->p=p; h->loads=offset?batch:0; h->cpu=helper_cpu;
143         if(pthread_create(&thread, NULL, helper, h)) return 4;
144         while(!atomic_load(&h->ready)) sched_yield();
145         if(atomic_load(&h->ready)<0) return 3;
146     }
147     unsigned long before_huge=huge_kib(mem);
148     struct rusage r0, r1; getrusage(RUSAGE_SELF, &r0);
149     uint64_t begin=mono();
150     for(uint64_t i=0; i<samples; i++) {
151         if(is_probe || is_cross || is_hot) {
152             sink=chase((uintptr_t)target, 32);

```

```

151         if(is_probe) p=chase(p,batch);
152         if(is_cross) {
153             atomic_store_explicit(&h->request, (unsigned)i+1, memory_order_release);
154             while(atomic_load_explicit(&h->done, memory_order_acquire) != (unsigned)i+1) {}
155         }
156     }
157     uint64_t a=tick_start();
158     if(is_probe || is_cross || is_hot) sink=chase((uintptr_t)target,1);
159     else if(!is_empty) p=chase(p,batch);
160     raw[i]=tick_stop()-a;
161 }
162 uint64_t end=mono(); getrusage(RUSAGE_SELF,&r1);
163 if(h) {atomic_store(&h->request, UINT32_MAX); pthread_join(thread, NULL); free(h);}
164 unsigned long after_huge=huge_kib(mem);
165 sink=p;
166 FILE *f=fopen(argv[10], "wb");
167 if(!f || fwrite(raw,8,samples,f)!=samples || fclose(f)) return 5;
168 uint16_t endian=1;
169 #if defined(FALLBACK_TIMER)
170     const char *unit="monotonic_raw_ns";
171 #elif defined(__x86_64__)
172     const char *unit="TSC_ticks";
173 #else
174     const char *unit="CNTVCT_ticks";
175 #endif
176 printf("{ \"samples\":%" PRIu64 ", \"batch\":%" PRIu64 ", \"cpu\":%d, \"final_cpu\":%d, \"helper_cpu\":%d, \"unit\":"
        "\"%s\", \"counter_frequency\":%" PRIu64 ", \"empirical_ticks_per_second\":%.3f, \"elapsed_ns\":%" PRIu64
        ", \"little_endian\":%s, \"major_faults\":%ld, \"minor_faults\":%ld, \"involuntary_switches\":%ld, \"voluntary_switches\":%ld, \"alignment_shift\":%zu, \"page_policy\": \"%s\", \"mapped_bytes\":%zu, \"logical_bytes\":%zu, \"anon_huge_before_kib\":%lu, \"anon_huge_after_kib\":%lu, \"pressure_nodes\":%zu, \"target_in_pressure_mapping\":%s, \"address\": \"%p\", \"target_address\": \"%p\"} \n",
177     samples, batch, cpu, sched_getcpu(), is_cross?helper_cpu:-1, unit, freq, hz, end-begin, *(unsigned char *)&endian?
        "true":"false",
178     r1.ru_majflt-r0.ru_majflt, r1.ru_minflt-r0.ru_minflt, r1.ru_nivcsw-r0.ru_nivcsw, r1.ru_nivcsw-r0.ru_nivcsw,
179     align, argv[12], mapped, bytes, before_huge, after_huge, n, target_length?"false":"true", (void *)mem, (void *)
        target);
180     free(raw); munmap(mem, mapped); if(target_length) munmap(target, target_length); return 0;
181 }

```

Listing 2: Complete extension: aligned/page-controlled cycles, explicit conflict address lists and paired-core reload pressure. Source: src/followup_bench.c; includes Listing 1 for both ISAs.

```

1 #define _GNU_SOURCE
2 #include <stdint.h>
3 #include <errno.h>
4 #include <sys/resource.h>
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <string.h>
8 #include <sys/mman.h>
9 #include <unistd.h>
10
11 /* Supplemental Phase-I translation control. It includes the unchanged primary
12  * program and replaces ONLY its first anonymous allocation/advice/unmap path.
13  * Read OS page-backing metadata, never CPU cache geometry or counters. */
14 static void *control_mem;
15 static size_t control_length, control_logical_bytes, control_pmd;
16 static const char *control_raw;
17 static int control_advice_rc=-2;
18 static int control_first_mapping=1;
19 static unsigned long control_huge_before_kib, control_huge_after_kib;
20 static int control_usage_reads;
21
22 static size_t pmd_page_size(void) {
23     FILE *f=fopen("/sys/kernel/mm/transparent_hugepage/hpage_pmd_size", "r");
24     unsigned long n=0;
25     if(f) { if(fscanf(f, "%lu", &n)!=1) n=0; fclose(f); }
26     return (size_t)n;
27 }
28 static void *control_mmap(void *address, size_t length, int prot, int flags, int fd, off_t offset) {
29     if(!control_first_mapping) return mmap(address, length, prot, flags, fd, offset);
30     control_first_mapping=0; control_logical_bytes=length; control_pmd=pmd_page_size();
31     if(!control_pmd || (control_pmd & (control_pmd-1)) || control_pmd > 67108864) {
32         fprintf(stderr, "THP control unavailable: missing/unsupported PMD page size (resource limit 64 MiB)\n");
33         errno=ENOTSUP; return MAP_FAILED;
34     }
35     control_length=(length+control_pmd-1)/control_pmd*control_pmd;
36     size_t reserve=control_length+control_pmd;
37     void *raw=mmap(address, reserve, prot, flags, fd, offset);
38     if(raw==MAP_FAILED) return raw;
39     uintptr_t base=(uintptr_t)raw;
40     uintptr_t aligned=(base+control_pmd-1)&~(uintptr_t)(control_pmd-1);

```

```

41 size_t prefix=aligned-base, suffix=reserve-prefix-control_length;
42 if(prefix) munmap(raw,prefix);
43 if(suffix) munmap((void *) (aligned+control_length),suffix);
44 control_mem=(void *)aligned;
45 return control_mem;
46 }
47 static int control_madvise(void *address,size_t length,int advice) {
48     if(address==control_mem) {
49         control_advice_rc=madvice(address,control_length,MADV_HUGEPAGE);
50         return control_advice_rc;
51     }
52     return madvice(address,length,advice);
53 }
54 static unsigned long current_huge_kib(void) {
55     FILE *f=fopen("/proc/self/smaps","r");
56     char line[512]; int matched=0; unsigned long huge=0;
57     if(!f) return 0;
58     while(fgets(line,sizeof(line),f)) {
59         unsigned long start,end,value;
60         if(sscanf(line,"%lx-%lx",&start,&end)==2) {
61             if(matched) break;
62             matched=(uintptr_t)control_mem>=start && (uintptr_t)control_mem<end;
63             } else if(matched && sscanf(line,"AnonHugePages: %lu kB",&value)==1) huge=value;
64         }
65     fclose(f); return huge;
66 }
67 static int control_getrusage(int who, struct rusage *usage) {
68     if(control_usage_reads++==0) {
69         control_huge_before_kib=current_huge_kib();
70         return getrusage(who,usage);
71     }
72     int rc=getrusage(who,usage);
73     control_huge_after_kib=current_huge_kib();
74     return rc;
75 }
76 static void save_page_metadata(void) {
77     unsigned long kernel_kib=0,mmu_kib=0,huge_kib=0;
78     int matched=0;
79     FILE *f=fopen("/proc/self/smaps","r");
80     char line[512];
81     if(f) {
82         while(fgets(line,sizeof(line),f)) {
83             unsigned long start,end,value;
84             if(sscanf(line,"%lx-%lx",&start,&end)==2) {
85                 if(matched) break;
86                 matched=(uintptr_t)control_mem>=start && (uintptr_t)control_mem<end;
87             } else if(matched) {
88                 if(sscanf(line,"KernelPageSize: %lu kB",&value)==1) kernel_kib=value;
89                 if(sscanf(line,"MMUPageSize: %lu kB",&value)==1) mmu_kib=value;
90                 if(sscanf(line,"AnonHugePages: %lu kB",&value)==1) huge_kib=value;
91             }
92         }
93         fclose(f);
94     }
95     size_t len=strlen(control_raw)+32;
96     char *path=malloc(len);
97     if(!path) return;
98     snprintf(path,len,"%s.pages.json",control_raw);
99     f=fopen(path,"w");
100     if(f) {
101         fprintf(f,{"policy\":"MADV_HUGEPAGE for this allocation only","\logical_bytes\":%zu,"\mapped_bytes
            \":%zu,"\pmd_page_bytes\":%zu,"\advice_rc\":%d,"\mapping_found\":%s,"\kernel_page_kib\":%lu,"\
            mmu_page_kib\":%lu,"\anon_huge_kib\":%lu,"\anon_huge_before_kib\":%lu,"\anon_huge_after_kib\":%lu,"\
            note\":"Primary stdout field madvice_nohugepage_rc is the return code of this HUGEPAGE hook in this
            separate control binary\"}\n",control_logical_bytes,control_length,control_pmd,control_advice_rc,
            matched?"true":"false",kernel_kib,mmu_kib,huge_kib,control_huge_before_kib,control_huge_after_kib);
102         fclose(f);
103     }
104     free(path);
105 }
106 static int control_munmap(void *address,size_t length) {
107     if(address==control_mem) { save_page_metadata(); return munmap(address,control_length); }
108     return munmap(address,length);
109 }
110 #define mmap control_mmap
111 #define madvice control_madvise
112 #define munmap control_munmap
113 #define getrusage control_getrusage
114 #define main primary_benchmark_main
115 #include "cache_bench.c"
116 #undef mmap
117 #undef madvice
118 #undef munmap

```

```

119 #undef main
120 #undef getrusage
121 int main(int argc, char **argv) {
122     if(argc!=11) { fprintf(stderr, "Page control uses the primary benchmark arguments\n"); return 2; }
123     control_raw=argv[10];
124     return primary_benchmark_main(argc, argv);
125 }

```

Listing 3: Complete page-backing/translation control. Source: src/page_control.c; includes Listing 1 for both ISAs.

```

1 /* Matched dependency/parallelism diagnostic. Shared timers remain unchanged. */
2 #define main baseline_main
3 #include "cache_bench.c"
4 #undef main
5
6 /* Both loops issue four loads per iteration, with no timed-loop stack accesses.
7  * Independent chains start one quarter-cycle apart in the same random ring. */
8 __attribute__((noinline)) static uintptr_t serial4(uintptr_t p, uint64_t rounds) {
9 #if defined(__x86_64__)
10     __asm__ volatile("1: mov (%0),%0; mov (%0),%0; mov (%0),%0; mov (%0),%0; dec %1; jnz 1b"
11                     : "+&r"(p), "+&r"(rounds) :: "cc", "memory");
12 #elif defined(__aarch64__)
13     __asm__ volatile("1: ldr %0,[%0]; ldr %0,[%0]; ldr %0,[%0]; ldr %0,[%0]; subs %1,%1,#1; b.ne 1b"
14                     : "+&r"(p), "+&r"(rounds) :: "cc", "memory");
15 #else
16 #error This diagnostic requires its audited x86-64 or AArch64 kernel
17 #endif
18     return p;
19 }
20
21 __attribute__((noinline)) static uintptr_t parallel4(uintptr_t a, uintptr_t b,
22                                                     uintptr_t c, uintptr_t d, uint64_t rounds) {
23 #if defined(__x86_64__)
24     __asm__ volatile("1: mov (%0),%0; mov (%1),%1; mov (%2),%2; mov (%3),%3; dec %4; jnz 1b"
25                     : "+&r"(a), "+&r"(b), "+&r"(c), "+&r"(d), "+&r"(rounds) :: "cc", "memory");
26 #elif defined(__aarch64__)
27     __asm__ volatile("1: ldr %0,[%0]; ldr %1,[%1]; ldr %2,[%2]; ldr %3,[%3]; subs %4,%4,#1; b.ne 1b"
28                     : "+&r"(a), "+&r"(b), "+&r"(c), "+&r"(d), "+&r"(rounds) :: "cc", "memory");
29 #endif
30     return a ^ b ^ c ^ d;
31 }
32
33 int main(int argc, char **argv) {
34     if (argc != 12) {
35         fprintf(stderr, "usage: independent_load MODE BYTES STRIDE OFFSET SAMPLES BATCH SEED CPU ORDER RAW
36             STREAMS\n");
37         return 2;
38     }
39     const char *mode=argv[1];
40     size_t bytes=number(argv[2]), stride=number(argv[3]);
41     uint64_t samples=number(argv[5]), batch=number(argv[6]), seed=number(argv[7]), state=seed;
42     int cpu=(int)number(argv[8]), streams=(int)number(argv[11]);
43     if (bytes!=1024 || stride!=8 || number(argv[4])!=0 || !samples || samples>10000000 ||
44         batch<128 || batch>32768 || batch%128 || !seed || cpu<0 || cpu>=CPU_SETSIZE ||
45         (streams!=1 && streams!=4) || strcmp(argv[9],"random") ||
46         (strcmp(mode,"chase") && strcmp(mode,"overhead") && strcmp(mode,"loop"))) return 2;
47     cpu_set_t affinity; CPU_ZERO(&affinity); CPU_SET(cpu,&affinity);
48     if (sched_setaffinity(0, sizeof(affinity), &affinity)) { perror("affinity"); return 3; }
49     CPU_ZERO(&affinity);
50     if (sched_getaffinity(0, sizeof(affinity), &affinity) || CPU_COUNT(&affinity)!=1 ||
51         !CPU_ISSET(cpu, &affinity) || sched_getcpu()!=cpu) return 3;
52     unsigned char *mem=mmap(NULL, bytes, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);
53     uint64_t *raw=calloc(samples, sizeof(*raw));
54     if (mem==MAP_FAILED || !raw) return 4;
55     int advice=madvise(mem, bytes, MADV_NOHUGEPAGE);
56     memset(mem, 0, bytes);
57     for(uint64_t i=0; i<samples; i++) raw[i]=1;
58     size_t n=bytes/stride, perm[128];
59     for(size_t i=0; i<n; i++) perm[i]=i;
60     for(size_t i=n-1; i>0; i--) { size_t j=rng(&state)%(i+1), t=perm[i]; perm[i]=perm[j]; perm[j]=t; }
61     uint64_t ring_hash=14695981039346656037ULL;
62     for(size_t i=0; i<n; i++) {
63         *(uintptr_t *) (mem+perm[i]*stride)=(uintptr_t) (mem+perm[(i+1)%n]*stride);
64         ring_hash=(ring_hash ^ perm[i])*1099511628211ULL;
65     }
66     uintptr_t a=(uintptr_t) (mem+perm[0]*stride), b=(uintptr_t) (mem+perm[n/4]*stride);
67     uintptr_t c=(uintptr_t) (mem+perm[n/2]*stride), d=(uintptr_t) (mem+perm[3*n/4]*stride);
68     /* Functional identity checks: both kernels visit exactly the expected nodes. */
69     uintptr_t next4=(uintptr_t) (mem+perm[1]*stride) ^ (uintptr_t) (mem+perm[n/4+1]*stride) ^
70         (uintptr_t) (mem+perm[n/2+1]*stride) ^ (uintptr_t) (mem+perm[3*n/4+1]*stride);
71     if (serial4(a, 1) != (uintptr_t) (mem+perm[4]*stride) || parallel4(a, b, c, d, 1) != next4 ||
72         serial4(a, n/4) != a || parallel4(a, b, c, d, n) != (a^b^c^d)) return 6;

```

```

72  uint64_t warm=0, start=mono();
73  do {
74      sink=streams==1 ? serial4(a,batch/4) : parallel4(a,b,c,d,batch/4);
75      warm+=batch;
76  } while(mono()-start<100000000ULL);
77  struct rusage before,after; getrusage(RUSAGE_SELF,&before);
78  uint64_t begin=mono();
79  for(uint64_t i=0;i<samples;i++) {
80      uint64_t lo=tick_start();
81      if(!strcmp(mode,"chase"))
82          sink=streams==1 ? serial4(a,batch/4) : parallel4(a,b,c,d,batch/4);
83      else if(!strcmp(mode,"loop")) loop_control(batch/4);
84      uint64_t hi=tick_stop(); raw[i]=hi-lo;
85  }
86  uint64_t elapsed=mono()-begin; getrusage(RUSAGE_SELF,&after);
87  FILE *f=fopen(argv[10],"wb");
88  if(!f || fwrite(raw,sizeof(*raw),samples,f)!=samples || fclose(f)) return 5;
89  uint16_t endian=1;
90  #if defined(__x86_64__)
91      const char *unit="TSC_ticks";
92  #else
93      const char *unit="CNTVCT_ticks";
94  #endif
95  printf("{ \"mode\": \"%s\", \"streams\": %d, \"bytes\": %zu, \"stride\": %zu, \"samples\": %\" PRIu64
96          \", \"batch\": %\" PRIu64 \", \"seed\": %\" PRIu64 \", \"cpu\": %d, \"final_cpu\": %d, \"unit\": \"%s\" \"
97          \", \"elapsed_ns\": %\" PRIu64 \", \"warmup_loads\": %\" PRIu64 \", \"ring_hash\": \"%016\" PRIx64 \" \"
98          \", \"functional_checks_passed\": true, \"little_endian\": %s, \"major_faults\": %ld, \"minor_faults\": %ld \"
99          \", \"involuntary_switches\": %ld, \"voluntary_switches\": %ld, \"madvise_nohugepage_rc\": %d} \n\",
100         mode, streams, bytes, stride, samples, batch, seed, cpu, sched_getcpu(), unit, elapsed, warm, ring_hash,
101         *(unsigned char*)&endian? \"true\": \"false\", after.ru_majflt-before.ru_majflt, after.ru_minflt-before.
102         ru_minflt,
103         after.ru_nivcsw-before.ru_nivcsw, after.ru_nivcsw-before.ru_nivcsw, advice);
104  free(raw);munmap(mem,bytes);return 0;
105 }

```

Listing 4: Complete matched independent-load diagnostic. Source: src/independent_load.c; reuses the unchanged timer/utilities in Listing 1. Both native dependency structures, pointer-identity checks, warm-up and raw output are shown.

```

1  def lower_parameter_factory(out,period,scrub_n):
2      allocation=134217728;origin=2097152
3      directory=out/'address-sets';directory.mkdir(exist_ok=True)
4      def params(stride,n,seed,variant=1,scrub=scrub_n,mode='probe',label='coarse'):
5          fixed=[((2 if variant==1 else 4)*i+1)*period for i in range(scrub)]
6          candidates=[origin+(i+1)*stride for i in range(n)]
7          assert not set(fixed)&set(candidates) and 0 not in fixed+candidates
8          assert all(v+8<allocation for v in fixed+candidates)
9          content='\\n'.join(map(str,fixed+candidates))+ '\\n';digest=hashlib.sha256(content.encode()).hexdigest()
10         path=directory/(digest+'.txt')
11         if not path.exists():path.write_text(content)
12         return dict(mode=mode,bytes=allocation,stride=period,page='huge',batch=2*(scrub+n),seed=seed,
13                    address_list=str(path),address_list_sha256=digest,l2_candidate_stride=stride,l2_candidate_count=n,
14                    upper_scrub_count=scrub,upper_scrub_period=period,upper_scrub_variant=variant,l2_label=label,
15                    index_hypothesis='Fixed odd multiples of L1 candidate period versus target/lower-candidate next
16                    index bit zero')
17     return params

```

Listing 5: Complete lower-level parameter factory used to construct controlled target/scrub address lists. Exact function from scripts/followup_worker.py; native execution is Listing 2.

```

1  def conflict_offsets(period, stride, count, scrub, variant=1):
2      # Every address shares the timing-derived L1 index hypothesis. Fixed odd
3      # multiples spread scrub traffic over other candidate lower-level indices.
4      if stride % (2*period):
5          raise ValueError('Candidate stride must preserve the next index bit')
6      fixed = [((2 if variant == 1 else 4)*i+1)*period for i in range(scrub)]
7      lower = [2097152+i*stride for i in range(count)]
8      result = fixed+lower
9      if len(set(result)) != len(result) or any(v % 8 for v in result):
10         raise ValueError('Invalid conflict construction')
11     return result

```

Listing 6: Complete additional mixed-ring address construction from scripts/uncertainty_worker.py. Period is timing-derived; physical index congruence is not asserted.

```

1  def detected_edges(records, axis):
2      rows = sorted(records, key=lambda r: r['parameters'][axis])
3      edges = []

```

```

4  for a,b in zip(rows, rows[1:]):
5      if a.get('status') != 'passed' or b.get('status') != 'passed': continue
6      sa,sb = a['statistics'],b['statistics']
7      if a['selected']['cpu'] != b['selected']['cpu']: continue
8      if sb['median'] > 1.15*sa['median'] and sb['q1'] > sa['q3']:
9          edges.append([a['parameters'][axis],b['parameters'][axis]])
10     return edges

```

Listing 7: Complete prospective adjacent-edge acquisition rule from scripts/uncertainty_worker.py. It checks quality and common CPU before scheduling refinement; it is not a statistical significance test.

```

1  def estimator(rows, host):
2      data=sorted([r for r in rows if r['family']=='software_metric'],key=lambda r:r['parameters']['bytes'])
3      if len(data)!=3 or any(r['qualification']!='verified_clean' for r in data):
4          return dict(status='unavailable: calibration/workload incomplete or noisy')
5      hot, target, cold=data
6      upper=hot['statistics']['p95']; lower=cold['statistics']['p05']
7      if upper >= lower:
8          return dict(status='unresolved: hot/cold calibration distributions overlap',hot_p95=upper,cold_p05=lower)
9
10     sources=[r['record_path'] for r in data]
11     threshold=(upper+lower)/2
12     raw=read_raw(ROOT/target['raw_path'],target['measurement']['little_endian'])
13     hits=sum(v<=threshold for v in raw); n=len(raw); rate=hits/n
14     # Wilson interval is a binomial diagnostic; correlated chain samples can violate independence.
15     z=1.96; den=1+z*z/n; center=(rate+z*z/(2*n))/den
16     radius=z*math.sqrt(rate*(1-rate)/n+z*z/(4*n*n))/den
17     blocks=[sum(v<=threshold for v in raw[i:i+10000])/len(raw[i:i+10000]) for i in range(0,n,10000)]
18     return dict(status='timing-classified cache-residency proxy; not a validated hardware hit rate',
19                 classified_hits=hits,total_timed_accesses=n,estimate=rate,threshold_raw_ticks=threshold,
20                 threshold_sensitivity_rate_range=[sum(v<=upper for v in raw)/n,sum(v<=lower for v in raw)/n],
21                 threshold_sensitivity_limits_raw_ticks=[upper,lower],
22                 wilson_95_independent_sample_assumption=[center-radius,center+radius],block_rate_range=[min(
23                     blocks),max(blocks)],
24                 standardized_workload_bytes=1048576,unit=target['measurement']['unit'],sources=sources,
25                 calibration_and_workload_cpus=[r['selected']['cpu'] for r in data],same_logical_cpu=len({r['selected']['cpu'] for r in data})==1,
26                 limitations='Single-load timer overhead/quantization; smallest/largest footprints are empirical proxies, not PMU-confirmed classes. Adjacent samples are correlated. Wilson intervals omit calibration/classification error; threshold-sensitivity and block ranges expose additional uncertainty. Phase-II validation not performed.')

```

Listing 8: Complete timing classifier and uncertainty calculation from scripts/analyze.py (estimator). It consumes verified baseline timing rows and raw target intervals; no counter values are used.

VII-A. Native validation excerpts

Native audits pass on all eight hosts. These exact excerpts show dependent loads and the decrement/back-edge loop, with stack setup/teardown outside the inner loop. This overhead isolation is not claimed for reload wrappers.

```

1  0000000004013e7 <chase>:
2  4013e7: 55                push    %rbp
3  4013e8: 48 89 e5          mov     %rsp,%rbp
4  4013eb: 48 89 7d f8        mov     %rdi,-0x8(%rbp)
5  4013ef: 48 89 75 f0        mov     %rsi,-0x10(%rbp)
6  4013f3: 48 8b 55 f8        mov     -0x8(%rbp),%rdx
7  4013f7: 48 8b 45 f0        mov     -0x10(%rbp),%rax
8  4013fb: 48 8b 12          mov     (%rdx),%rdx
9  4013fe: 48 ff c8          dec     %rax
10 401401: 75 f8            jne     4013fb <chase+0x14>
11 401403: 48 89 55 f8        mov     %rdx,-0x8(%rbp)
12 401407: 48 89 45 f0        mov     %rax,-0x10(%rbp)
13 40140b: 48 8b 45 f8        mov     -0x8(%rbp),%rax
14 40140f: 5d              pop     %rbp
15 401410: c3              retq

```

Listing 9: Inspected x86-64 chase on Artemisia, from the saved follow-up native-kernel audit.

```

1  000000000401124 <chase>:
2  401124: d10043ff        sub     sp, #0x10
3  401128: f90007e0        str     x0, [sp, #8]
4  40112c: f90003e1        str     x1, [sp]
5  401130: f94007e1        ldr     x1, [sp, #8]
6  401134: f94003e0        ldr     x0, [sp]
7  401138: f9400021        ldr     x1, [x1]
8  40113c: f1000400        subs    x0, x0, #0x1
9  401140: 54ffffc1        b.ne    401138 <chase+0x14>  // b.any

```

```
10 401144: f90007e1  str x1, [sp, #8]
11 401148: f90003e0  str x0, [sp]
12 40114c: f94007e0  ldr x0, [sp, #8]
13 401150: 910043ff  add sp, sp, #0x10
14 401154: d65f03c0  ret
```

Listing 10: Inspected AArch64 chase on Thunderbird, from the saved follow-up native-kernel audit.

VIII. Phase II: Performance-Counter Verification

Phase II was conducted only after the Phase-I timing measurements and boundary interpretations had been frozen. The purpose of this stage was to determine whether the timing-only transitions identified in Phase I correspond to measurable changes in hardware-cache and translation behavior, without using counter measurements to retroactively select or move a Phase-I boundary. The Phase-II measurements therefore use the same frozen benchmark binary, access pattern, stride, batch size, seed and CPU affinity whenever the corresponding Phase-I configuration was available.

VIII-A. Testing Procedure

The performance-counter study uses the course-provided `pmu_stat` wrapper around Linux `perf_event_open`. The wrapper attaches a group of hardware events to the unchanged Phase-I `cache_bench` process and reports both `time_enabled` and `time_running`. A measurement is treated as valid only when the requested events successfully open and `time_running` is nonzero; measurements with incomplete scheduling are retained as diagnostic evidence but are not used as counter results.

The target event set contains the eight project-level metrics:

`cycles`, `instructions`, `L1D-read-access`,
`L1D-read-miss`, `LL-read-access`, `LL-read-miss`,
`DTLB-read-miss`, `cache-misses`.

The project wrapper maps the generic names to Linux hardware and hardware-cache event abstractions rather than requiring processor-specific event strings. Direct `perf` experiments were used only to diagnose event availability and scheduling behavior; the final Phase-II acquisition uses the project-provided event names through `pmu_stat`.

Initial attempts grouped all eight events, but Thunderbird/Sunbird-class PMU scheduling constraints prevented the complete group from running. In particular, a four-event group containing `cycles`, `instructions` and the two L1D events produced `time_running=0`, while the L1D events ran successfully when isolated as a two-event group. Similarly, the LLC and DTLB/cache event pairs were successfully measured as two-event groups. The final acquisition therefore uses two events per PMU run:

- 1) `cycles`, `instructions`;
- 2) `L1D-read-access`, `L1D-read-miss`;
- 3) `LL-read-access`, `LL-read-miss`;
- 4) `DTLB-read-miss`, `cache-misses`.

This batching does not change the benchmark workload; it only separates counter groups so that each group can be scheduled completely.

For each capacity point, the benchmark is executed independently for each event pair using the same logical

CPU, seed, stride, batch size, sample count and access order. The completed boundary-correlation experiment uses one million measured intervals per point. Each PMU batch records the benchmark metadata, event names, raw counter values, enabled time, running time, command line, and `pmu_stat` diagnostics. The raw binary timing output is retained when required for later verification.

A valid PMU batch satisfies

$$T_{\text{running}} > 0$$

and preferably

$$T_{\text{running}} = T_{\text{enabled}},$$

which indicates that the event group was continuously scheduled and was not multiplexed. Equal enabled and running times were observed for the verified Phase-II measurements reported below.

VIII-B. Counter Availability and Validation

The PMU environment was first characterized on Sunbird, whose processor is an Intel Xeon E5-2680 v3 (Haswell-EP). Linux reports 48 logical CPUs, 24 physical cores and a 60 MiB last-level cache. The project-level L1D, LLC, DTLB and generic cache event abstractions were tested before the full sweep.

The L1D events illustrate why event validation is necessary. Direct Linux `perf` measurements exposed processor-specific L1 load events, but the project wrapper intentionally uses the generic `PERF_TYPE_HW_CACHE` abstraction. The project-defined `L1D-read-access` and `L1D-read-miss` events both opened and ran successfully individually and as a two-event group. A representative L1-resident run produced approximately 5.91×10^8 L1D reads and 4.6×10^7 L1D misses with 100% running time. By contrast, combining these events with `cycles` and `instructions` resulted in a zero running-time measurement. This behavior was treated as a PMU scheduling constraint rather than as evidence that the L1 events were unavailable.

The LLC and DTLB events also passed direct validation. The two-event `LL-read-access/LL-read-miss` group ran for the full measurement interval, as did `DTLB-read-miss/cache-misses`. Representative runs therefore provide complete, unsullied counter observations rather than values reconstructed from partial multiplexed execution.

VIII-C. Stride and Associativity Verification

Phase II also provided an independent verification of the cache line size and associativity estimates obtained during Phase I. Rather than relying only on timing transitions, the verification experiments used hardware performance monitoring data to determine whether the changes in cache behavior occurred at the same stride and conflict levels predicted by the Phase I measurements.

The verification was performed across all eight machines to determine whether the observed cache behavior was consistent across the different systems.

For stride and line-size verification, the experiments varied the byte spacing between consecutive memory accesses while monitoring the resulting cache-access and cache-miss behavior. A transition in the miss behavior at a consistent stride provides evidence for the spatial granularity of the cache, since accesses within the same cache line can be satisfied without requiring an additional line fill, while accesses separated beyond the line boundary require a new cache line. The Phase I experiments had already identified a visible 64-byte spatial boundary on all eight machines, and the Phase II measurements were used to determine whether the PMU measurements exhibited a corresponding change in behavior. This provides independent support for the 64-byte line-size observation without relying solely on the timing-based inference. The verification is treated as evidence for the observed boundary rather than as an absolute hardware specification, since PMU event availability and counter behavior differed between machines.

Associativity was verified using controlled conflict accesses in which multiple addresses were selected to map to the same cache set. The number of conflicting cache lines was progressively increased while monitoring cache misses. If the number of simultaneously conflicting lines exceeds the cache’s effective associativity, previously accessed lines should begin to be evicted, producing an increase in the measured miss rate. This approach directly tests the conflict behavior underlying the associativity candidates identified during Phase I. The repository’s Phase I methodology used conflict-stride/count sweeps, empirical candidate sets, and held-out tests to identify these candidates, while Phase II supplied an independent PMU-based measurement for comparison.

Tables VIII and IX report the acquired PMU values directly. The L1 conflict test has a large miss-rate rise at or immediately above every timing-derived way candidate. The three stride points around 64 B are broad rather than a sharp PMU edge, and Sunbird’s stride event group has zero running time. Thus the PMU conflict data corroborate the conditional L1 way candidates, while the 64 B result remains principally timing-supported. Generic LL events either show no corresponding L2 conflict edge or are unavailable; they do not validate the conditional L2 way candidates.

VIII-D. Boundary-Correlation Workload

To test the specific requirement of correlating a timing-derived boundary with a hardware miss-count transition, the frozen Phase-I boundary was used to define a dense follow-up sweep. For the Sunbird L1-like candidate, the stored Phase-I interval is 32–36 KiB. The follow-up sweep extends approximately 50% below and above that interval and includes the exact interval endpoints. The resulting logical footprints are 16,384, 18,132, 20,066, 22,207,

24,576, 27,198, 30,099, 32,768, 33,310, 36,864, 40,797, 45,149, 49,966 and 55,296 bytes.

The same dependent randomized **chase** workload is used at every point with stride 8 B, batch size 512, seed 5,922,026, one million samples and a fixed measuring CPU. The purpose is not to locate a new boundary from PMU data, but to determine whether the already inferred timing transition is accompanied by an increase in the corresponding cache miss count. This preserves the Phase-I/Phase-II ordering required by the study.

VIII-E. Cross-Machine Boundary-Correlation Overview

Boundary-correlation sweeps (dense sampling across the frozen Phase-I bracket plus 50% margin on each side, per host and per level) are complete for all eight machines. Rather than presenting each host’s full sweep individually, this section summarizes the trend across the machine set; the team’s imported figure sheet provides the full set of per-machine curves referenced below.

a) L1.

Chart 17 classifies each machine’s L1 sweep by comparing the maximum miss rate observed strictly below the frozen bracket against the maximum miss rate observed within it. A transition is called *clean* when the within-bracket rate exceeds at least three times the below-bracket rate.

17

18

19

Seven of eight machines show a clean L1 transition coincident with the frozen Phase-I bracket. Charnwood is the one exception: its below-bracket maximum (8.48%) is already elevated relative to the other seven machines, so the within-bracket rise to 23.85% (a $2.8\times$ increase) falls just short of the $3\times$ threshold used here, though a real rise is still present. This is reported as a weaker, not absent, correlation.

b) LLC.

Table XI repeats the classification for LLC, using **LL-read-miss/LL-read-access** where available (recall §VIII-G: unavailable on Skylark and Thunderbird).

Four of the six machines with LL-* available show a clean LLC transition. Artemisia and Ookay do not: Artemisia’s below-bracket rate is already low and the within-bracket rate barely rises above it, while Ookay’s below-bracket rate (8.51%) is already substantial before the bracket is reached, suggesting the true transition may sit lower than the frozen bracket’s low edge for this host, or that the swept margin did not extend low enough to capture a clean baseline. Both are reported as open disagreements rather than resolved in either direction.

c) L2.

Table XII reports the L2 sweep outcome for all eight machines, using **L1D-read-miss** (the closer of the two

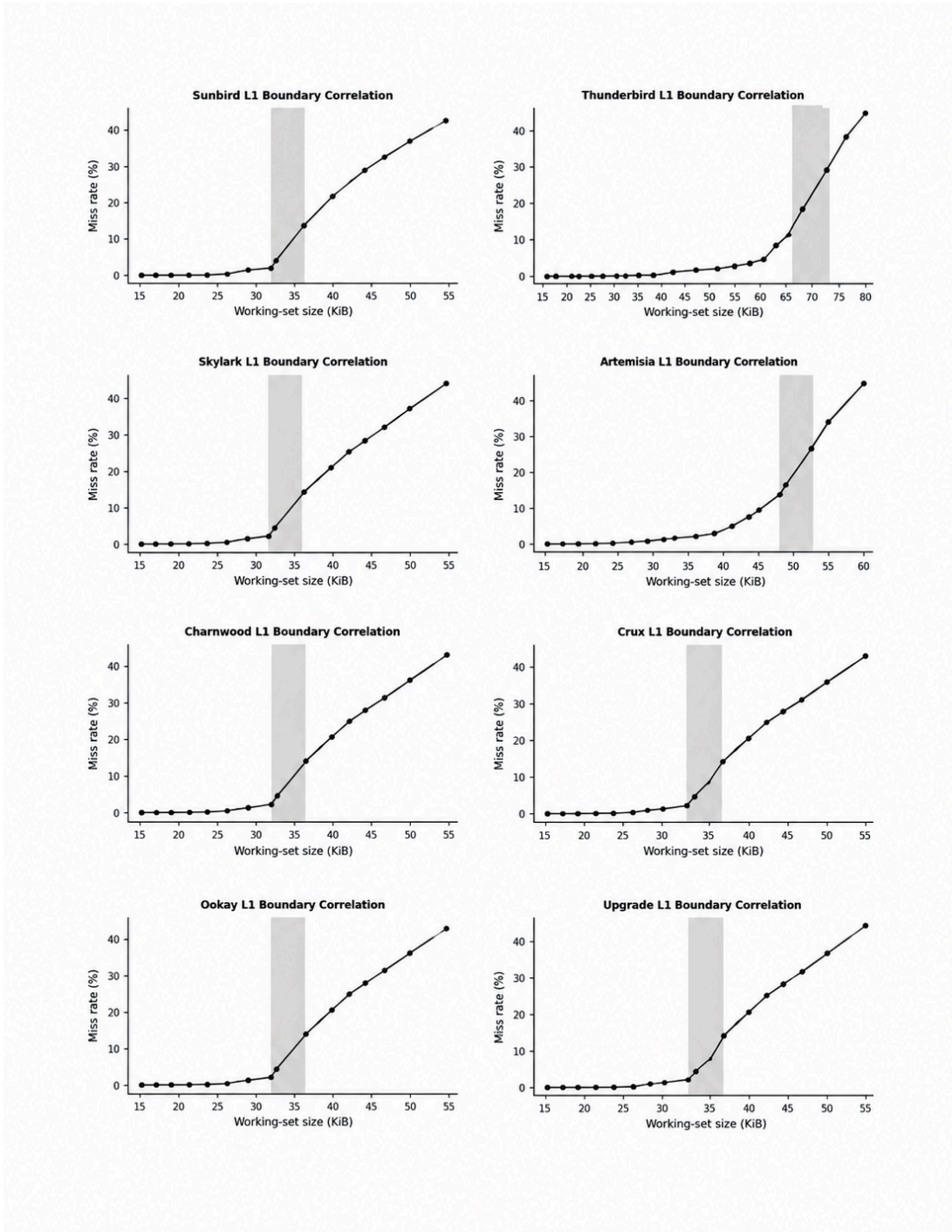


Fig. 17: Cross-machine, cross-regime normalized counter comparison. *Primary contributor: Justin Kuethe.*

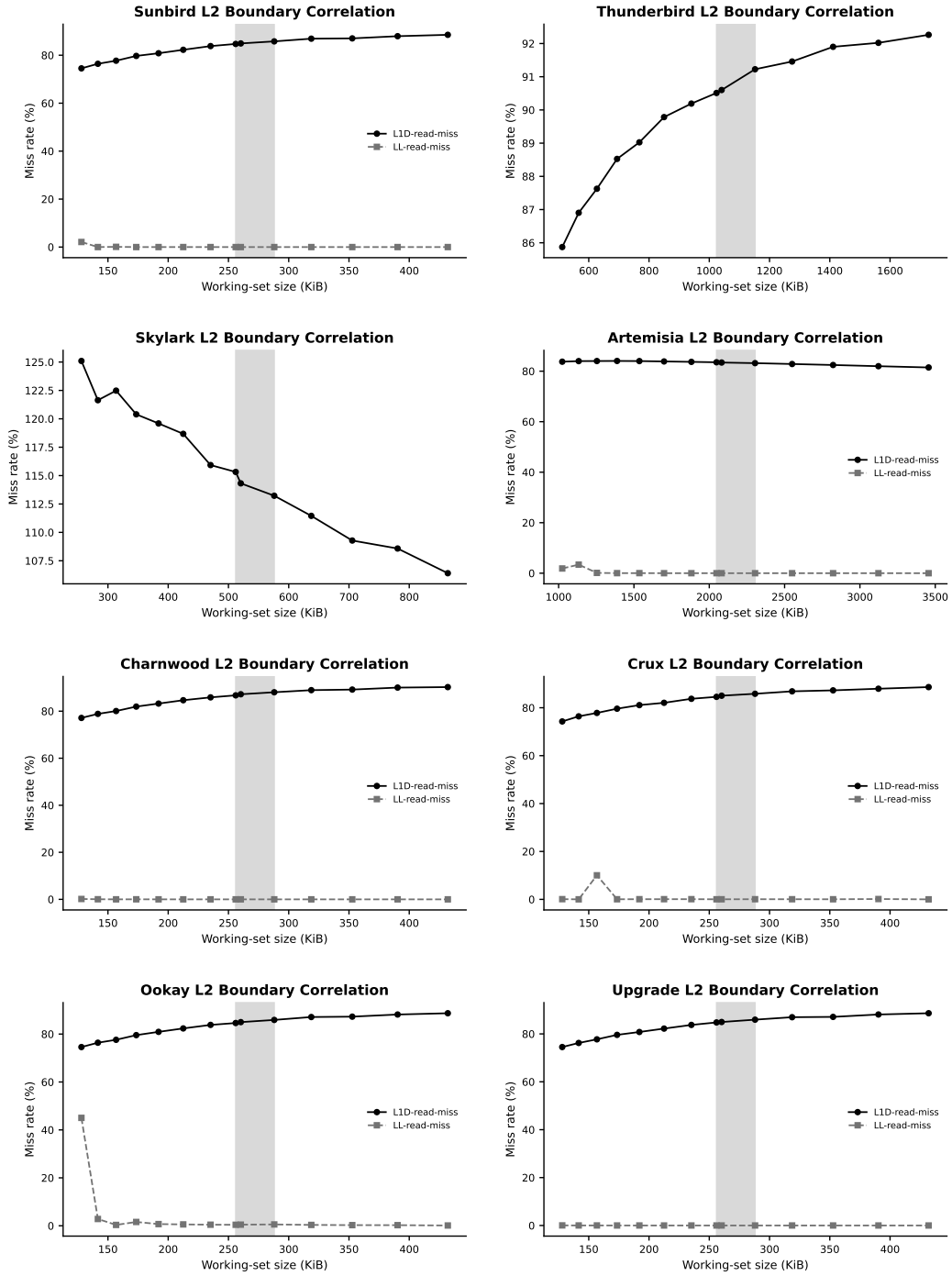


Fig. 18: Cross-machine, cross-regime normalized counter comparison. *Primary contributor: Justin Kuethe.*

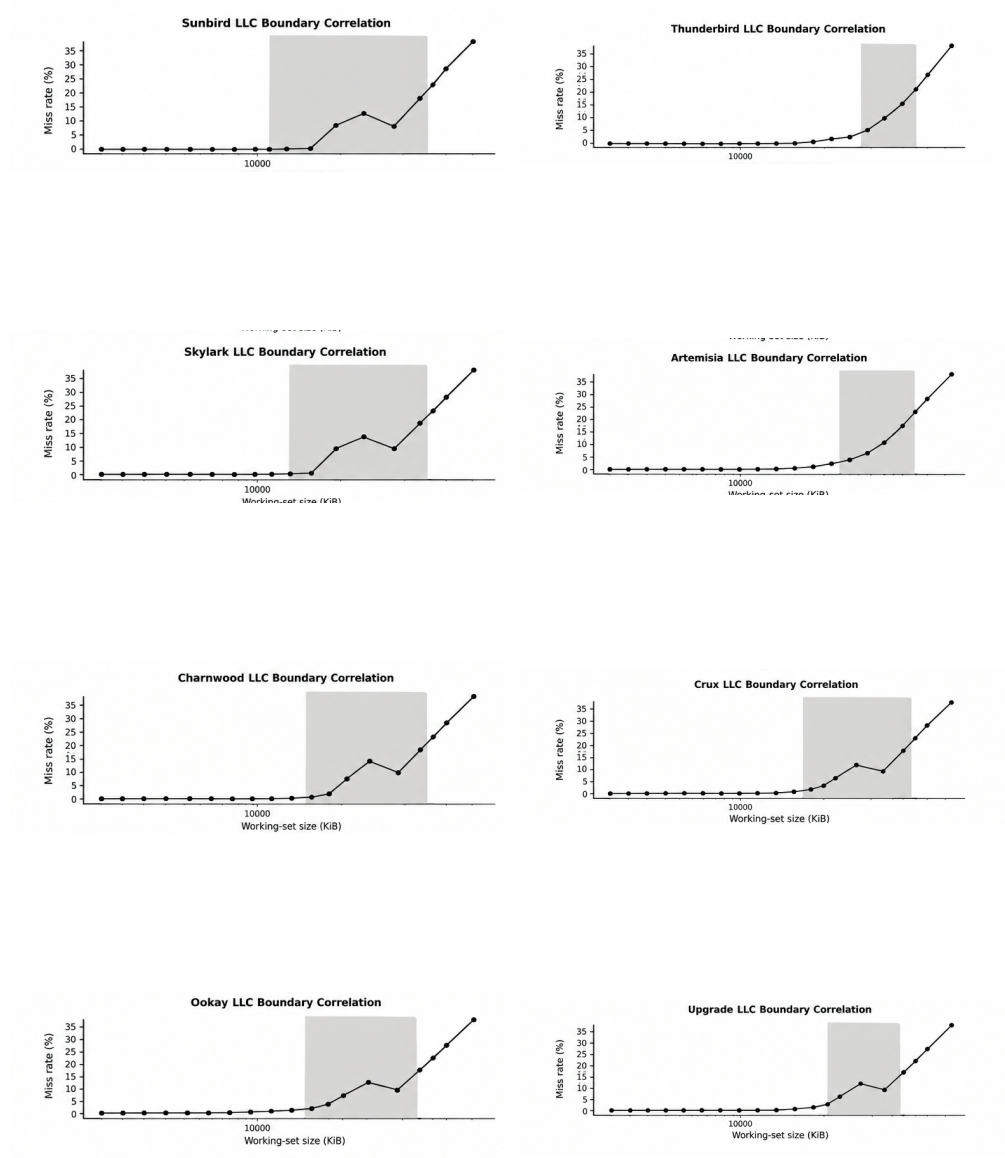


Fig. 19: Cross-machine, cross-regime normalized counter comparison. *Primary contributor: Justin Kuethe.*

TABLE VIII: Phase-II stride evidence: L1D-read-miss/L1D-read-access immediately below, at and above the 64 B timing candidate. These are the actual normalized PMU values; “–” denotes zero running time. *Primary contributor: Justin Kuethe.*

Machine	56 B	64 B	72 B	Assessment
Sunbird	–	–	–	unavailable
Thunderbird	0.4898	0.4831	0.5435	broad/no sharp edge
Skylark	0.7383	0.7264	0.7944	broad/no sharp edge
Artemisia	0.5202	0.4978	0.5600	broad/no sharp edge
Charnwood	0.6710	0.6077	0.6846	broad/no sharp edge
Crux	0.6529	0.6012	0.6752	broad/no sharp edge
Ookay	0.6404	0.5819	0.6229	broad/no sharp edge
Upgrade	0.6663	0.5870	0.6722	broad/no sharp edge

TABLE IX: Phase-II associativity evidence. Triples are miss/access at conflict counts $w - 1/w/w + 1$, where w is the frozen timing candidate. L1 uses L1D events and L2 uses generic LL events. “Edge” applies the same threefold separation used for the capacity-correlation summaries: the $w + 1$ rate exceeds three times the $w - 1$ rate. “–” denotes an unavailable event. *Primary contributor: Justin Kuethe.*

Machine	L1 w	L1 miss/access	L1 result	L2 w	L2 miss/access	L2 result
Sunbird	8	0.0081/0.0225/0.0890	edge	8	0.0222/0.0290/0.0001	no edge
Thunderbird	4	0.0010/0.0363/0.2611	edge	8	–/–/–	unavailable
Skylark	8	0.0027/0.0732/0.3841	edge	8	–/–/–	unavailable
Artemisia	12	0.0005/0.1535/0.4583	edge	16	0.2546/0.2784/0.2589	no edge
Charnwood	8	0.0094/0.1037/0.4307	edge	4	0.3946/0.4078/0.3872	no edge
Crux	8	0.0015/0.0600/0.4181	edge	4	0.5534/0.5793/0.4766	no edge
Ookay	8	0.0020/0.0648/0.5528	edge	4	0.2444/0.2501/0.2378	no edge
Upgrade	8	0.0018/0.0619/0.5694	edge	4	0.1864/0.2166/0.1984	no edge

TABLE X: L1 boundary-correlation overview, all eight machines. *Primary contributor: Justin Kuethe.*

Machine	Bracket (KiB)	Below-bracket max (%)	Within-bracket max (%)	Clean
Sunbird	32–36	1.61	14.31	Yes
Artemisia	48–52	1.20	8.82	Yes
Charnwood	32–36	8.48	23.85	No
Crux	32–36	0.28	14.04	Yes
Ookay	32–36	0.34	13.82	Yes
Skylark	32–36	0.69	19.02	Yes
Thunderbird	64–72	0.87	13.48	Yes
Upgrade	32–36	1.22	14.11	Yes

available generic counters, though still not L2-specific – see §VIII-F).

VIII-F. Why L2 Cannot Be Correlated With This Abstraction

No machine shows a step at its L2 bracket, and this is expected rather than a data-quality failure across any individual host. The Linux `PERF_TYPE_HW_CACHE` abstraction used by `pmu_stat` defines cache identifiers only for L1D, L1I, LL (last-level), DTLB, ITLB, BPU, and NODE – there is no generic L2 identifier. Consequently, no counter requested through this abstraction is L2-specific on any of the eight machines: `L1D-read-miss` reflects L1 residency, already largely saturated at these much-larger-than-L1 footprints (hence the smooth climb toward 80–90% seen on six of eight hosts), and `LL-read-miss` reflects last-level residency, still near zero at these much-smaller-than-LLC footprints. This is recorded as a measurement-coverage

limitation of the generic hardware-cache abstraction, applying uniformly across the machine set, rather than evidence against any host’s Phase-I L2 bracket.

VIII-G. Architecture-Dependent PMU Event Availability

Phase II validation revealed that not all project-defined performance-counter events were exposed consistently across the evaluated processors. Table XIII summarizes `LL-read-access`/`LL-read-miss` availability across all eight machines, drawn from the eight-counter acquisition batches.

On both non-Intel systems, `perf_event_open` failed for `LL-read-access` and `LL-read-miss` with an error indicating the event is not exposed on the PMU, while the same two events opened and ran successfully (100% running time) on all six Intel systems. By contrast, `DTLB-read-miss` and `cache-misses` were available and

TABLE XI: LLC boundary-correlation overview, all eight machines. *Primary contributor: Justin Kuethe.*

Machine	Bracket (MiB)	Below-bracket max (%)	Within-bracket max (%)	Clean
Sunbird	8–32	0.0019	18.12	Yes
Artemisia	24–26	0.1349	0.2178	No
Charnwood	4–8	0.9318	32.14	Yes
Crux	4–16	2.0678	60.35	Yes
Ookay	5–5.5	8.5081	14.99	No
Skylark	16–18	LL-* unavailable (§VIII-G)	—	—
Thunderbird	16–18	LL-* unavailable (§VIII-G)	—	—
Upgrade	2–16	0.0043	39.07	Yes

TABLE XII: L2 sweep overview, all eight machines: no clean transition is expected or observed at any host. *Primary contributor: Justin Kuethe.*

Machine	Bracket (KiB)	L1D-rate range across sweep	Pattern
Sunbird	256–288	74.5%–88.5%	Monotonic climb
Artemisia	2048–2304	83.8%–81.5%	Roughly flat/declining
Charnwood	256–288	77.1%–90.2%	Monotonic climb
Crux	256–288	74.3%–88.6%	Monotonic climb
Ookay	256–288	74.6%–88.7%	Monotonic climb
Skylark	512–576	125.1%–106.4%	Roughly flat (see §IX-C)
Thunderbird	1024–1152	85.9%–92.3%	Monotonic climb
Upgrade	256–288	74.5%–88.6%	Monotonic climb

TABLE XIII: LL-read-access/LL-read-miss availability through PERF_TYPE_HW_CACHE across all eight machines. *Primary contributor: Justin Kuethe.*

Machine	Processor	LL-* available
Sunbird	Intel Xeon E5-2680 v3	Yes
Artemisia	Intel Xeon Gold 5420+	Yes
Charnwood	Intel Core i7-6700	Yes
Crux	Intel Core i7-9700	Yes
Ookay	Intel Core i7-7700	Yes
Upgrade	Intel Core i7-8700	Yes
Skylark	AMD EPYC 7532 (Zen 2)	No
Thunderbird	Ampere Q80-30 (Neoverse N1)	No

measurable on every machine, including Skylark and Thunderbird. These two events were therefore treated as unavailable specifically on Skylark and Thunderbird rather than substituted with a different, potentially non-equivalent event, since a substitute would not necessarily count the same underlying population of accesses.

The pattern is a clean split along vendor lines rather than two unrelated per-machine quirks: the generic PERF_TYPE_HW_CACHE last-level-cache abstraction is fully supported by the kernel’s Intel PMU driver on every Intel generation tested (Haswell-EP through the Xeon Gold 5420+ generation), but is not implemented for either the AMD Zen 2 PMU driver or the Arm Neoverse N1 PMU driver in this environment. This demonstrates that PMU event availability through a generic, vendor-agnostic abstraction is itself architecture-dependent, and that a cross-generation study using this abstraction must treat LL-* comparisons as an Intel-only comparison, while DTLB-read-miss and cache-misses remain usable across all three vendor families represented in this study.

VIII-H. Post-Phase-I Validation Table

Table XIV reports the required post-freeze comparison across all eight machines, combining the preserved timing inferences, Phase-II PMU evidence and published organization. Published values never replace the measured brackets.

Seven machines show clean L1 boundary correlation and Charnwood is weaker. Four of six LL-capable machines show clean LLC correlation; Artemisia and Ookay remain ambiguous. Published L1/L2 capacities, conditional ways and 64 B granularity generally agree with the timing candidates, while the documented Thunderbird, Artemisia and Ookay LLC capacities lie outside their timing brackets. Timing-derived latency, sharing and inclusion fields that remain unestablished have no numerical error assigned.

VIII-I. PMU Scheduling and Experimental Limitations

Hardware-counter scheduling is an experimental constraint on this platform. Eight requested metrics cannot be assumed to form one schedulable hardware-counter group even when every event is individually recognized

TABLE XIV: Phase-II published validation for every ECE processor. Timing and published capacity units are shown in each entry; “w” and “s” mean ways and derived sets. Published set counts are derived only when capacity, way count and 64 B line size are documented. PMU boundary classifications come from Tables X–XII; conflict/stride classifications come from Tables VIII and IX. *Primary contributor: Justin Kuethe.*

Machine	Timing inference	PMU evidence	Published/system value	Source/page	Error/agreement
Sunbird	L1 32–36 KiB/8w/64s; L2 256–288 KiB/8w/512s; LLC 8–32 MiB; 64 B	L1 boundary clean; L2 event gap; LLC clean; L1 conflict edge; stride counters unscheduled	L1 32 KiB/8 ways/64s; L2 256 KiB/8 ways/512s; LLC 30 MiB; line 64 B	[8], Table 10.2, p.147; [15], CPU Specifications	L1/L2 bounds contain published; ways, derived sets and line match; LLC inside, scope unresolved
Thunderbird	L1 64–72 KiB/4w/256s; L2 1024–1152 KiB/8w/2048s; LLC 16–18 MiB; 64 B	L1 boundary clean; L2 event gap; LLC LL events unavailable; L1 conflict edge; stride broad	L1 64 KiB/4 ways/256s; L2 1024 KiB/8 ways/2048s; LLC 32 MiB; line 64 B	[11], Sections 2.3–2.5, p.9; [11], Section 2.5, p.9	L1/L2 bounds contain published; ways, derived sets and line match; LLC outside
Skylark	L1 32–36 KiB/8w/64s; L2 512–576 KiB/8w/1024s; LLC 16–18 MiB; 64 B	L1 boundary clean; L2 event gap; LLC LL events unavailable; L1 conflict edge; stride broad	L1 32 KiB/8 ways/64s; L2 512 KiB/8 ways/1024s; LLC 16 MiB; line 64 B	[8], Table 22.3, p.237; [12], HPC tuning guide, cache-domain diagram	L1/L2 bounds contain published; ways, derived sets and line match; LLC inside CCX, scope unresolved
Artemisia	L1 48–52 KiB/12w/64s; L2 2048–2304 KiB/16w/2048s; LLC 24–26 MiB; 64 B	L1 boundary clean; L2 event gap; LLC ambiguous; L1 conflict edge; stride broad	L1 48 KiB/12 ways/64s; L2 2048 KiB/not established/-s; LLC 52.5 MiB; line 64 B	[13], Table 1; [8], [9], Fog Table 13.1, p.174; Intel section 2.1 identifies Golden Cove; [13], Table 1 gives capacity only; [14], CPU Specifications; [8], [9], Fog Table 13.1, p.174; Intel section 2.1	L1/L2 bounds contain published; L1 ways/set and line match; L2 ways/set undocumented; LLC outside
Charnwood	L1 32–36 KiB/8w/64s; L2 256–288 KiB/4w/1024s; LLC 4–8 MiB; 64 B	L1 boundary weak; L2 event gap; LLC clean; L1 conflict edge; stride broad	L1 32 KiB/8 ways/64s; L2 256 KiB/4 ways/1024s; LLC 8 MiB; line 64 B	[9], Table 2-14, p.2-32; [16], i7-6700 row	L1/L2 bounds contain published; ways, derived sets and line match; LLC inside, scope unresolved
Crux	L1 32–36 KiB/8w/64s; L2 256–288 KiB/4w/1024s; LLC 4–16 MiB; 64 B	L1 boundary clean; L2 event gap; LLC clean; L1 conflict edge; stride broad	L1 32 KiB/8 ways/64s; L2 256 KiB/4 ways/1024s; LLC 12 MiB; line 64 B	[10], Core i7-8700K entries (family comparator); [17], CPU Specifications; [9], Table 2-14, p.2-32	L1/L2 bounds contain published; ways, derived sets and line match; LLC inside, scope unresolved
Ookay	L1 32–36 KiB/8w/64s; L2 256–288 KiB/4w/1024s; LLC 5–5.5 MiB; 64 B	L1 boundary clean; L2 event gap; LLC ambiguous; L1 conflict edge; stride broad	L1 32 KiB/8 ways/64s; L2 256 KiB/4 ways/1024s; LLC 8 MiB; line 64 B	[10], Core i7-7700 entries; [16], i7-7700 row; [9], Table 2-14, p.2-32	L1/L2 bounds contain published; ways, derived sets and line match; LLC outside
Upgrade	L1 32–36 KiB/8w/64s; L2 256–288 KiB/4w/1024s; LLC 2–16 MiB; 64 B	L1 boundary clean; L2 event gap; LLC clean; L1 conflict edge; stride broad	L1 32 KiB/8 ways/64s; L2 256 KiB/4 ways/1024s; LLC 12 MiB; line 64 B	[10], Core i7-8700K entries (family comparator); [16], i7-8700 row; [9], Table 2-14, p.2-32	L1/L2 bounds contain published; ways, derived sets and line match; LLC inside; refined timing edge differs

by Linux. The final procedure therefore uses small groups and verifies `time_running` for every group. Results with `time_running=0`, failed `perf_event_open` calls, or other incomplete scheduling are not interpreted as measurements.

Separate PMU runs also mean that counter groups are not sampled simultaneously. The comparison is therefore supported by the deterministic benchmark configuration, fixed seeds and unchanged Phase-I workload rather than by cycle-level simultaneity between counter groups. This distinction is preserved in the raw batch metadata.

The verified counter results establish the acquisition mechanism and provide completed L1, L2 and LLC boundary-correlation sweeps for all eight machines, summarized in §VIII-E and Table XIV.

IX. Eight-Counter Cross-Generation Study

IX-A. Study Design

The eight-counter study extends the verified Phase-II acquisition procedure to the complete laboratory machine set. Exactly eight project-defined metrics are retained:

cycles, instructions, L1D-read-access,
L1D-read-miss, LL-read-access, LL-read-miss,
DTLB-read-miss, cache-misses.

The measurements are collected in four two-event groups to avoid the PMU scheduling failures observed when larger groups are used.

Each processor is evaluated with the three standardized working regimes specified for the cross-generation study: an L1-resident working set, an LLC-sized working set, and a working set substantially larger than the LLC. The exact footprint values are taken from the standardized acquisition plan rather than selected from the PMU results.

The same stride, batch, sample count, randomization and CPU-binding policy are retained for a given host.

The final dataset preserves both the raw event counts and explicitly normalized values. Normalization is performed only after confirming the denominator required by the standardized workload definition, rather than assuming that all counters represent the same kind of access. In particular, cycles and instructions are retained as execution counters, while cache and translation events are normalized using the workload access denominator specified by the experiment.

IX-B. Cross-Machine Comparison

All eight machines have completed acquisition for all three standardized workloads. Values below are normalized per dependent access using the workload’s own known access count (samples $\times$ batch size), rather than as a ratio of two independently-measured PMU counters against each other – the latter is avoided because, as discussed in §20 below, two counters drawn from the same generic abstraction do not always count strictly nested populations.

20

Table XVIII supplies the required ranking for every event and workload. It is computed from the same per-dependent-access dataset shown above; no unavailable LL event is imputed.

Two observations follow directly from the L1-resident table. First, Artemisia and Thunderbird – the newest Intel generation tested and the sole Arm system – show L1D miss rates roughly two orders of magnitude lower than the other six machines at this footprint (0.02–0.04% vs. 7.4–20.8%), which is a substantially larger gap than the generation-to-generation differences among the remaining six Intel machines. Second, DTLB-read-miss/access is comparable in magnitude across all eight machines at

TABLE XV: L1-resident workload: selected per-access-normalized counters. *Primary contributor: Justin Kuethe.*

Machine	Processor	L1D-miss/acc.	LL-miss/acc.	DTLB-miss/acc.	Cache-miss/acc.
Artemisia	Xeon Gold 5420+	0.000235	0.000001	0.000001	0.000002
Thunderbird	Ampere Q80-30	0.000406	unavail.	0.000008	0.000472
Upgrade	Core i7-8700	0.093123	0.000005	0.000001	0.000380
Ookay	Core i7-7700	0.093658	0.000114	0.000001	0.000637
Sunbird	Xeon E5-2680 v3	0.090941	0.000001	0.000001	0.000007
Crux	Core i7-9700	0.100486	0.000027	0.000001	0.000379
Skylark	AMD EPYC 7532	0.120159	unavail.	0.000004	0.000029
Charnwood	Core i7-6700	0.208420	0.000056	0.000030	0.000596

TABLE XVI: LLC-sized workload: selected per-access-normalized counters. *Primary contributor: Justin Kuethe.*

Machine	Processor	L1D-miss/acc.	LL-miss/acc.	DTLB-miss/acc.	Cache-miss/acc.
Thunderbird	Ampere Q80-30	1.022780	unavail.	1.012648	1.023421
Skylark	AMD EPYC 7532	1.050438	unavail.	0.616234	1.029886
Artemisia	Xeon Gold 5420+	1.440995	0.000456	0.660693	0.002370
Crux	Core i7-9700	1.561330	0.723613	0.712663	2.194494
Upgrade	Core i7-8700	1.563545	0.483900	0.714020	1.413563
Ookay	Core i7-7700	1.565241	0.781558	0.713681	2.358107
Sunbird	Xeon E5-2680 v3	1.621880	0.040909	0.815788	0.016805
Charnwood	Core i7-6700	1.761708	0.921950	0.857314	2.702377

TABLE XVII: Larger-than-LLC workload: selected per-access-normalized counters. *Primary contributor: Justin Kuethe.*

Machine	Processor	L1D-miss/acc.	LL-miss/acc.	DTLB-miss/acc.	Cache-miss/acc.
Thunderbird	Ampere Q80-30	1.139046	unavail.	1.100530	1.138764
Skylark	AMD EPYC 7532	1.162142	unavail.	1.034703	1.565392
Artemisia	Xeon Gold 5420+	2.074302	0.720816	1.006892	0.759501
Sunbird	Xeon E5-2680 v3	2.170758	0.896094	1.035312	0.971311
Crux	Core i7-9700	2.177075	1.055958	1.041804	3.232185
Ookay	Core i7-7700	2.174315	1.071594	1.042502	3.270327
Upgrade	Core i7-8700	2.179194	1.017924	1.042363	3.072257
Charnwood	Core i7-6700	2.236582	1.087072	1.070409	3.359072

TABLE XVIII: High-to-low machine ranking for all eight normalized counters and all three workloads. Abbreviations: Ar=Artemisia, Ch=Charnwood, Cr=Crux, Oo=Ookay, Sb=Sunbird, Sk=Skylark, Th=Thunderbird, Up=Upgrade. LL rankings contain only the six machines exposing those events. *Primary contributor: Justin Kuethe.*

Normalized event	L1-resident	LLC-sized	Larger than LLC
cycles	Ch>Cr>Up>Oo>Sk>Ar>Sb>Th	Cr>Oo>Sk>Up>Ch>Th>Ar>Sb	Sk>Cr>Up>Oo>Th>Sb>Ar>Ch
instructions	Cr>Up>Oo>Sk>Ch>Ar>Sb>Th	Th>Up>Sb>Sk>Oo>Cr>Ch>Ar	Th>Up>Sk>Oo>Cr>Sb>Ch>Ar
L1D-read-access	Up>Cr>Sk>Oo>Sb>Ch>Ar>Th	Sk>Th>Sb>Ar>Up>Cr>Ch>Oo	Sk>Th>Sb>Cr>Up>Ar>Ch>Oo
L1D-read-miss	Ch>Sk>Cr>Oo>Up>Sb>Th>Ar	Ch>Sb>Oo>Up>Cr>Ar>Sk>Th	Ch>Up>Cr>Oo>Sb>Ar>Sk>Th
LL-read-access	Ch>Oo>Up>Cr>Sb>Ar	Ch>Cr>Up>Oo>Sb>Ar	Ch>Cr>Oo>Up>Sb>Ar
LL-read-miss	Oo>Ch>Cr>Up>Ar>Sb	Ch>Oo>Cr>Up>Sb>Ar	Ch>Oo>Cr>Up>Sb>Ar
DTLB-read-miss	Ch>Th>Sk>Sb>Up>Cr>Oo>Ar	Th>Ch>Sb>Up>Oo>Cr>Ar>Sk	Th>Ch>Oo>Up>Cr>Sb>Sk>Ar
cache-misses	Oo>Ch>Th>Up>Cr>Sk>Sb>Ar	Ch>Oo>Cr>Up>Sk>Th>Sb>Ar	Ch>Oo>Cr>Up>Sk>Th>Sb>Ar

this footprint (all $\leq 3 \times 10^{-5}$ per access), making it one of the more portable normalized metrics in this study, consistent with §VIII-G’s finding that DTLB-read-miss is also the more *available* of the two translation/cache-adjacent events across vendors.

IX-C. A Generic-Event Semantic Caveat

At the LLC-sized and larger-than-LLC regimes, several machines report L1D-read-miss/access values *greater than 1.0* (e.g. Sunbird: 1.62 at LLC-sized, 2.17 at larger-than-LLC) – more counted misses than dependent accesses in the workload. This was verified not to be a multiplexing artifact: the corresponding batches show $T_{\text{running}} = T_{\text{enabled}}$

(100% scheduled) in every case. The raw counts themselves are internally consistent (L1D-read-miss > L1D-read-access is what `pmu_stat` actually read from the PMU), which indicates that on this microarchitecture, the generic `PERF_TYPE_HW_CACHE` mapping for L1D-read-access and L1D-read-miss do not necessarily count a strictly nested access/miss pair from the same event population – for example, the underlying raw event chosen by the kernel for “access” may undercount relative to a fill- or replacement-based “miss” event at large working-set sizes. This is recorded here as an explicit, unresolved disagreement in event semantics rather than clipped, hidden, or silently renormalized, consistent with the requirement that uncer-

tain or contradictory PMU evidence remain visible in the analysis.

IX-D. Interpretation Requirements

Counter evidence is treated as corroboration of timing observations rather than as a replacement for the Phase-I inference procedure. A timing boundary that coincides with an increasing miss-count transition provides stronger evidence for an effective residency transition, but it does not by itself establish physical cache capacity, associativity, inclusion policy or cache sharing domain. Conversely, disagreement between timing and counter behavior is retained as a result requiring analysis rather than discarded.

The final cross-generation study distinguishes among three cases, each represented in the cross-machine results above: agreement between a timing transition and the expected counter transition (seven of eight machines at L1, four of six LL-*capable machines at LLC; §VIII-E); counter evidence that cannot address the question due to a coverage gap in the abstraction rather than a physical disagreement (L2, uniformly across all eight machines; §VIII-F); and an unresolved measurement-semantics disagreement flagged rather than concealed (the L1D mis-/access ratio exceeding 1.0 at larger footprints, §IX-C). This distinction follows the Phase-I requirement that uncertain measurements remain visible and that unsupported physical interpretations not be substituted for missing evidence.

X. Software-Only Estimator: PMU Comparison

For §8.5.4, we ran an additional matched comparison on all eight ECE machines, preserving the Phase-I benchmark and calibration rule. Each point used 10^6 single-load intervals, random order, 8-byte spacing and seed 5922026. The 1 KiB/256 MiB timing-only controls set the threshold; the 1 MiB workload tests the estimator. Thresholds use no counter values. The original Phase-I estimates remain unchanged.

A separate Phase-II preload observer, `region_pmu.c`, enables paired user-mode access/miss counters at the first `getrusage` call and disables them at the second. These existing calls bracket the frozen benchmark’s sampling loop, excluding allocation, warm-up and file output. The estimator still classifies only timings. L1D pairs worked on all eight hosts; LL pairs were unavailable on Skylark and Thunderbird and were not retried after the smoke tests. All acquired groups ran without multiplexing. Verification checked 66 million full-run intervals, raw hashes and statistics after 24 smoke points; failed event openings and context switches remain recorded.

For each available reference, $H_{\text{PMU}} = 100(1 - M/A)$, where A and M are the paired region counts. We report $E_{\text{abs}} = |\hat{H} - H_{\text{PMU}}|$ in percentage points and $E_{\text{rel}} = 100E_{\text{abs}}/H_{\text{PMU}}$ in percent; a zero denominator leaves relative error undefined. Table XIX uses timings

from the same run as its counters. The archive also reports all three footprints, threshold-sensitivity ranges, 10,000-sample block ranges and differences from unobserved timing runs. There is one full run per configuration, not independent repeated trials. Automatic idle-core reselection caused the marked calibration/core mismatches, which remain qualified rather than hidden.

The comparison does not validate a universal hardware hit-rate estimator. The timed-region counters include timer, loop and stack traffic as well as pointer loads; LL hit rates are conditional on the LL access-event population, not unconditional hits anywhere in the hierarchy. Generic access/miss mappings can also differ by processor. Consequently, the cold control can have few timing-classified hits but many L1 region hits (Fig. 21). Coarse timer quantization, threshold sensitivity, interference and core changes add error. Small numerical differences therefore do not prove correct individual-load labels; large differences document failure to reproduce the named reference.

The isolated `estimator_validation/` package contains the observer, acquisition/analysis scripts, commands, source hashes, compiler flags, CPU/topology and idle evidence, all raw archives and comparison CSVs.

XI. Phase III: Completed Hazel and Prediction Results

XI-A. Hazel execution and evidence

We completed Haswell, Cascade Lake, Ice Lake-SP (8358), Genoa and Turin: five distinct generations from 2014–2024, including two AMD generations. Table XX identifies the allocated processors and baseline bindings; Ice Lake 6326 and 8358 are one generation, not two. Broadwell, Skylake-SP and Sapphire Rapids were excluded at the team’s direction; the five-generation minimum alone does not establish the specification’s all-accessible-generation target.

The native Phase-I implementations in Listings 1–4 were reused unchanged. Slurm jobs inherited their CPU allocation, selected an idle allowed core from repeated utilization observations, and recorded any subsequent core changes. Cross-core pressure used distinct physical cores on the same socket. Each full tested point contained 10^6 timed samples; smaller smoke runs preceded full acquisition. The completed analysis verified 1,612 baseline and 3,606 follow-up points. All 211 noisy follow-up points and unsuccessful attempts were retained with qualification records. Commands, seeds, compiler/version/flags, source hashes, timestamps, affinity/topology, job logs and raw-data hashes accompany the repository package; Table XX is a compact execution index rather than an exhaustive job log.

Figures 22–25 show the controlled sweeps and sample distributions supporting Table XXI. The smallest timing class and repeated conflict edges support L1-like geometry. Only Haswell has a supported lower-level way

TABLE XIX: Matched 1 MiB estimator/reference comparisons. SW, sensitivity range and PMU are percentages; the range varies the timing threshold between hot P95 and cold P5 and is not a confidence interval; absolute error is percentage points (pp), relative error is percent. “Same core” means both timing-calibration controls and the observed run used one logical CPU; it does not establish equivalent event populations. “Core change” retains a calibration/core mismatch. *Contributors: Hong Chan Lee and Justin Kuethe.*

Machine	Reference	SW (%)	Sensitivity (%)	PMU (%)	Abs. error (pp)	Rel. error (%)	Calibration
artemisia	L1D	97.693	29.075–98.866	96.426	1.267	1.314	same core
artemisia	LL	98.104	30.416–98.779	44.104	53.999	122.435	core change
charnwood	L1D	99.997	1.274–99.997	95.915	4.082	4.256	same core
charnwood	LL	99.995	1.187–99.995	99.984	0.010	0.010	same core
crux	L1D	99.203	2.014–99.681	95.914	3.288	3.428	same core
crux	LL	99.172	2.007–99.666	98.935	0.237	0.239	same core
ookay	L1D	99.179	0.970–99.519	95.913	3.267	3.406	same core
ookay	LL	99.093	0.927–99.462	98.832	0.261	0.264	core change
skylark	L1D	99.839	74.381–99.988	97.180	2.659	2.737	same core
skylark	LL	–	–	–	–	–	unavailable
sunbird	L1D	76.571	0.719–99.985	95.913	19.341	20.166	same core
sunbird	LL	77.611	0.749–99.811	99.999	22.389	22.389	same core
thunderbird	L1D	67.157	67.157–99.141	96.755	29.598	30.591	same core
thunderbird	LL	–	–	–	–	–	unavailable
upgrade	L1D	99.188	1.128–99.505	95.913	3.276	3.415	core change
upgrade	LL	99.174	1.117–99.499	98.918	0.256	0.259	core change

TABLE XX: Hazel baseline execution index. Binding entries are logical CPU/physical core/socket/NUMA node; multiple entries indicate cores used across points, not simultaneous execution of one dependency chain. Full baseline/follow-up job histories are retained in the metadata package. *Primary contributor: Hong Chan Lee.*

Constraint	Exact allocated CPU model	Host	Baseline job IDs	CPU/core/socket/NUMA
haswell	Xeon E5-2650 v3 @ 2.30GHz	c207n01	810638	10/10/1/1; 11/11/1/1
cascadelake	Xeon Gold 6226R @ 2.90GHz	c020n02	810641	25/25/1/1; 26/26/1/1
icelake_8358	Xeon Platinum 8358 @ 2.60GHz	c059n01	810646	0/0/0/0; 32/32/1/1
genoa	AMD EPYC 9654 96-Core	n0398	810648,810673,811148	0/0/0/0; 96/96/1/1
turin	AMD EPYC 9655 96-Core	n0405	810649	80/80/0/0

TABLE XXI: Frozen Hazel timing inferences. Capacities are sweep brackets, not confidence intervals or rounded hardware specifications. Ways are conditional conflict/capacity candidates. L1-like latency uses the matched 64-byte-spacing representative and its empirical timer frequency; “–” means unresolved. Every host shows a 64-byte spatial boundary; every global inclusion classification remains uncertain. *Primary contributor: Hong Chan Lee.*

Generation	L1 KiB	L1 ways	L2-like KiB	L2 ways	LLC-like MiB	L1-like ns/access	Residency proxy (%)
Haswell	32–36	8	256–288	8	16–128	1.591	89.4925
Cascade Lake	32–36	8	–	–	–	–	99.9255
Ice Lake 8358	48–52	12	–	–	–	2.036	85.5640
Genoa	32–36	8	1024–1152	–	8–64	2.113	98.4675
Turin	48–52	12	1024–1152	–	32–64	1.252	99.9696

candidate; Cascade Lake and Ice Lake have no resolved final deeper-capacity bracket. Broad LLC-like intervals can mix cache, translation, replacement and memory effects. Physical cache-level counts and isolated L2/LLC hit and miss penalties are consequently not supplied. Cross-core pressure does not establish lower-only eviction or independently verify upper-cache residency, so it cannot settle global inclusion policy.

All 30 planned independent-load pairs qualified across the five generations; dependent/independent median ratios range from 3.905 to 3.953. These are throughput comparisons, not individual-load latency estimates. Functional, sample-count, affinity and noise checks remained active.

XI-B. Chronological dataset and unchanged predictions

Table [XXII](#) combines eight ECE and five Hazel machines by processor-generation introduction year, not installation year or individual SKU refresh date. The complete CSV adds vendor/ISA, all inferred cache fields, brackets, conditional ways, line size, inclusion, residency uncertainty and source references. Capacity midpoints are geometric summaries of the original brackets for plotting and scoring; the brackets and original geometry candidates are preserved. No published value replaces a timing inference.

Forecasts used only ECE observations; Hazel measurements were neither fitted nor used to adjust parameters. The server subset was Sunbird, Skylark, Thunderbird

TABLE XXII: Chronological timing dataset, compact view. L1/L2/LLC entries are geometric bracket midpoints (KiB/KiB/MiB); they are not new physical-cache identifications. Latency is the matched L1-like dependent median in ns/access. Process nodes are published background metadata; chiplet CPU and I/O processes are distinguished. *Primary contributor: Hong Chan Lee.*

Year	Machine	Generation	Process	L1 KiB	L2 KiB	LLC MiB	ns/access
2014	sunbird	Haswell	22 nm	33.94	271.5	16.00	1.653
2014	haswell	Haswell	22 nm	33.94	271.5	45.25	1.591
2015	charnwood	Skylake	14 nm	33.94	271.5	5.66	1.535
2017	ookay	Kaby Lake	14 nm	33.94	271.5	5.24	1.011
2017	upgrade	Coffee Lake	14 nm	33.94	271.5	5.66	0.921
2018	crux	Coffee Lake Refresh	14 nm	33.94	271.5	8.00	0.902
2019	skylark	Zen 2 / Rome	7 nm	33.94	543.1	16.97	1.272
2019	cascadelake	Cascade Lake	14 nm	33.94	—	—	—
2020	thunderbird	Neoverse N1 / Altra	TSMC N7	67.88	1086.1	16.97	1.367
2021	icelake_8358	Ice Lake-SP	10 nm	49.96	—	—	2.036
2022	genoa	Zen 4 / Genoa	5 nm CCD / 6 nm IOD	33.94	1086.1	22.63	2.113
2023	artemisia	Sapphire Rapids	Intel 7	49.96	2172.2	24.98	2.625
2024	turin	Zen 5 / Turin	4 nm CCD / 6 nm IOD	49.96	1086.1	45.25	1.252

TABLE XXIII: Unchanged L2-like forecasts (KiB). The interval is the original heuristic 95% model interval. Error is absolute relative error against the timing-bracket midpoint; missing observations are not scored. *Primary contributor: Hong Chan Lee.*

Target	Prediction	Model interval	Error (%)
Haswell	232.1	[80.7, 667.2]	14.52
Cascade Lake	804.0	[340.2, 1899.9]	—
Ice Lake 8358	1321.5	[582.8, 2996.7]	—
Genoa	1694.3	[755.2, 3801.3]	56.00
Turin	2785.0	[1241.3, 6248.3]	156.42

and Artemisia (2014–2023), pooled across Intel, AMD and Arm. The saved model, prediction table and dashed-curve samples remain byte-identical. Only Turin (2024) tests future-year extrapolation beyond the newest training system; the other Hazel targets test transfer to unseen machines within the training-year range. Solid connections in Fig. 28 remain within vendor/ISA groups; dashed curves start at 2023, and distinct markers reveal Hazel.

The L2-like model is specified below. Other numerical forecasts are constant server summaries: L1-like capacity 41.951 KiB (empirical envelope 32–72), L1 ways 8 (4–12), L1-like latency 1.510 ns/access (1.272–2.625), L2 ways 8 (8–16), LLC-like capacity 16.971 MiB (8–32), and the residency proxy 99.90095% (threshold-sensitivity envelope 0.7233–99.9988%). Spatial granularity is 64 B. These empirical envelopes are not statistical prediction intervals. No numerical forecast is supplied for unsupported physical hit/miss levels, LLC ways or exact sharing normalization; inclusion remains categorical “uncertain.”

Table XXIV joins the frozen L2-like forecast to the later Hazel observation for each generation. Cascade Lake and Ice Lake remain unscored because no supported L2-like timing bracket was measured.

Spatial granularity is the strongest prediction: all five Hazel values match. The eight-way L1 point forecast misses the twelve-way Ice Lake and Turin candidates by four ways. L1 capacity varies stepwise rather than following a smooth increase, and L1-like latency is non-

monotonic. Although the four-server L2 trend resembles Moore-style exponential growth, its transfer to newer hardware is weak. LLC bounds are too broad to establish precise growth, and physical hit/miss penalties are insufficiently isolated to quantify a capacity–latency tradeoff. Year, vendor/ISA, server/desktop design, topology and memory platform are confounded in this small sample; the plots do not establish which causes the observed differences.

The residency estimator uses the same 1 MiB workload on all thirteen machines. Large threshold-sensitivity spans explain why range overlap can coexist with poor point predictions; this is not evidence of equal hardware hit rates. Figure 29 adds two Phase-II generic-event series for the L1-resident regime. Both use counts per dependent access. Generic kernel mappings and different working-set sizes limit cross-architecture comparability, so neither curve is treated as a hardware hit/miss probability or fitted law. The larger-footprint semantic disagreement discussed in Section IX-C remains unchanged.

XI-C. Published Hazel organization

Published comparisons were added after the Hazel timing results were frozen. Intel’s generation overview and SKU tables establish the per-core and socket capacities; AMD documentation distinguishes a CCD’s shared LLC from the package aggregate [2], [3], [4], [5], [6], [7]. Agner Fog supplies corroborating core-geometry information where applicable [8]. Table XXVI, read with Tables XXI and XXIII, keeps inference, prediction and documentation separate; the repository also provides a direct three-way table with per-field source locators and errors. The AMD package total is 384 MiB, but its 32 MiB CCD domain is the relevant conditional comparison for a pinned core. Haswell, Genoa and Turin have L2-like brackets containing their documented capacities; Cascade Lake and Ice Lake lack a resolved timing bracket and are not counted as agreement. Published inclusion categories cannot repair the unresolved timing experiment. Figure 30 shows normalization by published domain core count, explicitly

TABLE XXIV: Required frozen-prediction/Hazel-reveal summary for the modeled L2-like capacity. Values are point [lower, upper] in KiB; prediction ranges are heuristic model intervals and measured ranges are timing brackets. The signed direction in the assessment compares point estimates. *Primary contributor: Hong Chan Lee.*

Target generation	Frozen prediction	Hazel measured	Accuracy	Explanation
Haswell (2014)	232.1 [80.7, 667.2]	271.5 [256, 288]	14.5% low	Point is below the bracket; broad model interval overlaps.
Cascade Lake (2019)	804.0 [340.2, 1899.9]	unresolved	not scored	No accepted Hazel L2-like bracket; not scored.
Ice Lake-SP (2021)	1321.5 [582.8, 2996.7]	unresolved	not scored	No accepted Hazel L2-like bracket; not scored.
Genoa (2022)	1694.3 [755.2, 3801.3]	1086.1 [1024, 1152]	56.0% high	Point is above the bracket; broad model interval overlaps.
Turin (2024)	2785.0 [1241.3, 6248.3]	1086.1 [1024, 1152]	156.4% high	Future extrapolation overpredicts; model interval and bracket are disjoint.

TABLE XXV: Prediction accuracy across supported Hazel observations. MARE is mean absolute relative error, $100|\hat{y} - y|/y$. Capacity scores use geometric bracket midpoints; full CSVs also give endpoint-relative errors. Overlap compares the saved model range with timing brackets, latency P5–P95, or classifier sensitivity, as appropriate; these ranges have different meanings. *Primary contributor: Hong Chan Lee.*

Quantity	Scorable	MARE (%)	Range overlap	Largest point error (%)
l1_capacity	5/5	20.57	5/5	haswell (23.60%)
l1_associativity	5/5	13.33	5/5	icelake_8358 (33.33%)
l1_latency	4/5	20.02	3/4	genoa (28.52%)
l2_capacity	3/5	75.65	2/3	turin (156.42%)
l2_associativity	1/5	0.00	1/1	haswell (0.00%)
llc_capacity	3/5	50.00	3/3	haswell (62.50%)
line_size	5/5	0.00	5/5	haswell (0.00%)
software_metric	5/5	5.99	5/5	icelake_8358 (16.76%)

TABLE XXVI: Published cache capacities and sharing domains, separate from timing inferences. L1/L2 are KiB per core; LLC is MiB per listed domain. All five documented L1 capacities agree with the timing candidates; broad deeper-bracket containment is weaker evidence. *Primary contributor: Hong Chan Lee.*

Target	L1	Ways	L2	LLC	Domain (cores)
Haswell	32	8	256	25	socket (10)
Cascade Lake	32	8	1024	22	socket (16)
Ice Lake 8358	48	12	1280	48	socket (32)
Genoa	32	8	1024	32	CCD (8)
Turin	48	12	1024	32	CCD (8)

conditional on correspondence between the measured and documented domains.

XI-D. Two scoped cache laws and the 2029 forecast

Kueth–Lee L2 Capacity Law. Across the four ECE servers, the timing-derived L2-like bracket midpoint approximately doubles every 2.79 years:

$$\log_2 C_{\text{KiB}} = \log_2(2172.232032) + 0.358490566(y - 2023).$$

The anchored fit has $R^2 = 0.925$; unanchored leave-one-out RMSE is $0.682 \log_2$ KiB versus 1.491 for a constant model. This is a pooled, four-observation empirical rule, not an AMD-specific or universal physical-cache law. The Haswell point forecast is below its bracket and Genoa is overpredicted, although their model intervals overlap the measured brackets. Turin is overpredicted by 156.42% relative to the bracket midpoint, and its bracket lies outside the saved heuristic 95% interval. This fails the

2024 future-year test and weakens transferability without changing the original descriptive fit.

Kueth–Lee Spatial Granularity Law. The smallest timing-visible boundary is approximately constant: $B(y) = 64$ bytes. Eight ECE observations establish the scoped rule; all five Hazel observations, including Turin, support it. A constant observed range is not a guarantee for unmeasured designs or a separate line-size measurement at each level. This is the behavior law because the verified data do not support an isolated deeper-cache latency-cost law.

Capacity growth faces SRAM density, leakage, access energy, wire delay and banking/interconnect limits; die-area budgets, SKU choices and acceptable hit latency impose additional economic/design limits. A piecewise or flattening trend is more plausible than indefinite exponential growth. Fixed spatial granularity balances reuse and tag overhead against false sharing and wasted bandwidth; changing coherence or sector organization could produce a design-driven step. Neither dataset locates a numerical scaling wall or its year. Unlike Moore’s component-count observation, these heterogeneous timing brackets support limited empirical rules rather than a universal scaling claim.

Turin’s generation year is 2024, so the existing 2029 forecast meets the five-year horizon: L2-like capacity 9.421 MiB with heuristic 95% interval 3.737–23.752 MiB, and spatial boundary 64 B. Figure 32 retains the original model despite its failed Turin test; chiplet organization, stacking, sharing and design goals could invalidate the

projection. No Hazel-informed refit is substituted.

XI-E. Reproducibility and remaining limits

The curated `phase3/` repository package contains the unchanged lab models/predictions, combined tables, required plots, native source snapshot, Slurm drivers, selected statistics, metadata and data-location/hash manifests. Analysis job 817293 verified 78 output hashes, 31 numerical comparisons and 13 chronological rows. The report figures are regenerated only from saved verified statistics, with source-record selection documented. Raw arrays and unsuccessful attempts remain in the external Hazel archive/workspace identified by the package; they are not duplicated in Git.

XII. Discussion and Conclusion

Timing evidence supports reproducible spatial granularity and conditional upper-level geometry through conflict/capacity agreement. Broad or inconsistent deeper regions, failed upper-only calibration and unqualified paired-core comparisons leave physical fields unresolved despite additional measurements. The resulting candidates remain traceable; completing acquisition does not establish physical identity.

The independent-load diagnostic confirms the latency/throughput distinction on all eight hosts. The software proxy remains strongly threshold-dependent and lacks hardware hit-rate validation; physical next-level penalties require successful residency calibration. Section **XI** evaluates chronological patterns, Hazel prediction accuracy and the two cache laws; isolated physical penalties remain insufficient to establish a capacity–latency tradeoff.

AI/LLM Assistance Disclosure

Codex assisted with benchmark/workflow implementation, debugging, acquisition automation, raw verification, analysis/plotting and report drafting/layout.

Appendix

A. Team Members and GitHub Accounts

B. Result and Data-Point Attribution

C. Team Meeting and Coordination Log

The team held no formal meetings or synchronous work sessions. Coordination occurred asynchronously through shared GitHub and Overleaf updates and direct project messages.

D. Code Contribution

E. Experimental-Running Contribution

Dates in Table **XXX** are ranges established from Git history and retained run provenance.

F. Plotting and Writing Contribution

G. Analysis and Interpretation Contribution

H. AI and LLM Assistance Disclosure

I. Final Contribution Confirmation

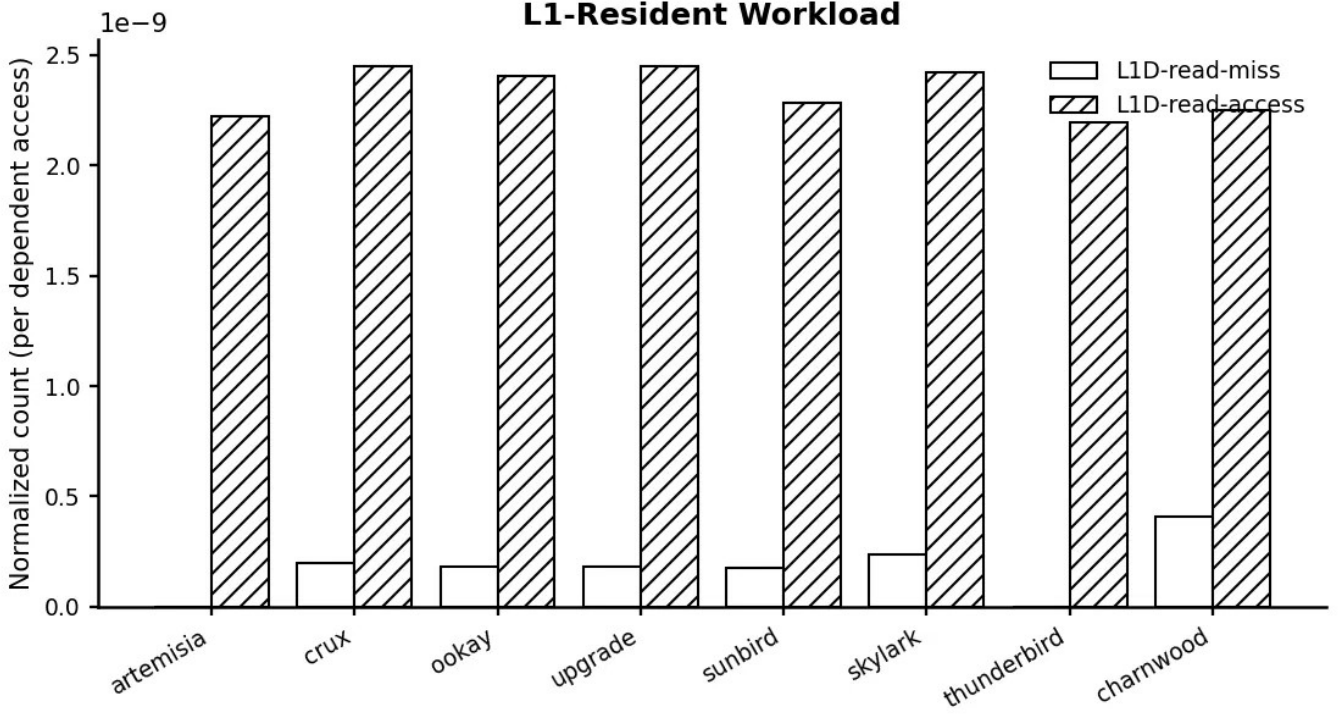


Fig. 20: Cross-machine, cross-regime normalized counter comparison. *Primary contributor: Justin Kuethe.*

TABLE XXVII: Team-member identification and GitHub accounts. *Contributors: Hong Chan Lee and Justin Kuethe.*

Member Name	Unity ID	GitHub Username
Hong Chan Lee	hlee58	hlee706
Justin Kuethe	jrkuethe	NovaCorz

TABLE XXVIII: Result/data-point attribution for reported results. *Contributors: Hong Chan Lee and Justin Kuethe.*

ID	Result, Figure, Table, Machine, or Data Series	Hong Chan Lee Contribution	Justin Kuethe Contribution	Raw Data / Script / Commit Reference
R1	L1 cache capacity and hierarchy measurements	Collected measurements and analyzed cache transitions	Reviewed measurements and verified cache-boundary interpretation	cache_bench.c; 1f687d7
R2	L2 cache capacity and hierarchy measurements	Collected and processed experimental data	Validated results and checked analysis	followup_bench.c; 1f687d7
R3	LLC cache capacity measurements	Ran experiments and analyzed latency transitions	Analyzed data and verified inferred LLC boundary	followup_bench.c; 1f687d7
R4	Cache line-size / stride measurements	Implemented and ran stride experiments	Analyzed results and verified line-size inference	followup_bench.c; 1f687d7
R5	Cache associativity / conflict measurements	Implemented and ran associativity experiments	Analyzed conflict transitions and verified results	followup_worker.py; 1f687d7
R6	Cache hit-latency measurements	Collected timing data and performed initial analysis	Analyzed distributions and verified latency classes	cache_bench.c; 1f687d7
R7	Cache miss / next-level latency measurements	Ran experiments and collected raw data	Analyzed latency distributions and verified transitions	followup_bench.c; 1f687d7
R8	Inclusion/exclusion behavior measurements	Implemented and ran eviction/reload experiments	Analyzed results and evaluated cache behavior	followup_worker.py; 1f687d7
R9	Phase II PMU verification	Compared PMU results against Phase I timing measurements	Collected PMU measurements and documented available events	main_code/pmu; f0b43d2, 1f2b902
R10	Cross-machine cache comparison	Collected/processed measurements across assigned machines	Compared architectures and generations	inferred-cache-table.csv; 62d3d2b
R11	Chronological cache trend analysis and prediction	Performed trend analysis and generated predictions	Reviewed model and validated interpretation	phase3/prediction; 62d3d2b
R12	Hazel HPC validation	Ran/processed held-out Hazel measurements	Compared measured results against frozen predictions	phase3/hazel; 62d3d2b

TABLE XXIX: Code, script, build, and infrastructure contributions. *Contributors: Hong Chan Lee and Justin Kuethe.*

Code Component / File / Module	Hong Chan Lee Contribution	Justin Kuethe Contribution	Git Reference
Phase-I timing suite	Wrote timing benchmarks, ECE acquisition controllers, analysis, plotting, validation, and reproducibility code	—	d41d0aa, 1f687d7
Phase-II PMU suite	—	Wrote PMU acquisition, stride/associativity verification, processing, and aggregation code	f0b43d2, 1f2b902
Phase-III Hazel and prediction suite	Wrote Slurm acquisition, analysis, prediction, comparison, plotting, and verification code	—	62d3d2b
Software estimator and competition package	Wrote the timing estimator, fixed-parameter runner, classifier adapter, and release checks	—	1f687d7, 62d3d2b
Repository and document infrastructure	—	Created the GitHub repository and initial file structure, report template, Overleaf project, and presentation slides	c2e77ee-d8161f1

TABLE XXX: Experimental-running contribution by machine or generation. *Contributors: Hong Chan Lee and Justin Kuethe.*

Experiment / Machine / Generation	Run Date(s)	Hong Chan Lee Contribution	Justin Kuethe Contribution
Phase I: eight ECE machines	Sept. 7–9, 2026	Ran the timing-only suite and retained retries and idle-core evidence	—
Phase II: eight ECE machines	Sept. 9–14, 2026	—	Ran the PMU boundary, eight-counter, stride, and associativity experiments
Phase III: five Hazel generations	Sept. 11–13, 2026	Submitted and monitored Slurm jobs and processed the Hazel measurements	—

TABLE XXXI: Plotting, table, and report-writing contributions. *Contributors: Hong Chan Lee and Justin Kuethe.*

Figure / Table / Report Section	Hong Chan Lee Contribution	Justin Kuethe Contribution
Phase-I figures, tables, and sections	Generated figures and tables and drafted and revised the Phase-I sections	Reviewed and analyzed the Phase-I results listed in Table XXVIII
Phase-II figures, tables, and sections	Integrated timing-to-PMU comparisons with the Phase-I results	Generated Phase-II PMU figures and tables and drafted and revised the Phase-II sections
Phase-III figures, tables, and sections	Generated Hazel, prediction, and evolution figures and tables and drafted and revised the Phase-III sections	Reviewed the prediction model and Hazel comparison
Final integration and revision	Integrated Phase-I, Phase-III, and competition material	Created the report template, Overleaf project, and presentation slides; integrated Phase-II material and cross-machine interpretation

TABLE XXXII: Analysis and interpretation contributions. *Contributors: Hong Chan Lee and Justin Kuethe.*

Analysis Question / Conclusion	Hong Chan Lee Contribution	Justin Kuethe Contribution
Timing-only cache inference and uncertainty	Identified timing classes and boundaries and retained material uncertainty	Reviewed cache-boundary, spatial, conflict, and latency interpretations
PMU verification and event interpretation	Compared PMU results with Phase-I timing inferences	Selected and documented events and analyzed PMU boundary, stride, conflict, and workload results
Chronological trends and frozen prediction	Built the ECE-only chronology, selected models, and froze the predictions	Reviewed the model and interpretation
Hazel comparison, cache laws, and future forecast	Compared held-out Hazel results, evaluated the cache laws, and produced the 2029 forecast	Compared Hazel measurements with the frozen predictions

TABLE XXXIII: AI/LLM assistance disclosure. *Contributors: Hong Chan Lee and Justin Kuethe.*

Tool	Task / Artifact	Type of Help Received	What the Team Verified and Can Explain
Codex	Benchmark workflow, acquisition automation, validation, analysis, plotting, and report drafting/layout	Implementation, automation, analysis, and writing assistance	debugging, verification, Hong reviewed and can explain the Phase-I and Phase-III code, acquisition, analysis, plots, and report material; Justin reviewed and can explain the Phase-II PMU code, data, figures, and report material

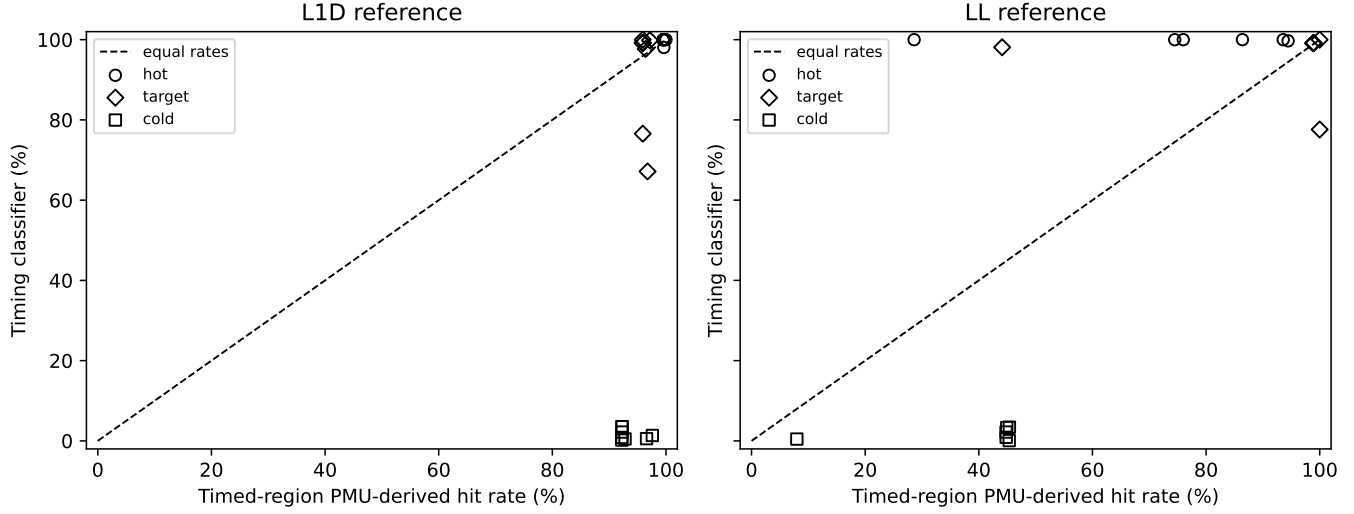


Fig. 21: All available matched comparisons at 1 KiB (hot), 1 MiB (target) and 256 MiB (cold). The dashed line denotes equal percentages, not an assumed equivalence of the classifier and counter populations. Unavailable LL pairs have no plotted value. *Contributors: Hong Chan Lee and Justin Kuethe.*

TABLE XXXIV: Final contribution-appendix confirmation by both team members. *Contributors: Hong Chan Lee and Justin Kuethe.*

Member Name	Confirmation / Date
Hong Chan Lee	Reviewed and confirmed — September 14, 2026
Justin Kuethe	Reviewed and confirmed — September 14, 2026

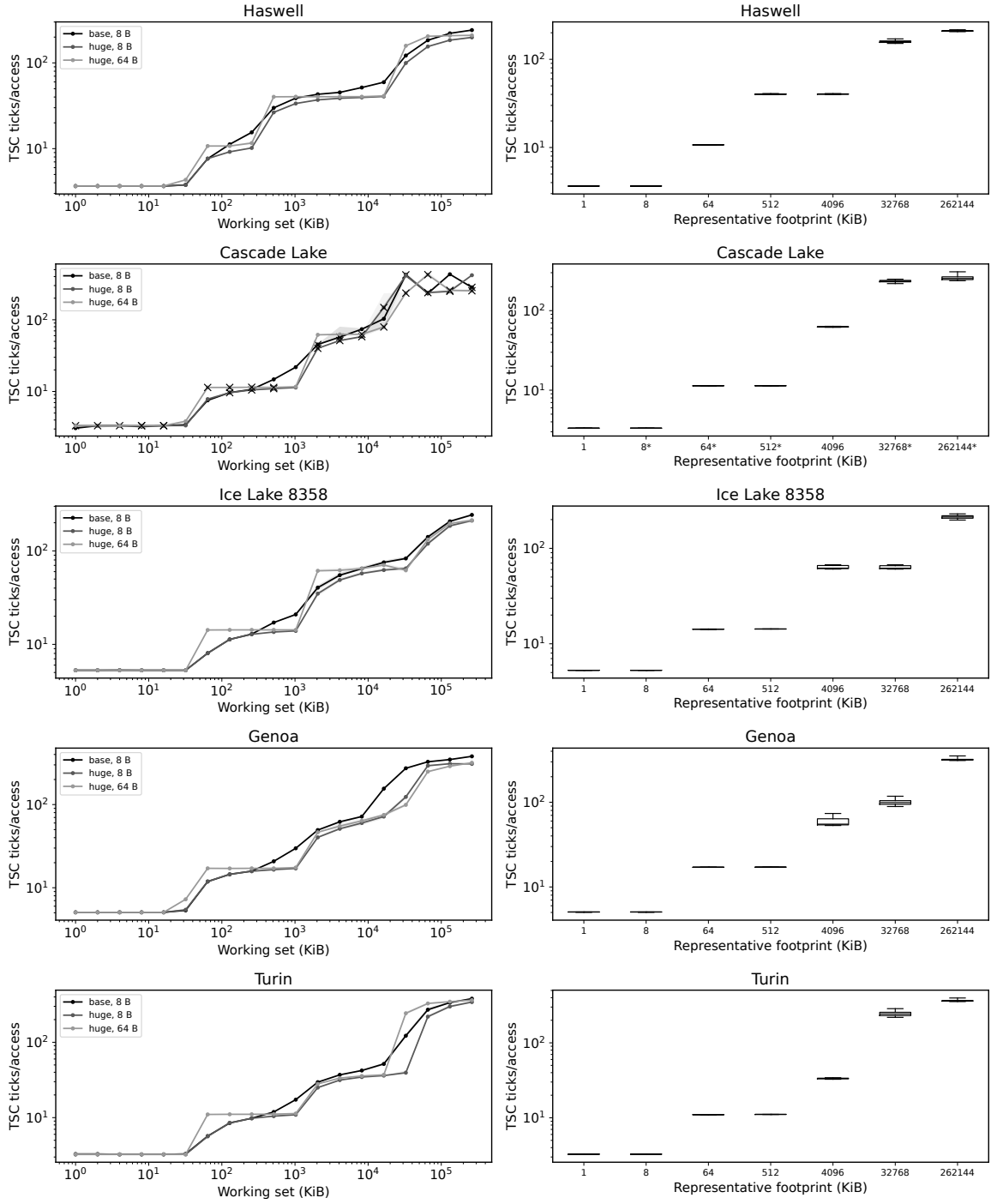


Fig. 22: Hazel capacity and latency evidence, one row per generation. Left: dependent-access medians and IQRs across page-backing/spacing controls. Right: distributions at fixed footprints spanning the sweep; boxes show median/IQR with P5–P95 whiskers. These are empirical latency classes, not independently isolated physical service levels. Crosses and asterisks identify noisy selected points; outliers remain in the data. *Primary contributor: Hong Chan Lee.*

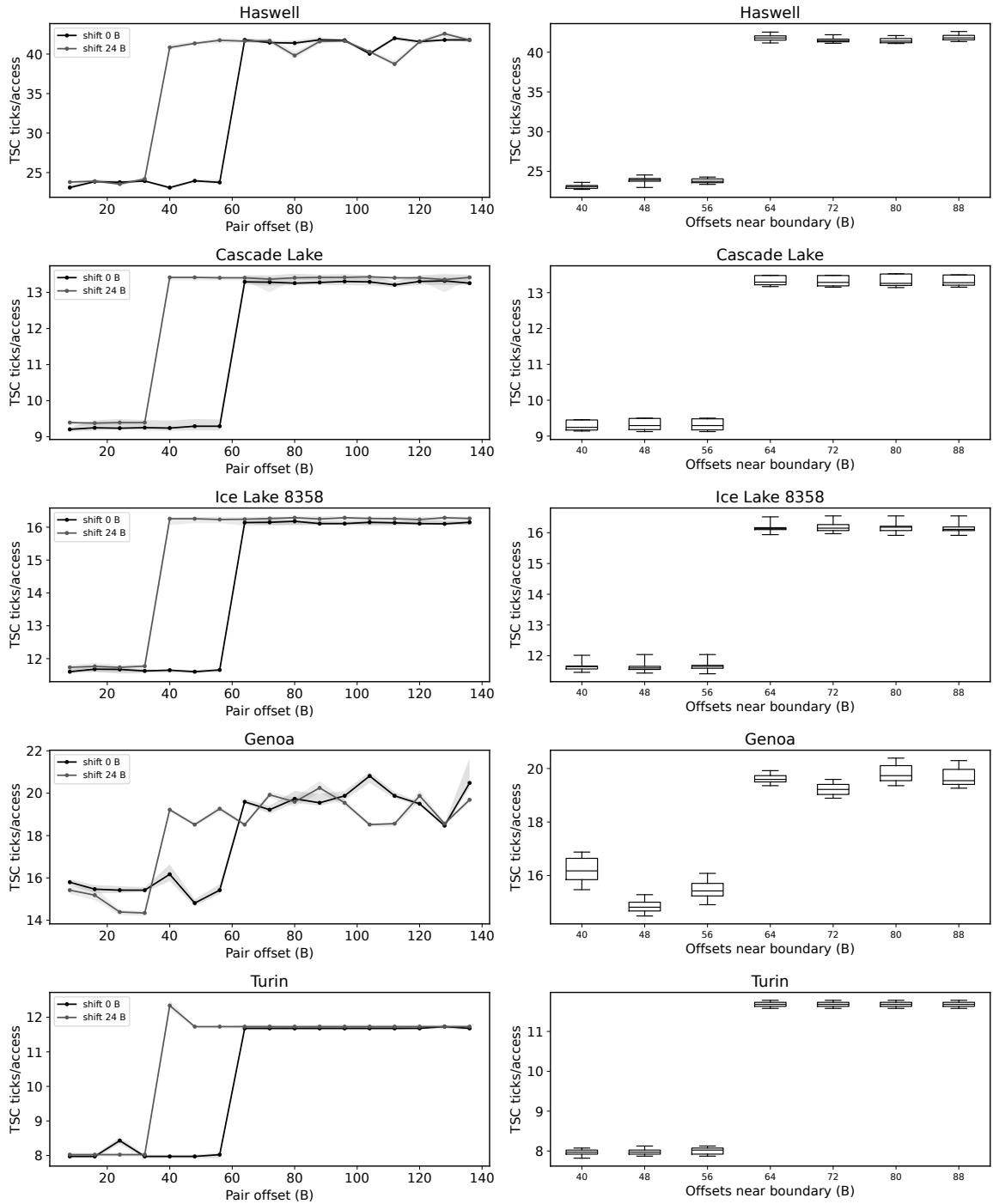


Fig. 23: Timing-visible spatial boundaries under shifted allocation bases (left) and distributions around the smallest boundary (right). Boxes use the smallest tested base shift; all shifts remain in the curves. The common 64-byte granularity supports the scoped spatial law, not separate line-size identification at every cache level. *Primary contributor: Hong Chan Lee.*

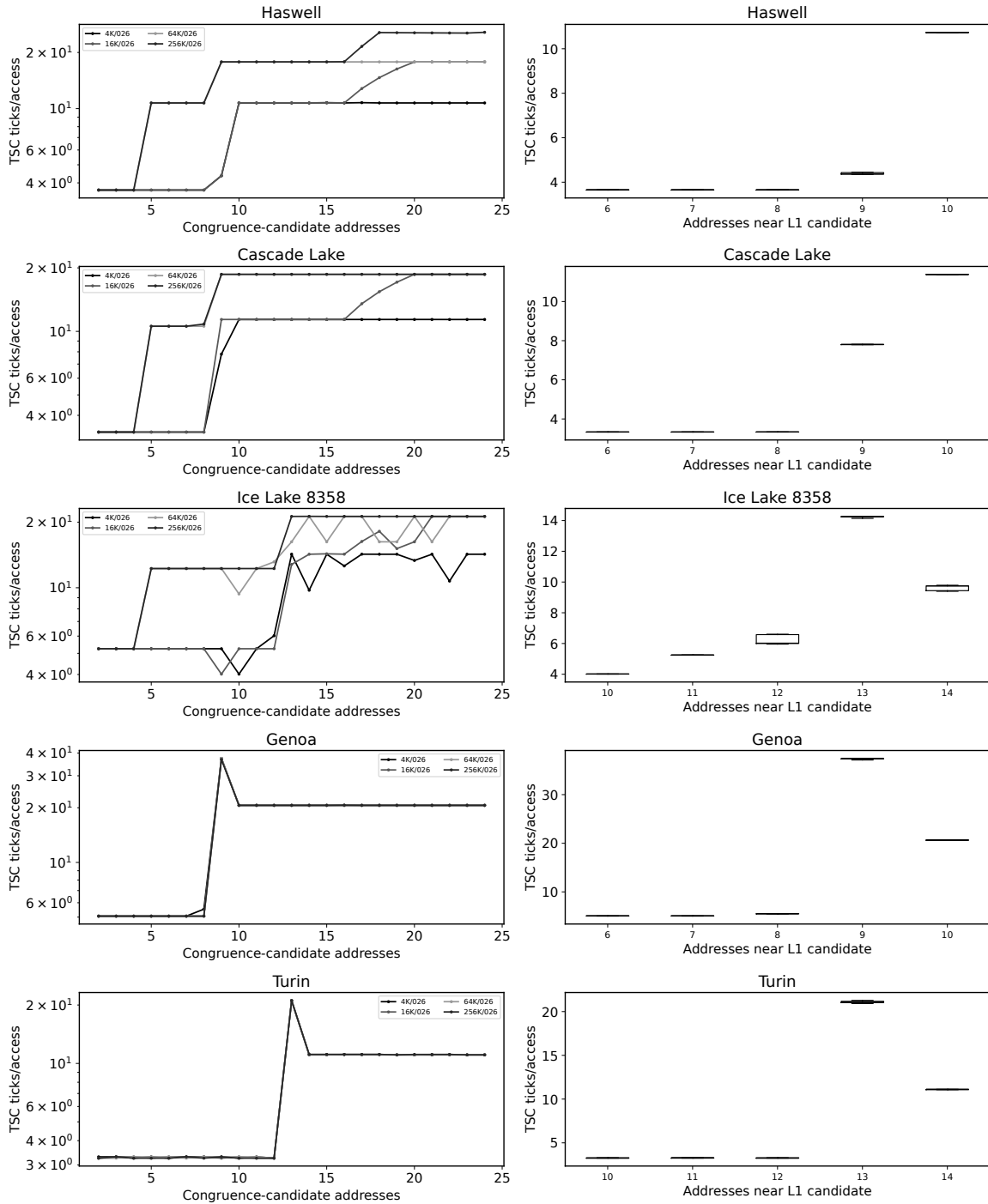


Fig. 24: Congruence-candidate address sweeps at multiple strides/seeds (left) and distributions around each L1 way candidate at the smallest shown stride (right). The capacity cross-check is necessary: an arbitrary conflict threshold alone does not identify associativity. *Primary contributor: Hong Chan Lee.*

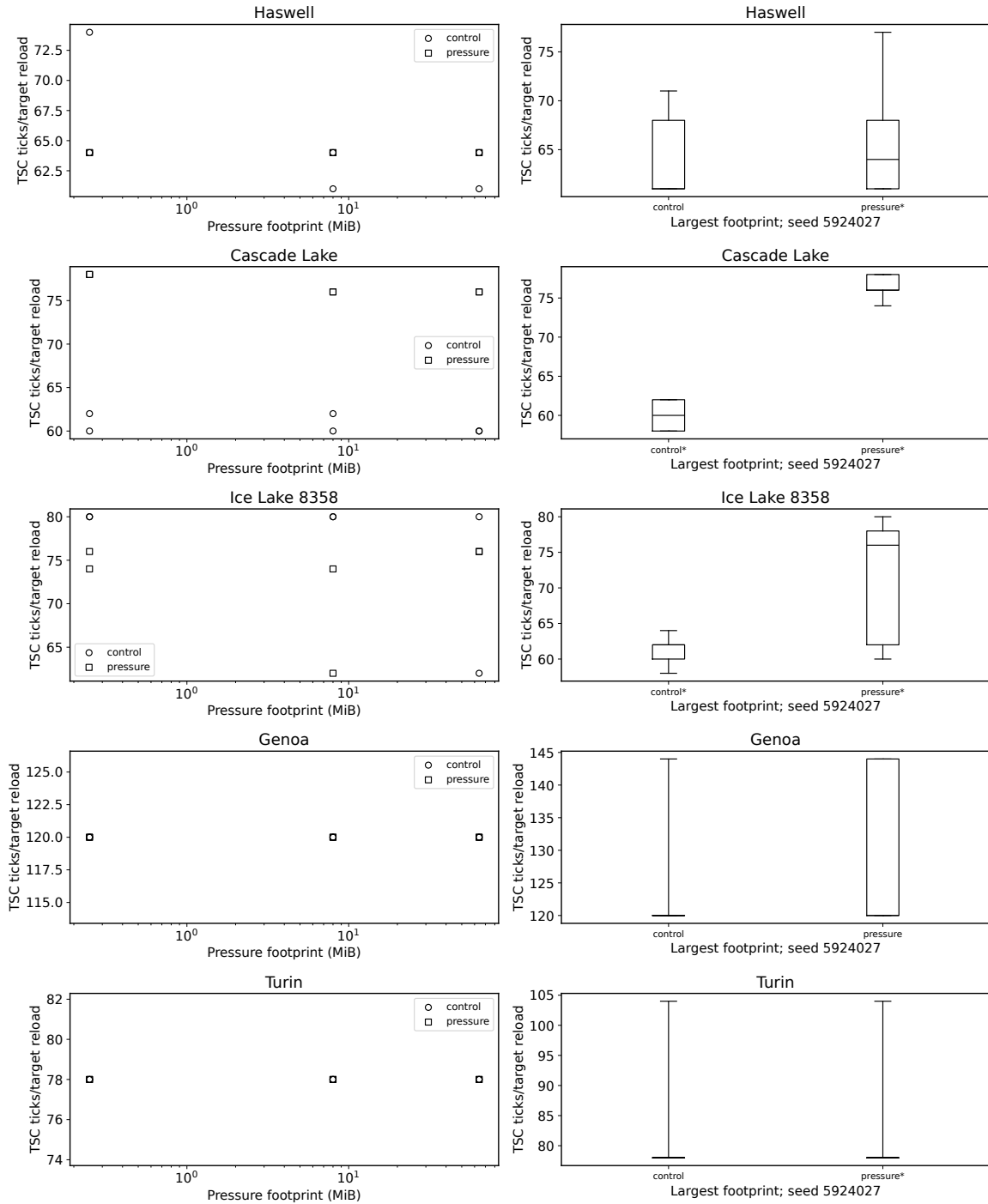


Fig. 25: Remote-pressure controls and target-reload medians (left); largest tested pressure footprint at the first seed (right). Differences demonstrate behavioral sensitivity to pressure but do not prove a global inclusive, exclusive or non-inclusive policy. Quantile-box conventions are as in Fig. 22. *Primary contributor: Hong Chan Lee.*

haswell | haswell constraint; exact allocated model recorded separately | x86_64

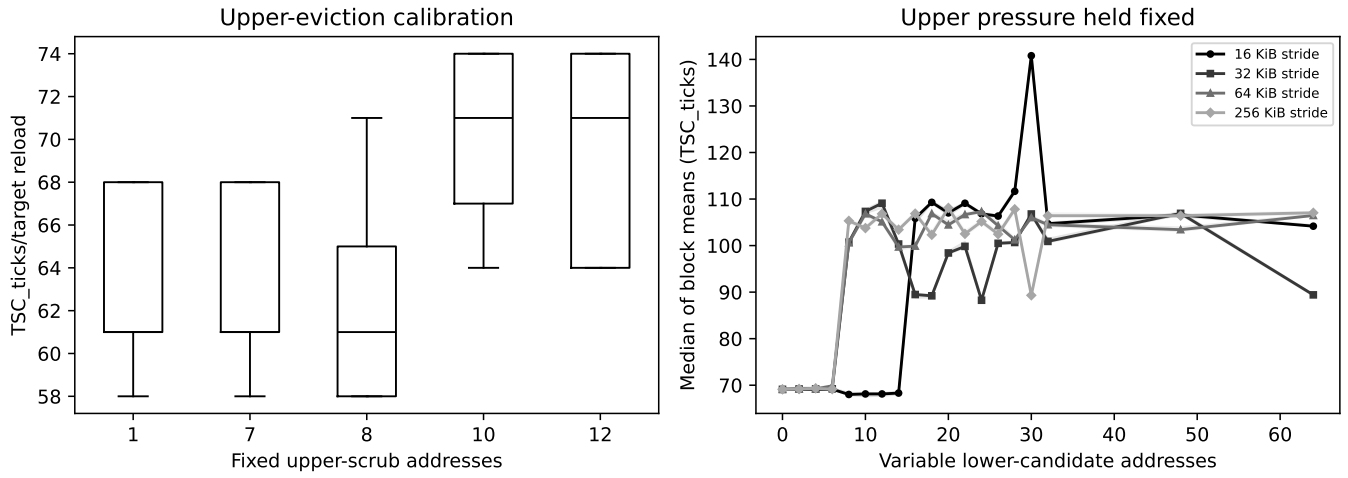


Fig. 26: Haswell lower-level associativity controls. Upper-scrub calibration and lower-conflict pressure are the saved verified evidence underlying the conditional eight-way L2-like result. The accompanying source records retain stride, scrub, seed and noise qualifications; no corresponding way value is inferred for the other four Hazel targets. *Primary contributor: Hong Chan Lee.*

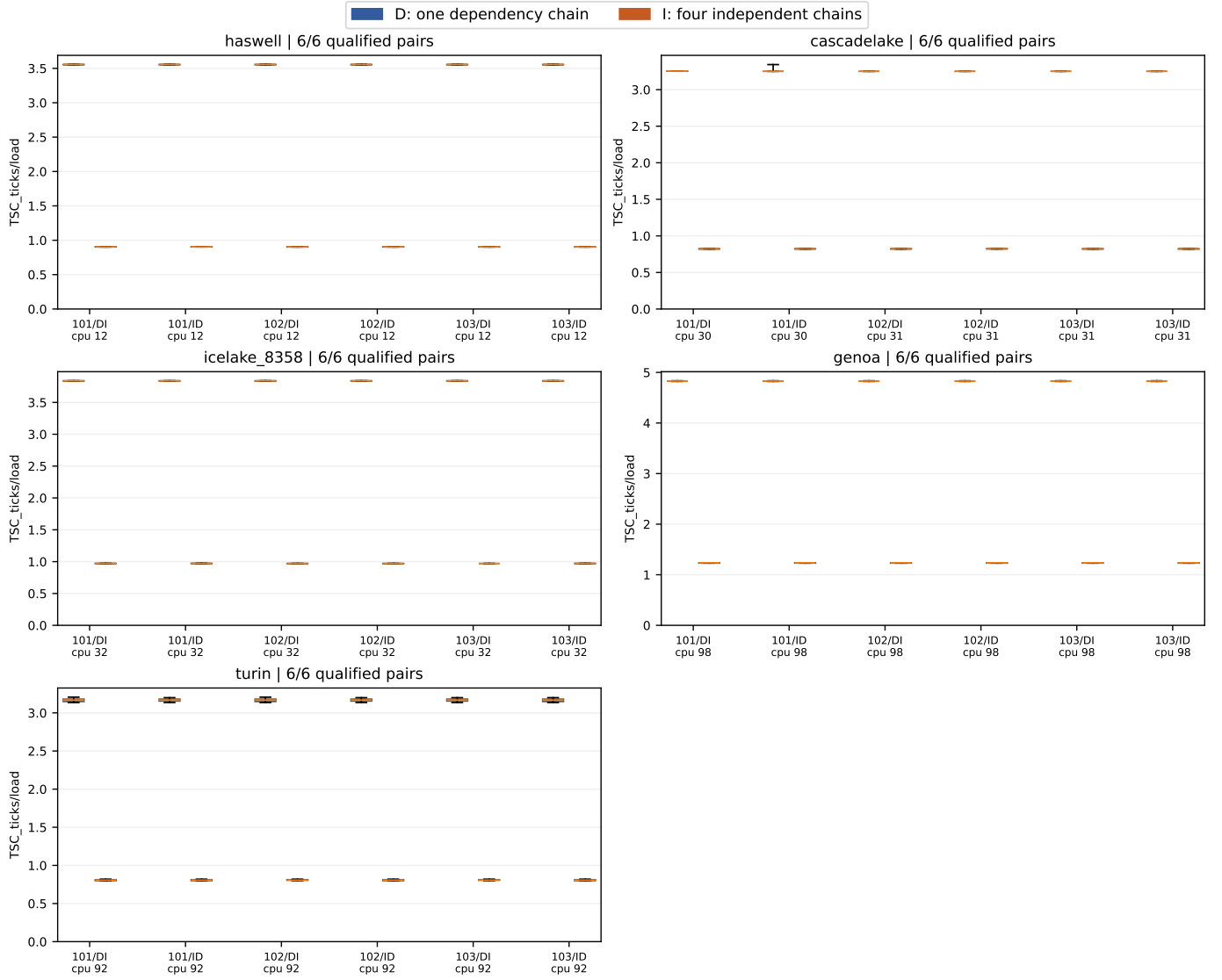


Fig. 27: Independent-load diagnostic distributions across the five Hazel generations. Each pair compares one dependency chain with four independent chains at matched settings. Boxes display median/IQR and P5–P95; these distributions compare throughput and are not physical-cache latency measurements. *Primary contributor: Hong Chan Lee.*

ECE observations and unchanged predictions with Hazel measurements

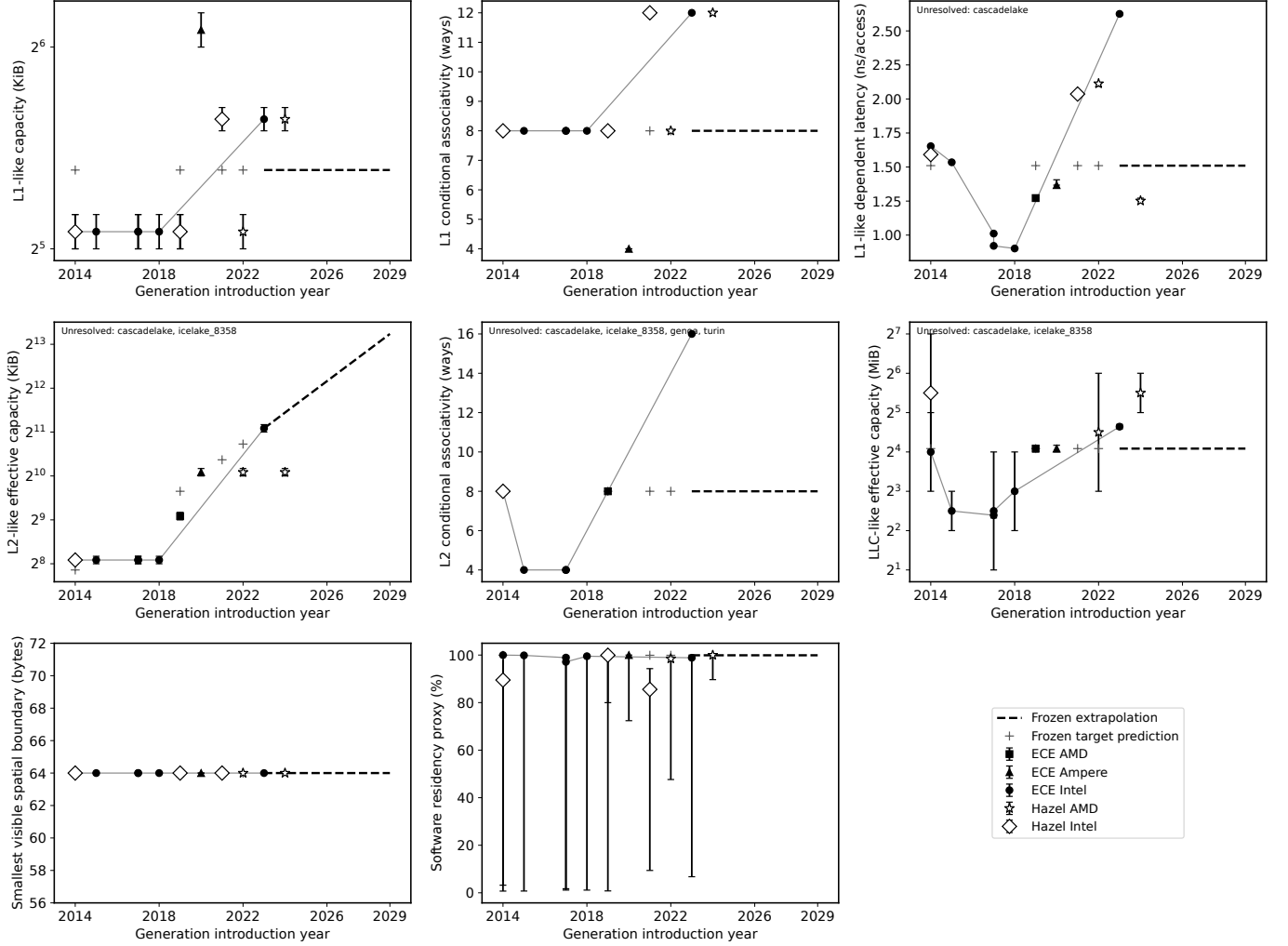


Fig. 28: Chronological observations and unchanged forecasts for supported quantities. Capacity axes are logarithmic; latency is ns/access converted using each run's empirical timer rate, never nominal GHz. Bars show sweep brackets, latency P5–P95 or threshold sensitivity, not interchangeable confidence intervals. Historical target predictions are plus signs; Hazel uses open diamonds/stars. Detailed machine labels and exact values are in the master CSV. *Primary contributor: Hong Chan Lee.*

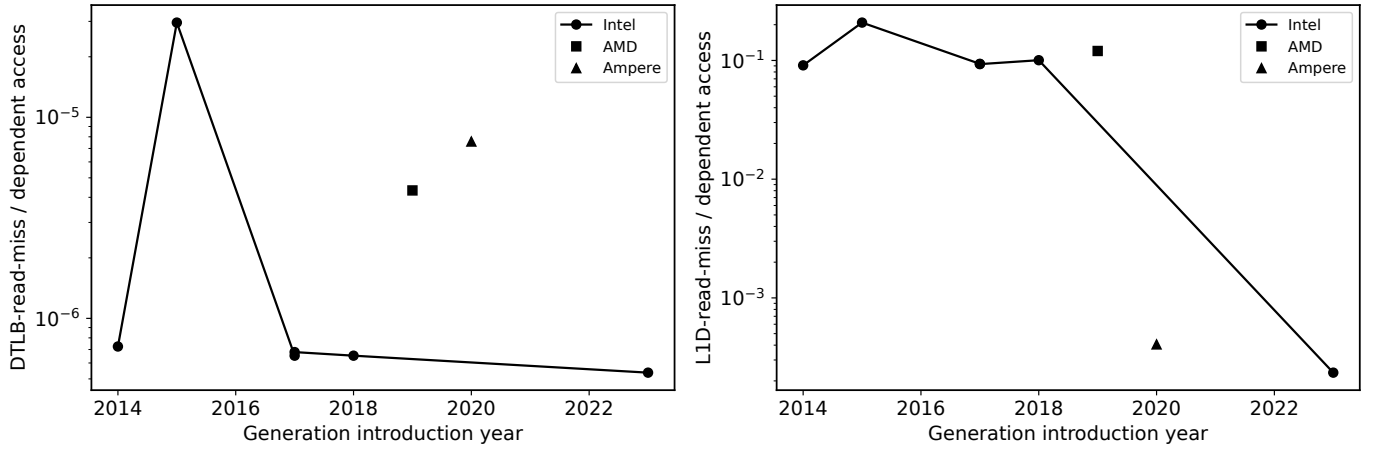


Fig. 29: Two chronological PMU series from the partner’s committed Phase-II L1-resident results (Git `f0b43d2`). The same event abstractions and denominator are used across machines; vendor markers distinguish implementations whose raw event populations may differ. No Hazel PMUs or prediction refitting are involved. *Primary contributor: Hong Chan Lee.*

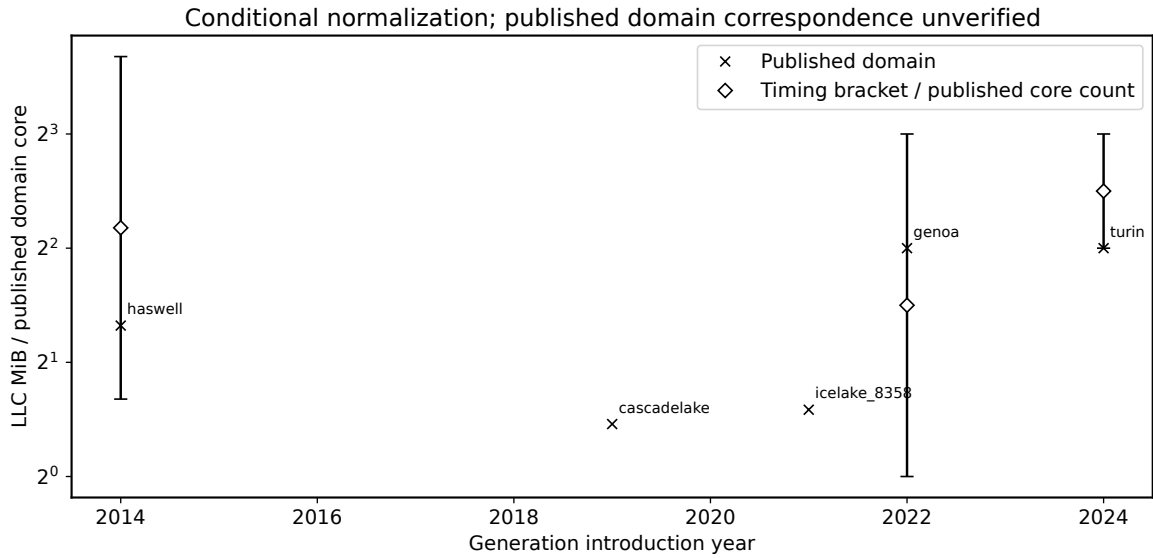


Fig. 30: LLC normalization after published-domain lookup. Crosses are published MiB/domain-core; diamonds and bounds divide timing brackets by that same domain core count. The normalization is conditional and does not establish the sharing domain experimentally. *Primary contributor: Hong Chan Lee.*

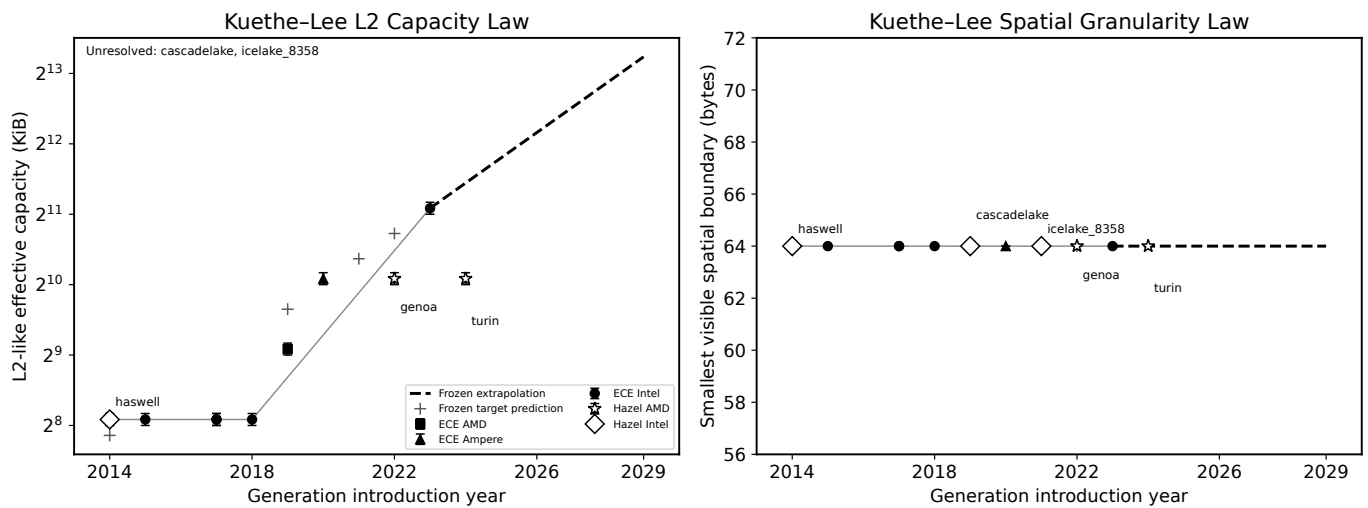


Fig. 31: The two unchanged ECE-only laws tested against Hazel. The capacity law's rising forecast misses Turin; the constant spatial boundary survives all five tests. Filled markers are lab observations; open markers are Hazel; dashed portions are the saved extrapolations. *Primary contributor: Hong Chan Lee.*

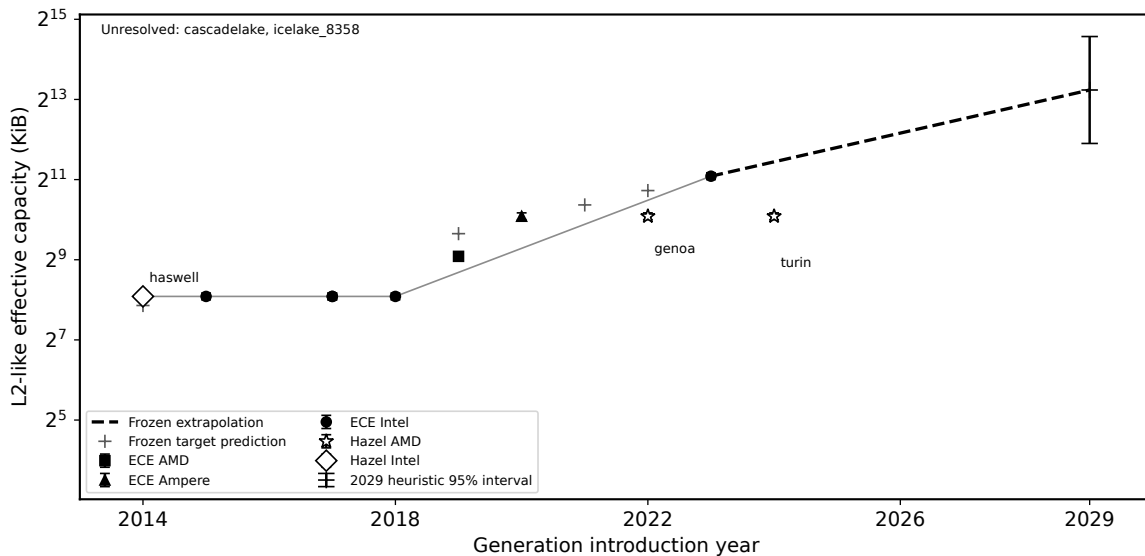


Fig. 32: Original 2029 quantitative forecast with its heuristic model interval. At least three ECE training observations remain visible; later Hazel data evaluate rather than refit the dashed prediction. *Primary contributor: Hong Chan Lee.*

Contributors

Contributions per week to main, excluding merge commits

Period: Last 3 months

Contributions: Commits

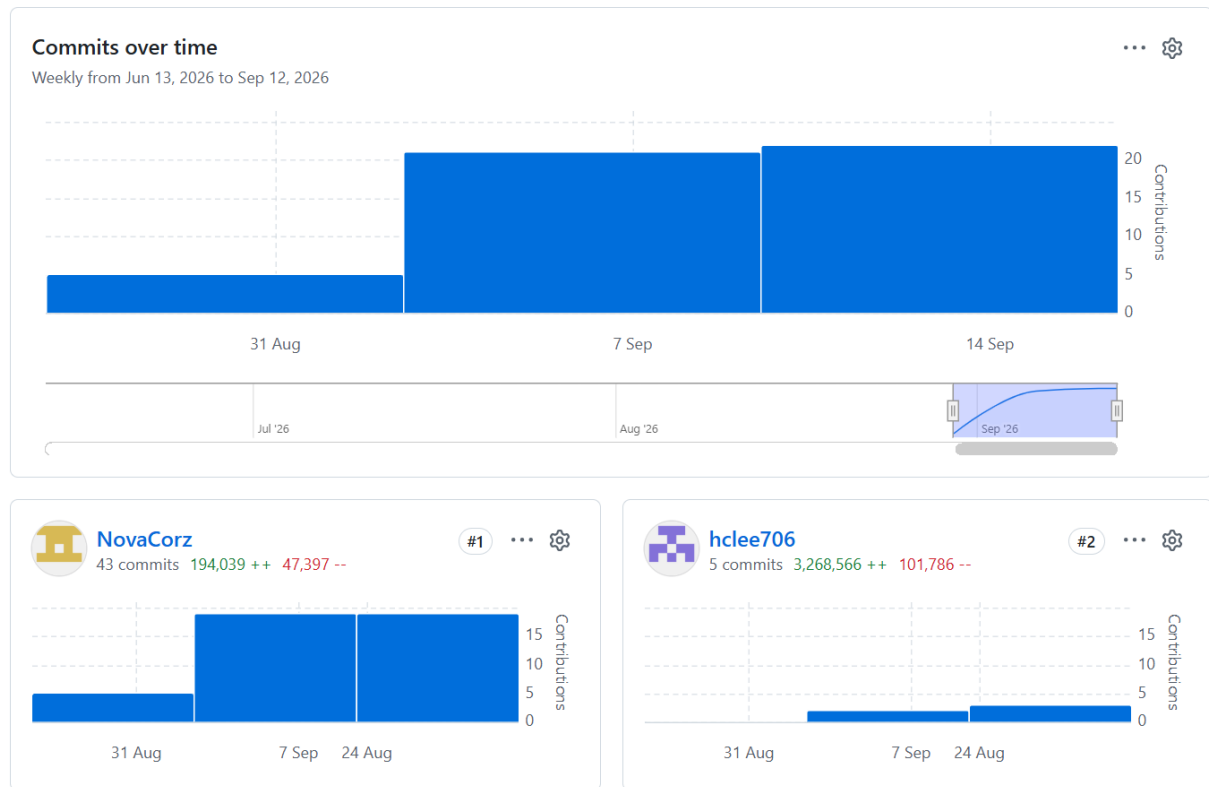


Fig. 33: GitHub contribution history. Hong Chan Lee corresponds to `hclee706`; Justin Kuethe corresponds to `NovaCorz`.
Contributors: Hong Chan Lee and Justin Kuethe.

The competition code/results snapshot is Git commit 62d3d2b1b8539303772e2783c575cd0972df3335, tag `competition-v1`, committed 2026-09-14 at 03:24:02 UTC. Estimator source hashes and parameters were frozen at 03:22:58 UTC that day in `competition/freeze.json`. The snapshot includes the Phase-III package, the matched estimator/PMU validation and the scorecard data. Build/run instructions are in `competition/README.md`; the exact release reference and raw-data handoff locations are recorded in `COMPETITION_HANDOFF.md`.

Table XXXV lists 48 ECE and 22 Hazel candidates with timing values preserved. Yes* recommends a qualified inference corroborated by post-freeze literature or PMU evidence; it does not claim an exact physical capacity. Hold identifies an unresolved measurement/domain or validation gap; No identifies a published-capacity disagreement. L1/L2 roles and ways remain conditional, including core-family comparisons to related SKUs. The stride PMU evidence is broad and the generic LL events do not isolate L2 conflict behavior. L1 and LLC PMU classifications are in Tables X and XI; L2 counters have the coverage gap described in Section VIII-F.

Published LLC sizes fall outside Thunderbird, Artemisia and Ookay’s brackets. Broad containment on other hosts does not establish the physical sharing domain, so those LLC candidates remain on hold. Crux and Thunderbird’s mixed-ring L2 way matches likewise do not resolve residency. Upgrade’s refined departure is not a second discovery. No derived-set, uncertain inclusion, unsupported deeper-latency or duplicate spatial result is added. Per-row source locators, comparison scope and decision reasons are retained in `report/scorecard/validation.csv`.

References

- [1] NC State ECE, *Project 1*, supplied file PROJECT 1 (3).pdf, 35 pages, especially §5, §8.1–8.5, §10–13. Requirements source only; no published cache descriptions were used as experimental answers.
- [2] Intel, “Third Generation Intel Xeon Processor Scalable Family on Two Socket Platforms,” Table 1, <https://www.intel.com/content/www/us/en/developer/articles/technical/third-generation-xeon-scalable-family-overview.html>.
- [3] Intel, “Xeon Gold 6226R Processor Specifications,” <https://www.intel.com/content/www/us/en/products/sku/199347/intel-xeon-gold-6226r-processor-22m-cache-2-90-ghz/specifications.html>.
- [4] Intel, “Products formerly Ice Lake,” Xeon Platinum 8358 row, <https://www.intel.com/content/www/us/en/ark/products/codename/74979/products-formerly-ice-lake.html>.
- [5] Intel, “Xeon E5 v3 Family,” E5-2650 v3 row, <https://www.intel.com/content/www/us/en/ark/products/series/78583/intel-xeon-processor-e5-v3-family.html>.
- [6] AMD, *EPYC 9004 BIOS and Workload Tuning Guide*, 58011 rev. 1.4, §2.5–2.8, pp. 4–6, https://docs.amd.com/api/khub/documents/goX~9ubv8i5r60A_Qrp3Rw/content.
- [7] AMD, *5th Gen EPYC Processor Architecture*, 70353 rev. B, pp. 5–8, <https://docs.amd.com/api/khub/documents/UIqhAbjRhgnzgzzdVU4pUw/content>.
- [8] A. Fog, *The Microarchitecture of Intel, AMD, and VIA CPUs*, May 23, 2026, Tables 10.2, 12.3 and 25.1 (core geometry; SKU-dependent LLC values not substituted), <https://www.agner.org/optimize/microarchitecture.pdf>.
- [9] Intel, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, vol.1, 248966-049US, §2.1 and Table 2-14 (p.2-32), <https://cdrdv2-public.intel.com/814198/248966-Optimization-Reference-Manual-V1-049.pdf>.
- [10] A. Abel and J. Reineke, “nanoBench Cache Analyzer: Caches,” Core i7-7700 and i7-8700K entries, <https://uops.info/cache.html>.
- [11] Ampere Computing, *Altra 64-Bit Multi-Core Processor Datasheet*, issue 1.30, §2.3–2.5, p.9, https://amperecomputing.com/assets/Altra_Rev_A1_DS_v1_30_20220728_8170025756.pdf.
- [12] AMD, *HPC Tuning Guide for EPYC 7002 ROME Series Processors*, 56827, cache-domain diagram, <https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/tuning-guides/amd-epyc-7002-tg-hpc-56827.pdf>.
- [13] Intel, “Fourth Generation Xeon Scalable Family Overview,” Table 1, <https://www.intel.com/content/www/us/en/developer/articles/technical/fourth-generation-xeon-scalable-family-overview.html>.
- [14] Intel, “Xeon Gold 5420+ Processor Specifications,” <https://www.intel.com/content/www/us/en/products/sku/232381/intel-xeon-gold-5420-processor-52-5m-cache-2-00-ghz/specifications.html>.
- [15] Intel, “Xeon E5-2680 v3 Processor Specifications,” Essentials and Supplemental Information, <https://www.intel.com/content/www/us/en/products/sku/81908/intel-xeon-processor-e52680-v3-30m-cache-2-50-ghz/specifications.html>.
- [16] Intel, *Core i7 Desktop Processor Comparison Chart*, p.1, i7-6700, i7-7700 and i7-8700 rows, <https://www.intel.com/content/dam/support/us/en/documents/processors/intel-core-i7-comparison-chart.pdf>.
- [17] Intel, “Core i7-9700 Processor Specifications,” CPU Specifications, <https://www.intel.com/content/www/us/en/products/sku/191792/intel-core-i79700-processor-12m-cache-up-to-4-70-ghz/specifications.html>.
- [18] Intel, “Core i7-6700 Processor Specifications,” Essentials and Supplemental Information, <https://www.intel.com/content/www/us/en/products/sku/88196/intel-core-i76700-processor-8m-cache-up-to-4-00-ghz/specifications.html>.
- [19] Intel, “Core i7-7700 Processor Specifications,” Essentials and Supplemental Information, <https://www.intel.com/content/www/us/en/products/sku/97128/intel-core-i77700-processor-8m-cache-up-to-4-20-ghz/specifications.html>.
- [20] Intel, “Core i7-8700 Processor Specifications,” Essentials and Supplemental Information, <https://www.intel.com/content/www/us/en/products/sku/126686/intel-core-i78700-processor-12m-cache-up-to-4-60-ghz/specifications.html>.
- [21] Intel, *8th and 9th Generation Intel Core Processor Family Specification Update*, doc. 337346-002, title page (9th Generation formerly Coffee Lake Refresh), Nov. 2019, <https://www.intel.com/content/dam/www/public/us/en/documents/specification-updates/8th-gen-core-spec-update.pdf>.
- [22] Intel, “Intel Announces World’s Best Gaming Processor: New 9th Gen Intel Core i9-9900K,” Intel Newsroom archive entry, Oct. 8, 2018, <https://newsroom.intel.com/new-technologies/intel-newsroom-archive-2018>.
- [23] AMD, “AMD EPYC 7532 Drivers and Downloads,” General Specifications (EPYC 7002 Series), <https://www.amd.com/en/support/downloads/drivers.html/processors/epyc/epyc-7002-series/amd-epyc-7532.html>.
- [24] AMD, “2nd Gen AMD EPYC Processors Set New Standard for the Modern Datacenter,” Aug. 7, 2019 (7002 family with Zen 2 cores), <https://ir.amd.com/news-events/press-releases/detail/904>.
- [25] Ampere Computing, “Ampere Altra—Industry’s First 80-Core Server Processor Unveiled,” Mar. 3, 2020 (Altra Q80 platform based on Arm Neoverse N1), <https://amperecomputing.com/press/ampere-altra-industrys-first-80-core-server-processor-unveiled>.

TABLE XXXV: ECE and Hazel reverse-engineering scorecard (70 candidates). Yes*: qualified recommendation; Hold: unresolved validation/interpretation; No: published size outside the timing bracket. Capacity containment is weaker than exact identification. Blank identities continue the preceding row. Source locators: Intel [9], Table 2-14; Fog [8], Tables 10.2, 11.2, 12.3, 13.1, 22.3, 25.1; other citations identify the exact SKU row, cache section or generation overview. *Contributors: Hong Chan Lee and Justin Kuethe.*

Machine / CPU	Generation	Candidate property	Timing value	Fig.	Published reference	Validation result	Count?
ECE lab machines (CPU identities in Table II)							
sunbird	Haswell	L1-like capacity	32–36 KiB	3	32 KiB [8]	inside bracket; PMU clean	Yes*
		L1 cond. ways	8	10	8 ways [8]	matches	Yes*
		L2-like capacity	256–288 KiB	3	256 KiB [8]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	8	11	8 ways [8]	matches; L2 PMU gap	Yes*
		LLC-like bracket	8–32 MiB	3	30 MiB [15]	inside; domain unproven	Hold
		Visible granularity	64 B	8	64 B [8]	matches	Yes*
thunderbird	Neoverse N1	L1-like capacity	64–72 KiB	3	64 KiB [11]	inside bracket; PMU clean	Yes*
		L1 cond. ways	4	10	4 ways [11]	matches	Yes*
		L2-like capacity	1024–1152 KiB	3	1024 KiB [11]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	8	11	8 ways [11]	matches; mixed-ring only	Hold
		LLC-like bracket	16–18 MiB	3	32 MiB [11]	outside bracket	No
		Visible granularity	64 B	8	64 B [11]	matches	Yes*
skylark	Zen 2	L1-like capacity	32–36 KiB	3	32 KiB [8]	inside bracket; PMU clean	Yes*
		L1 cond. ways	8	10	8 ways [8]	matches	Yes*
		L2-like capacity	512–576 KiB	3	512 KiB [8]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	8	11	8 ways [8]	matches; L2 PMU gap	Yes*
		LLC-like bracket	16–18 MiB	3	16 MiB [12]	inside CCX; scope unproven	Hold
		Visible granularity	64 B	8	64 B [8]	matches	Yes*
artemisia	Sapphire Rapids	L1-like capacity	48–52 KiB	3	48 KiB [13]	inside bracket; PMU clean	Yes*
		L1 cond. ways	12	10	12 ways [8], [9]	matches	Yes*
		L2-like capacity	2048–2304 KiB	3	2048 KiB [13]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	16	11	not established [13]	source lacks way count	Hold
		LLC-like bracket	24–26 MiB	3	52.5 MiB [14]	outside bracket	No
		Visible granularity	64 B	8	64 B [8], [9]	matches	Yes*
charnwood	Skylake	L1-like capacity	32–36 KiB	3	32 KiB [9]	inside bracket; PMU weak	Yes*
		L1 cond. ways	8	10	8 ways [9]	matches	Yes*
		L2-like capacity	256–288 KiB	3	256 KiB [9]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	4	11	4 ways [9]	matches; L2 PMU gap	Yes*
		LLC-like bracket	4–8 MiB	3	8 MiB [16]	inside; domain unproven	Hold
		Visible granularity	64 B	8	64 B [9]	matches	Yes*
crux	Coffee Lake R.	L1-like capacity	32–36 KiB	3	32 KiB [10]	inside bracket; PMU clean	Yes*
		L1 cond. ways	8	10	8 ways [10]	matches	Yes*
		L2-like capacity	256–288 KiB	3	256 KiB [10]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	4	11	4 ways [10]	matches; mixed-ring only	Hold
		LLC-like bracket	4–16 MiB	3	12 MiB [17]	inside; domain unproven	Hold
		Visible granularity	64 B	8	64 B [9]	matches	Yes*
ookay	Kaby Lake	L1-like capacity	32–36 KiB	3	32 KiB [10]	inside bracket; PMU clean	Yes*
		L1 cond. ways	8	10	8 ways [10]	matches	Yes*
		L2-like capacity	256–288 KiB	3	256 KiB [10]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	4	11	4 ways [10]	matches; L2 PMU gap	Yes*
		LLC-like bracket	5–5.5 MiB	3	8 MiB [16]	outside bracket	No
		Visible granularity	64 B	8	64 B [9]	matches	Yes*
upgrade	Coffee Lake	L1-like capacity	32–36 KiB	3	32 KiB [10]	inside bracket; PMU clean	Yes*
		L1 cond. ways	8	10	8 ways [10]	matches	Yes*
		L2-like capacity	256–288 KiB	3	256 KiB [10]	inside bracket; L2 PMU gap	Yes*
		L2 cond. ways	4	11	4 ways [10]	matches; L2 PMU gap	Yes*
		LLC-like bracket	2–16 MiB	3	12 MiB [16]	inside; refined edge differs	Hold
		Visible granularity	64 B	8	64 B [9]	matches	Yes*
Hazel compute-node processors (bindings in Table XX)							
E5-2650 v3	Haswell	L1-like capacity	32–36 KiB	22	32 KiB [8]	inside bracket	Yes*
		L1 cond. ways	8	24	8 ways [8]	matches	Yes*
		L2-like capacity	256–288 KiB	22	256 KiB [8]	inside bracket	Yes*
		L2 cond. ways	8	24	8 ways [8]	matches	Yes*
		LLC-like bracket	16–128 MiB	22	25 MiB [5]	inside; domain unproven	Hold
		Visible granularity	64 B	23	64 B [8]	matches	Yes*
Gold 6226R	Cascade Lake	L1-like capacity	32–36 KiB	22	32 KiB [2]	inside bracket	Yes*
		L1 cond. ways	8	24	8 ways [8]	matches	Yes*
		Visible granularity	64 B	23	64 B [8]	matches	Yes*
Platinum 8358	Ice Lake-SP	L1-like capacity	48–52 KiB	22	48 KiB [2]	inside bracket	Yes*
		L1 cond. ways	12	24	12 ways [8]	matches	Yes*
		Visible granularity	64 B	23	64 B [8]	matches	Yes*
EPYC 9654	Genoa / Zen 4	L1-like capacity	32–36 KiB	22	32 KiB [6]	inside bracket	Yes*
		L1 cond. ways	8	24	8 ways [6]	matches	Yes*
		L2-like capacity	1024–1152 KiB	22	1024 KiB [6]	inside bracket	Yes*
		LLC-like bracket	8–64 MiB	22	32 MiB [6]	inside; domain unproven	Hold
		Visible granularity	64 B	23	64 B [6]	matches	Yes*
EPYC 9655	Turin / Zen 5	L1-like capacity	48–52 KiB	22	48 KiB [7]	inside bracket	Yes*
		L1 cond. ways	12	24	12 ways [8]	matches	Yes*
		L2-like capacity	1024–1152 KiB	22	1024 KiB [7]	inside bracket	Yes*
		LLC-like bracket	32–64 MiB	22	32 MiB [7]	inside; domain unproven	Hold
		Visible granularity	64 B	23	64 B [8]	matches	Yes*